

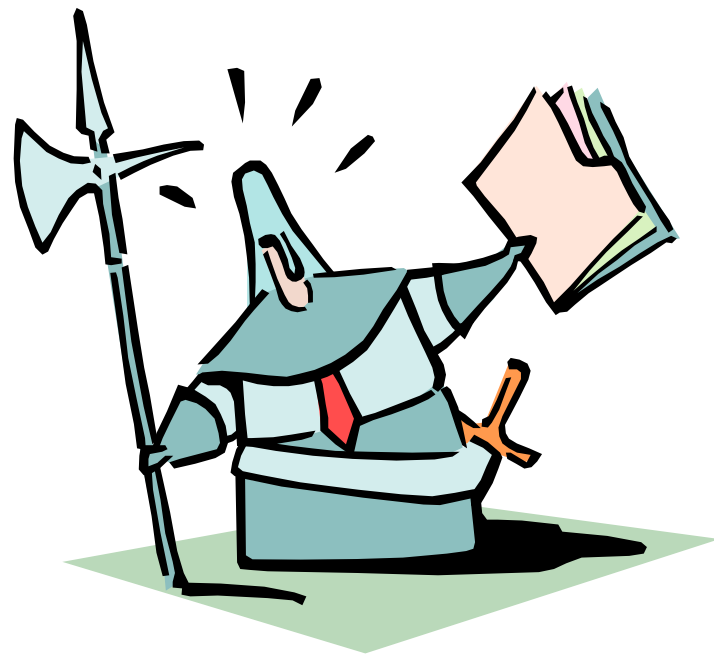


设计之美



Chapter 6 软件设计方法

- 6.1 设计的概念
- 6.2 分治
- 6.3 抽象
- 6.4 内聚和耦合
- 6.5 复用
- 6.6 设计描述方法
- 6.7 人-机界面的设计
- 6.8 相关文档规范





6.1 设计的概念

- 设计的总体原则
 - 不应陷入片面性
 - 追踪分析模型
 - 选择合适的技术
 - 适度分解
 - 可集成
 - 提高抽象层次
 - 可复用
 - 可维护和可扩展的
 - 有韧性
 - 一致性
 - 交互界面应该友好
 - 设计评审



6.1 设计的概念

- 软件构架

- UML:

- 指系统的组织结构，它可以递归解构为通过接口交互的部件、连接部件的关系以及组装部件的一些限制条件，通过接口交互的部件有类、构件和子系统。

- IEEE:

- 系统在其环境中的最高层概念



6.1 设计的概念

- 软件构架

- Mary Shaw和David Garlan

- 关于构成系统的元素、这些元素之间的交互、元素和元素之间的组成模式以及作用在这些组成模式上的约束等方面的描述

- Rational Unified Process

- 指通过接口交互的重要构件的组织或结构，这些构件又由一些更小的构件和接口组成。



6.1 设计的概念

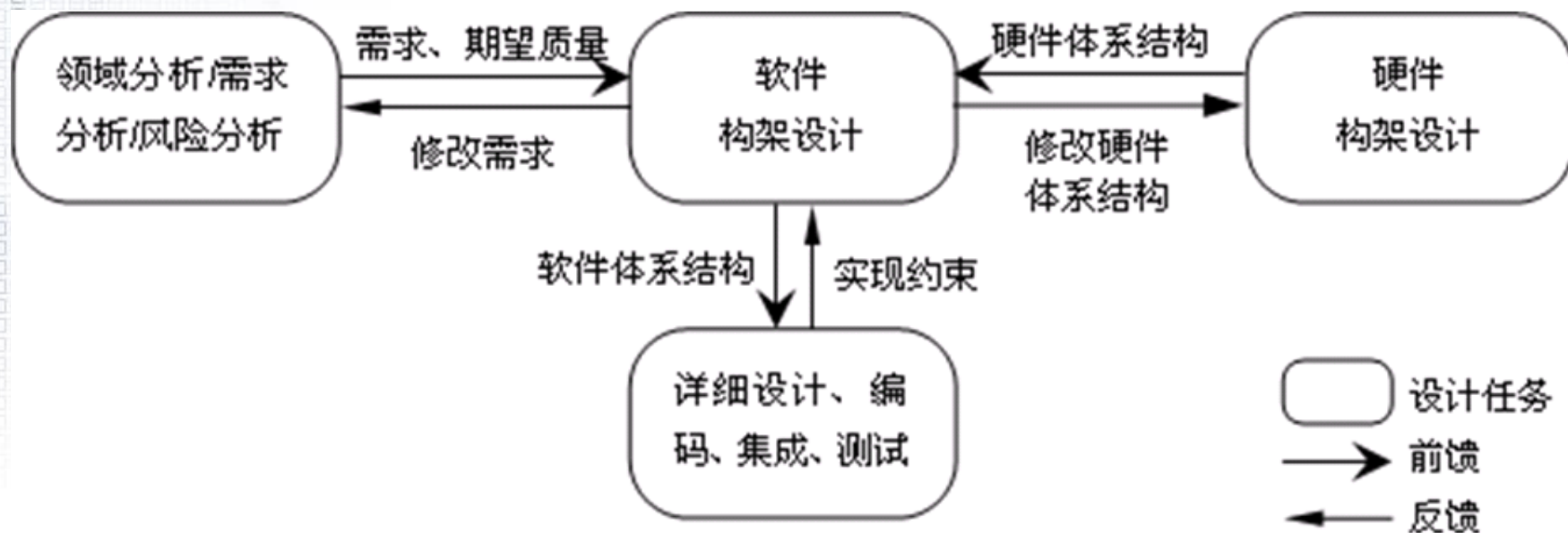
- 软件构架

- 对复杂的系统进行抽象，为系统设计规划蓝图，是关于系统构造及系统各构件工作机制的相对精简、却能清晰反映核心问题的模型，体现了项目早期的设计决策，可以作为相关涉众对系统理解和交流以及进行项目管理的基础，能够帮助开发人员进一步细化和实施系统
- 一个好的软件构架已经成为完成高质量软件的重要保证，它在软件开发中起到核心作用



6.1 设计的概念

- 软件体系结构和其它开发任务的关系





6.1 设计的概念

- 软件架构反映系统整体的组织结构和基本特征
 - 软件的层次结构
 - 模块相互作用的方式
 - 接口
 - 全局的、重要的数据变量和数据结构
 - 数据库的逻辑结构



6.1 设计的概念

- 详细设计
 - 在软件构架的基础上进一步确定如何实现目标系统，具体包括：
 - 系统的模块（如类及其方法）逻辑的详细设计
 - 系统数据结构的设计
 - 系统数据库结构的设计
 - 系统人-机接口的设计等



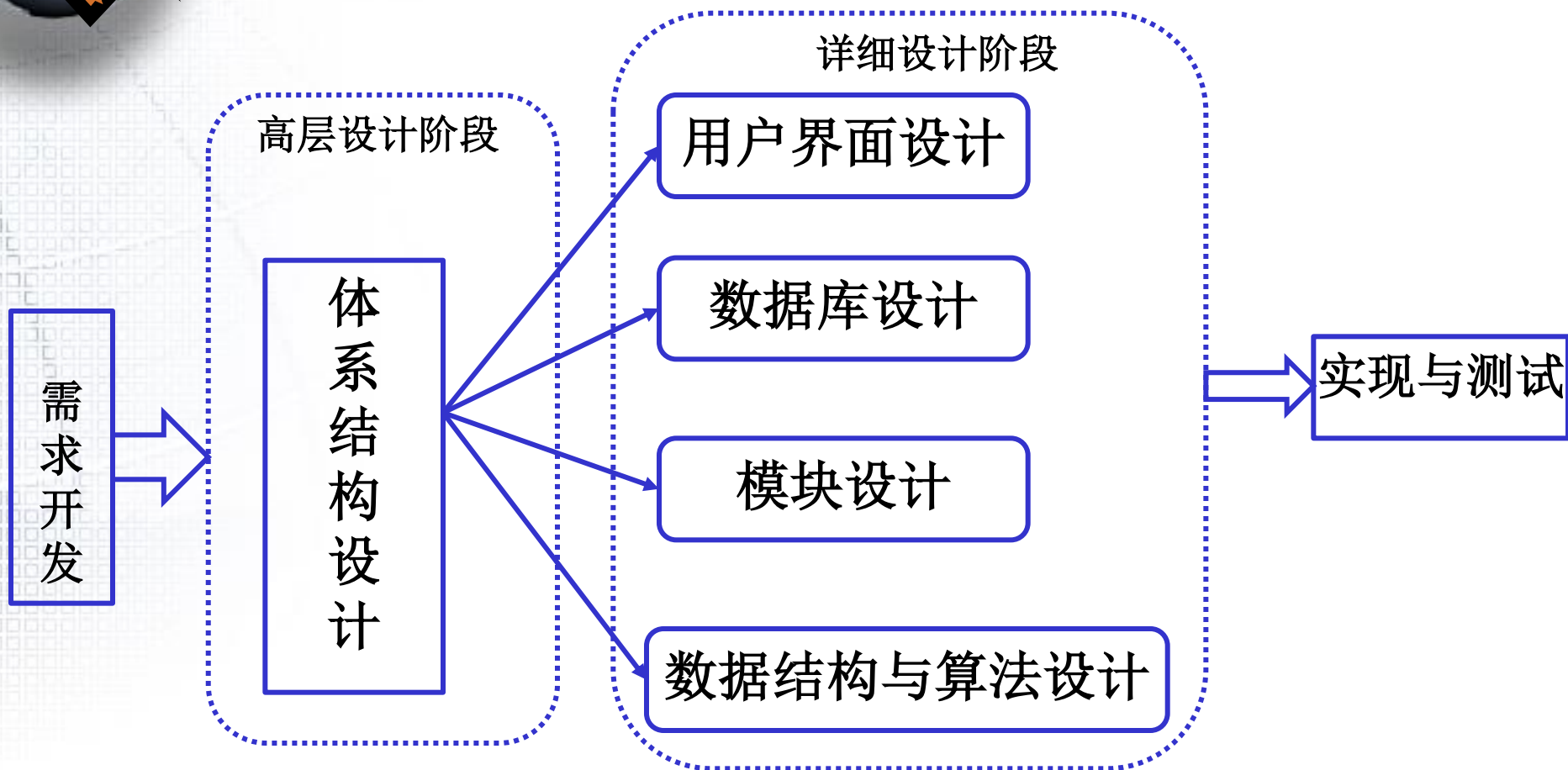
6.2 分治

- 分治思想

- 架构设计->将系统分解为一组子系统或模块，以及子系统或模块之间的接口。
- 详细设计->子系统或模块被再次细分为子模块
 - 传统软件工程中的过程、函数
 - 面向对象软件工程中的包、类、接口和方法等
- 应用：排序算法、二分查找算法等



6.2 分治





6.2 分治

- 分治的优点

- 1) 缩小问题空间，减少问题复杂度和工作量

$$C(p1) > C(p2) \Rightarrow E(p1) > E(p2) \text{ 或}$$

$$C(p1) < C(p2) \Rightarrow E(p1) > E(p2)$$

$$C(p1 + p2) > C(p1) + C(p2)$$

$$E(p1 + p2) > E(p1 ; p2)$$

$$E(p1 ; p2) = E(p1) + E(p2)$$

$$E(p1 + p2) > E(p1) + E(p2)$$

分开考虑
问题的工
作量



6.2 分治

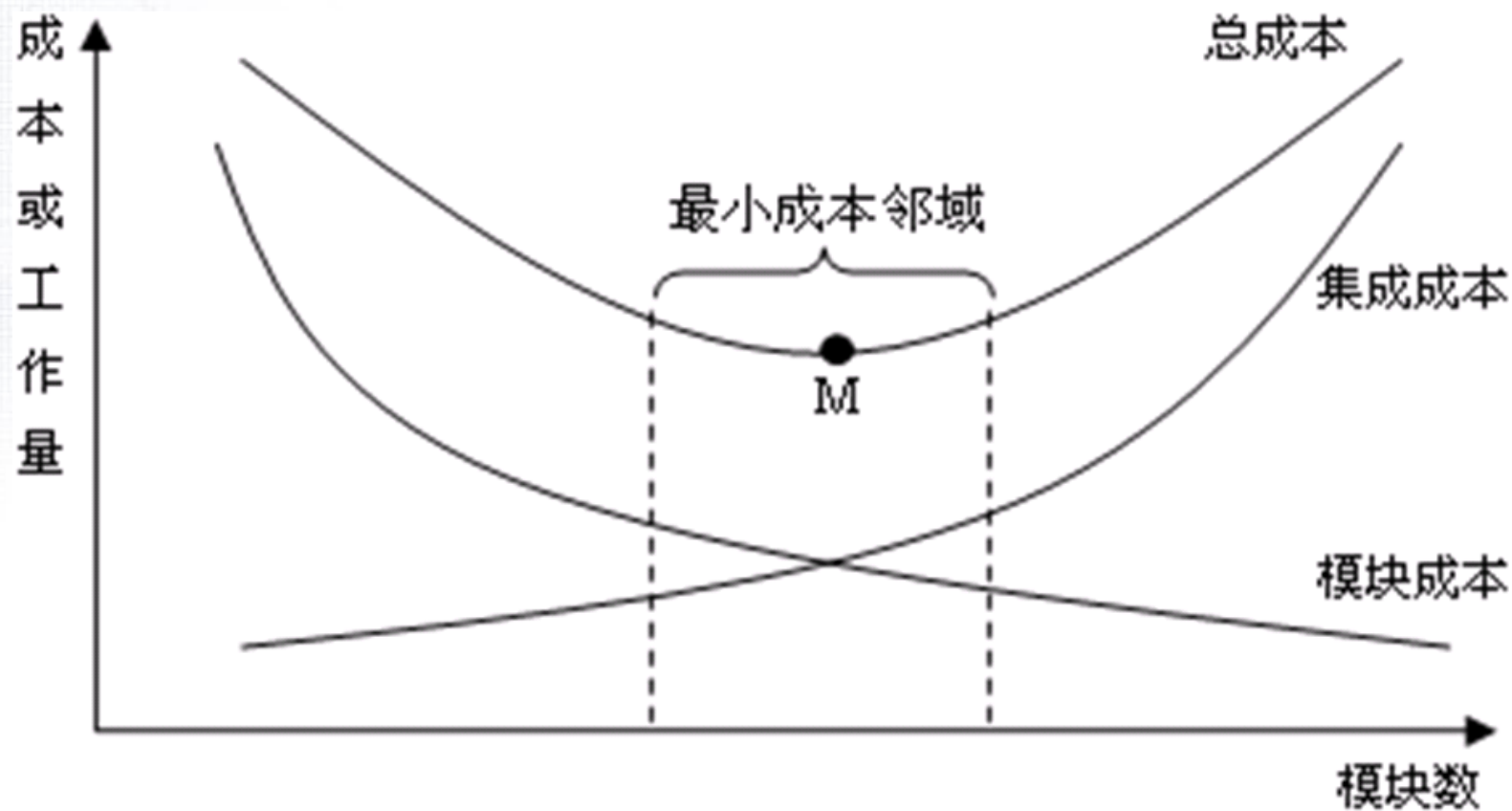
- 分治的优点
 - 2) 便于并发执行，缩短开发周期
 - 3) 适合团队协作，降低了实施难度
 - 4) 预防了开发中的多米诺骨牌效应
 - 5) 容易产生可复用部件





6.2 分治

- 分治要考虑的问题
 - 1) 分治是否可以无限分解，是否越细越好





6.2 分治

- 分治要考虑的问题
 - 2) 从技术的角度讲，如何分治？
 - 程序设计方法
 - 部件重用
 - 可理解性
 - 独立性
 - 有界性



6.2 分治

- 分治要考虑的问题
 - 3) 如何将任务分解到不同的时间域上?
 - 需求的优先级
 - 基础部件
 - 项目风险
 - 任务执行的难易程度



6.3 抽象

- 抽象思想

- 当一个过程不易理解，抽象能更好理解
 - 例读卡器工作原理：初始化、读卡、停止
- 为了降低复杂度，应该隐藏细节或推迟考虑细节

- 抽象的优点

- 有利于认识事物的普遍特征和基本原理
- 帮助设计人员制定出模块的“框架”
- 有利于软件的复用
- 提高系统的的可扩展性



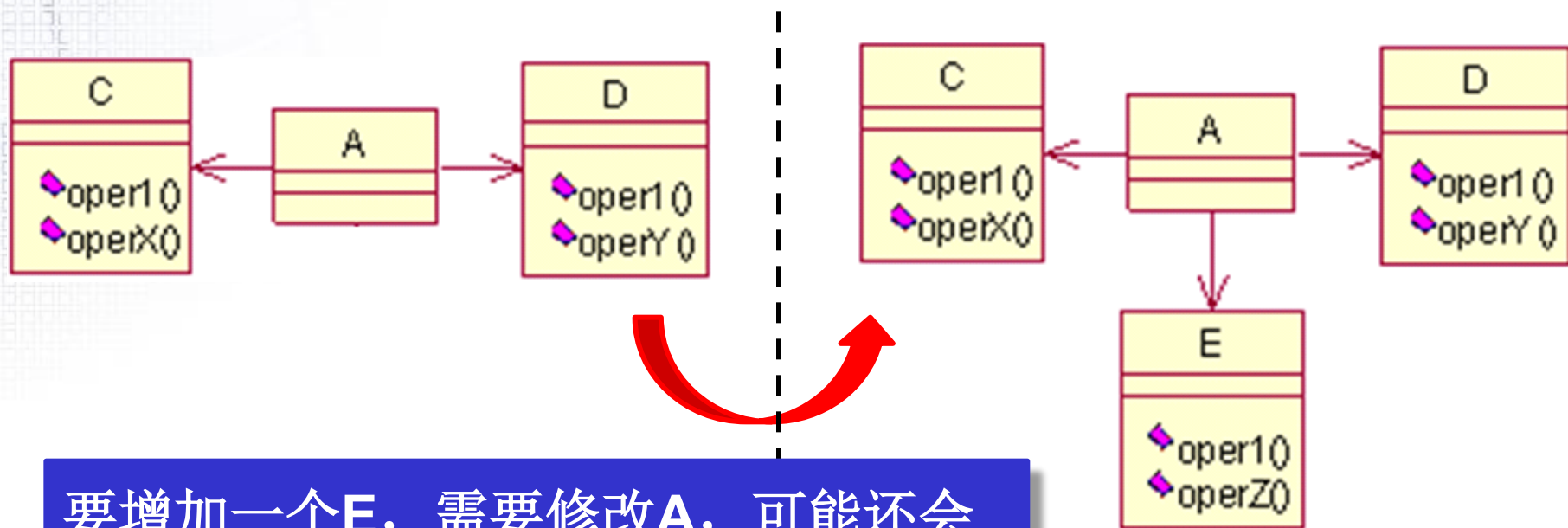
6.3 抽象

- 基于抽象的设计原则
 - 1) 里氏替换原则(Liskov Principle)
 - 子类可以替换父类，可以出现在父类能出现的任何地方
 - 如果对于每一个类型为 T_1 的对象 o_1 ，都有类型为 T_2 的对象 o_2 ，使得以 T_1 定义的所有程序 P 在所有的对象 o_1 都替换成 o_2 时，程序 P 的行为没有变化，那么类型 T_2 是类型 T_1 的子类型。



6.3 抽象

- 基于抽象的设计原则
 - 2) 开闭原则(Open-Closed Principle)
 - 即一个软件实体应当对扩展开放，对修改关闭



要增加一个E，需要修改A，可能还会涉及A与C、A与D的接口的修改

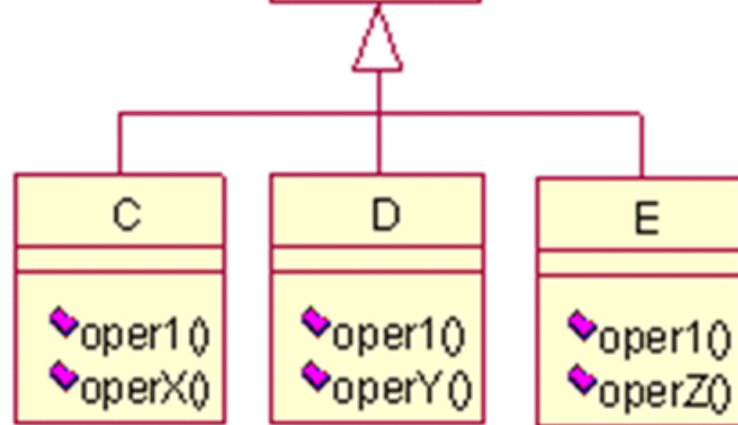
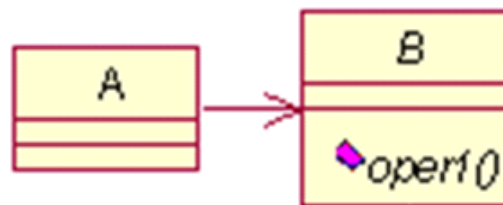
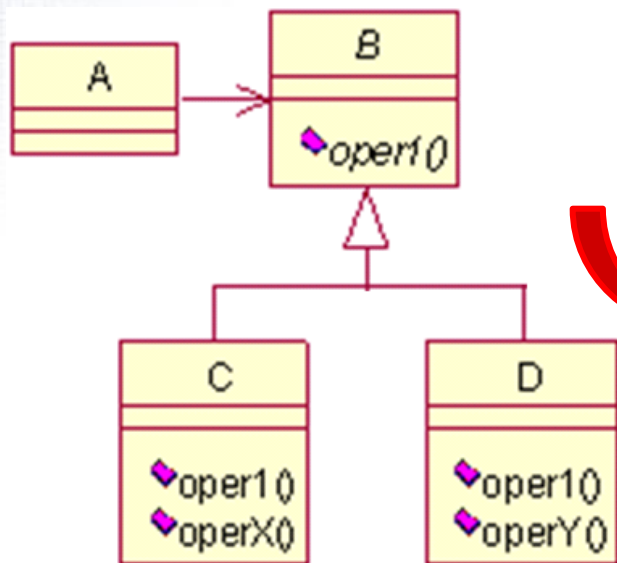
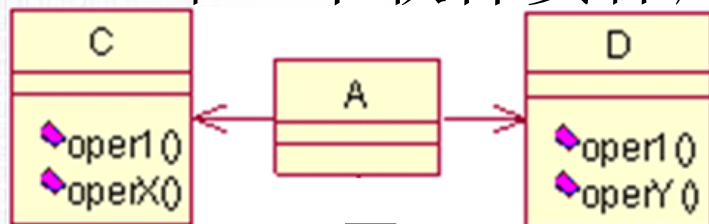


6.3 抽象

- 基于抽象的设计原则

- 2) 开闭原则(Open-Closed Principle) (续)

- 即一个软件实体应当对扩展开放，对修改关闭



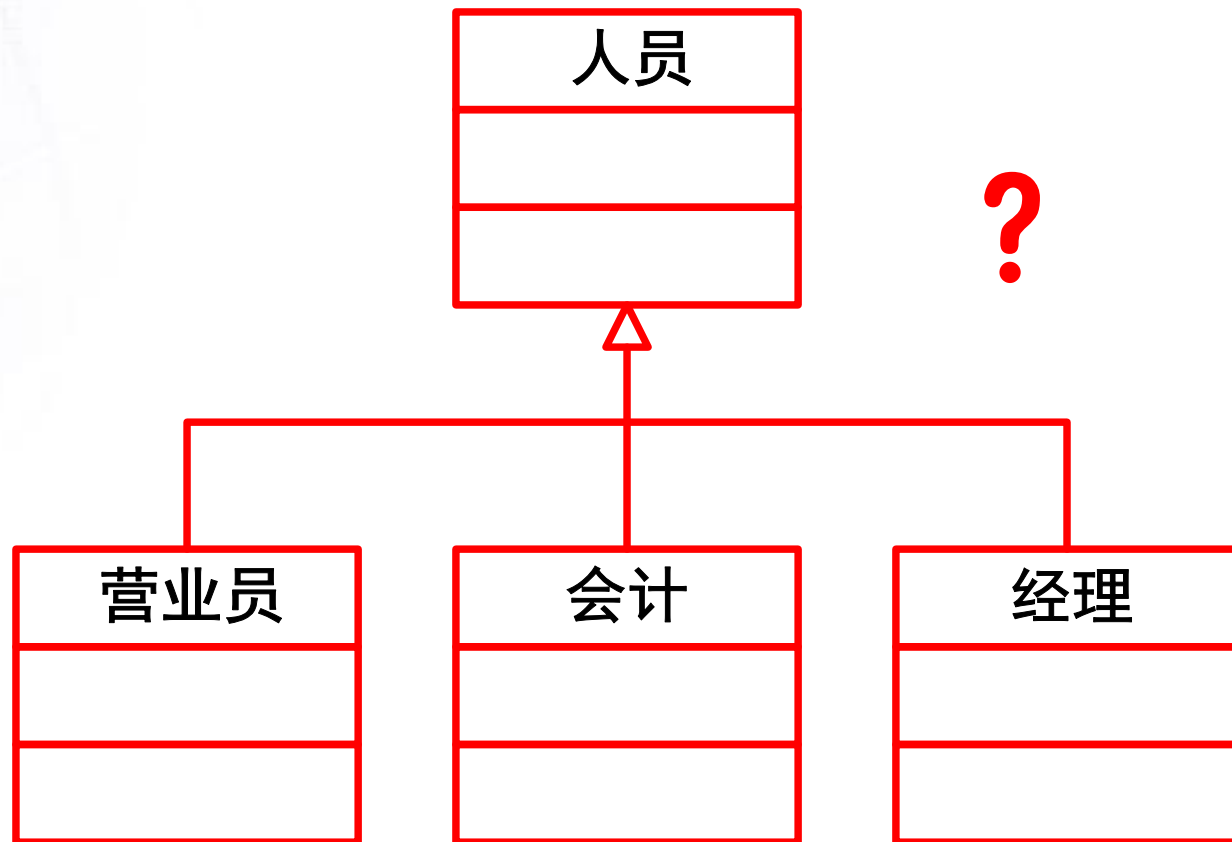
增加了一个E，无修改A



6.3 抽象

- 相关问题

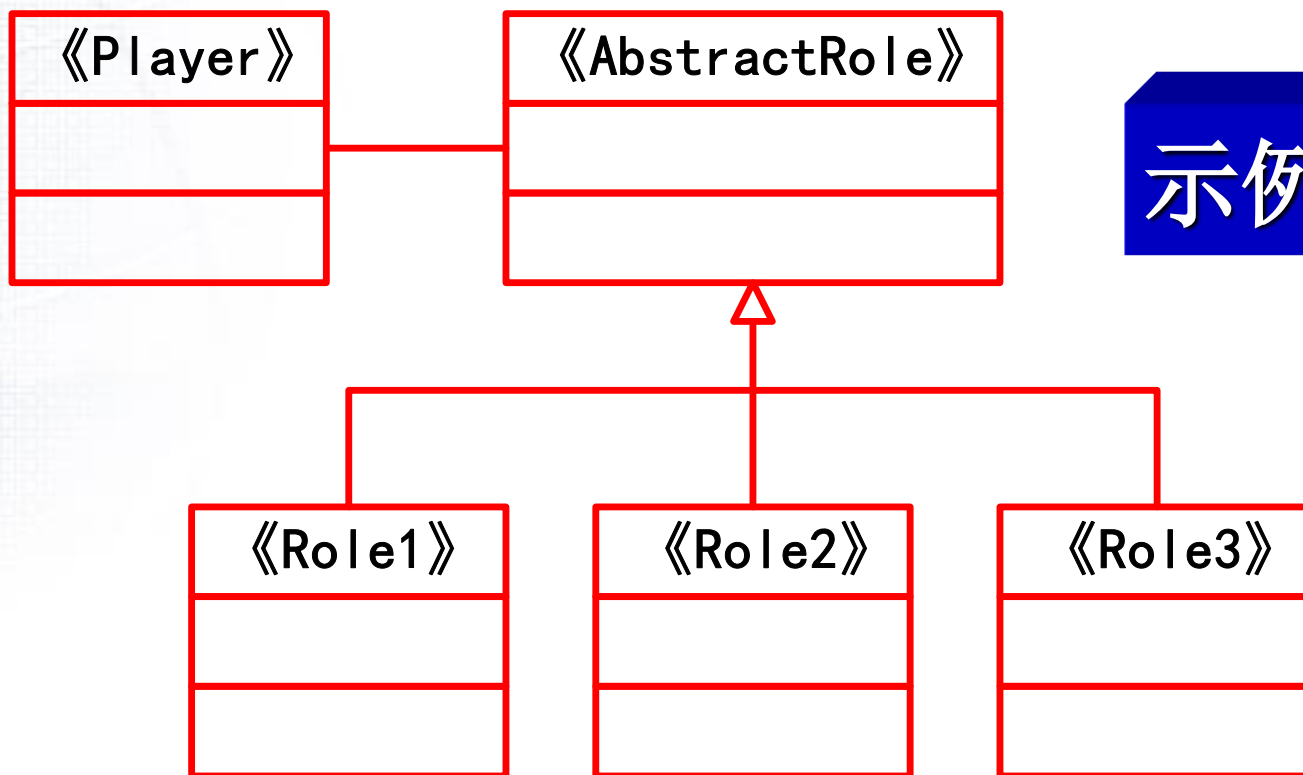
- 对象改变所属类(动态变化的对象特征)





6.3 抽象

- 利用玩家-角色模式(人员身份问题解决方法1)

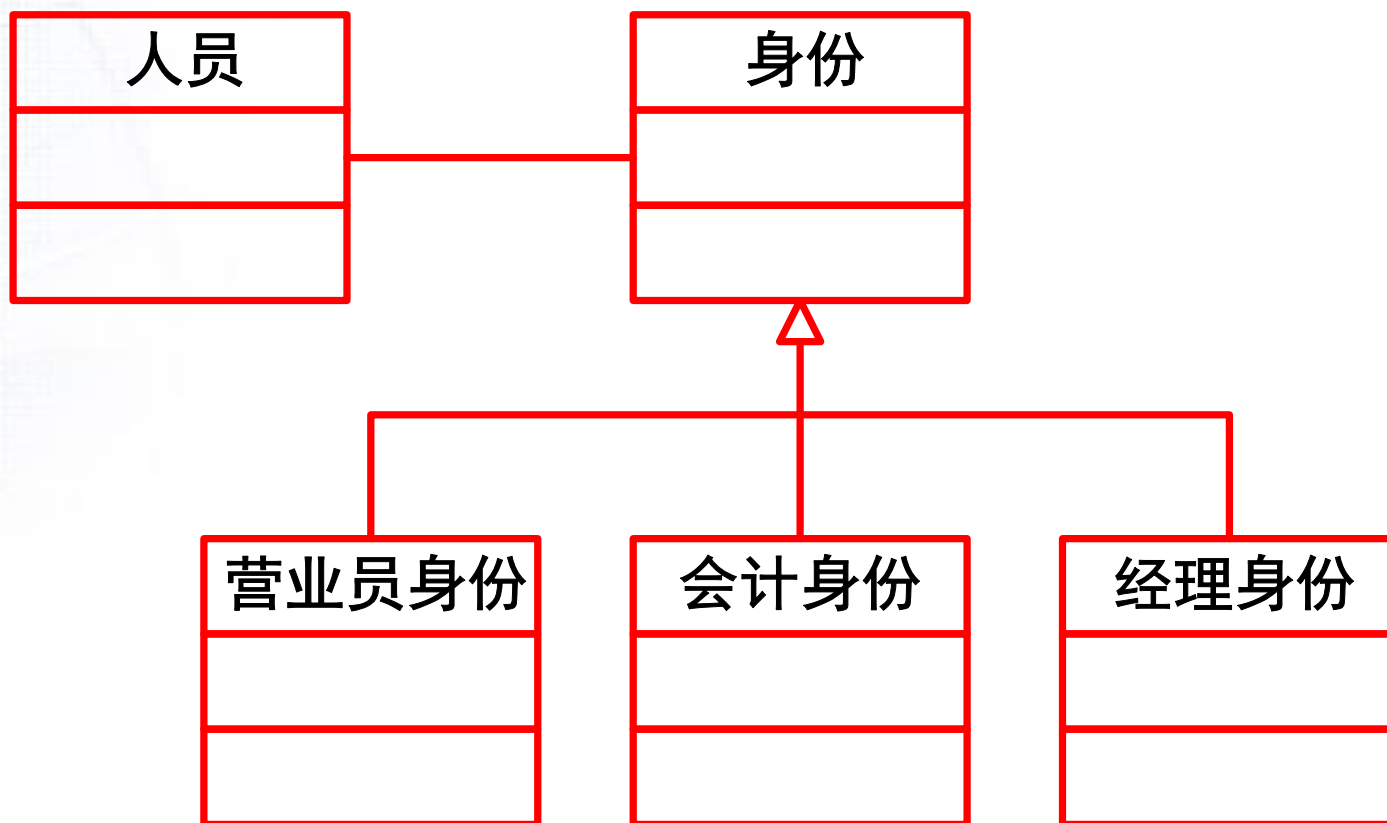


示例



6.3 抽象

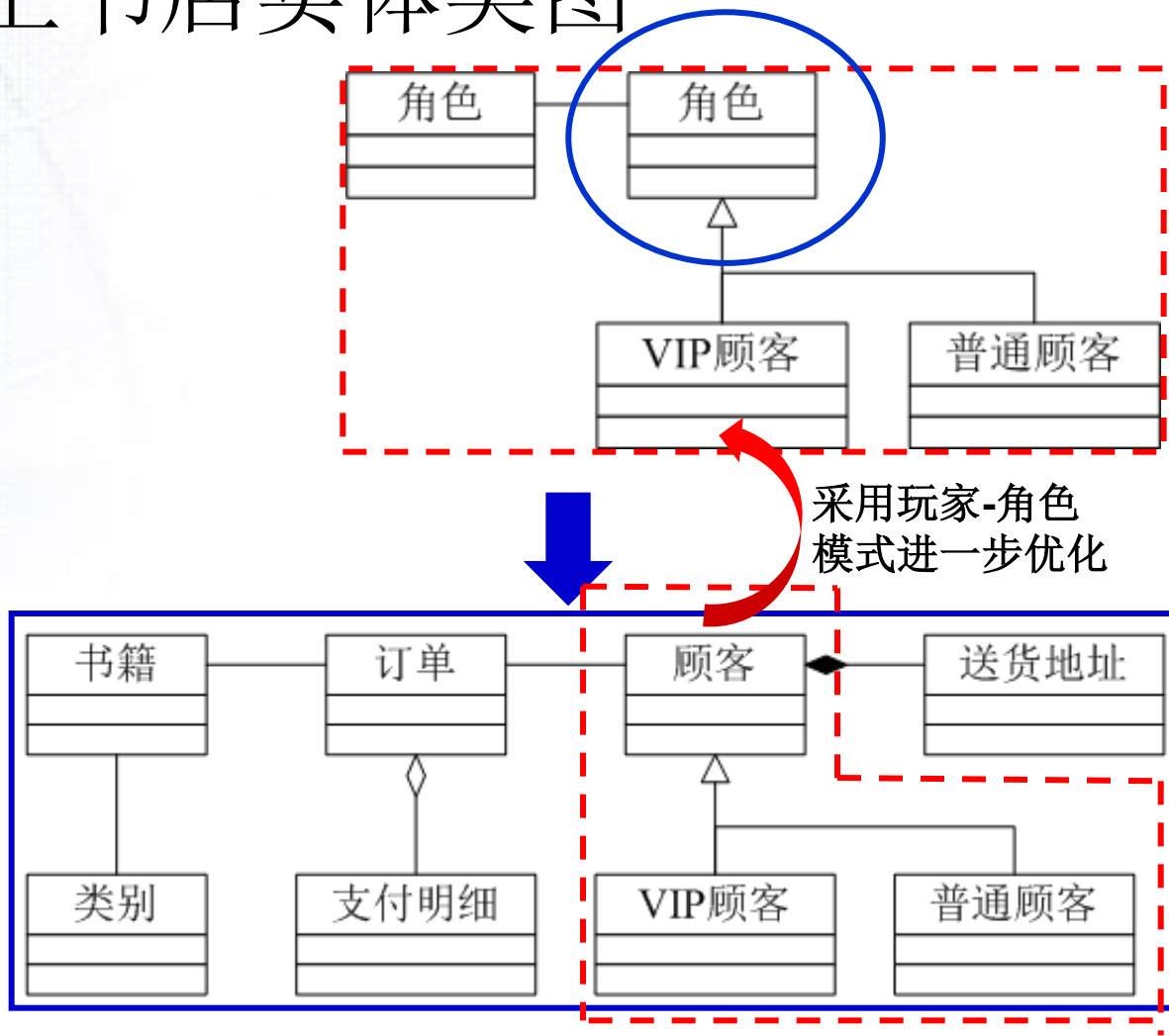
- 玩家-角色模式(人员身份问题解决方法1)





5.4 OOA方法——对象-关系建模 (附)

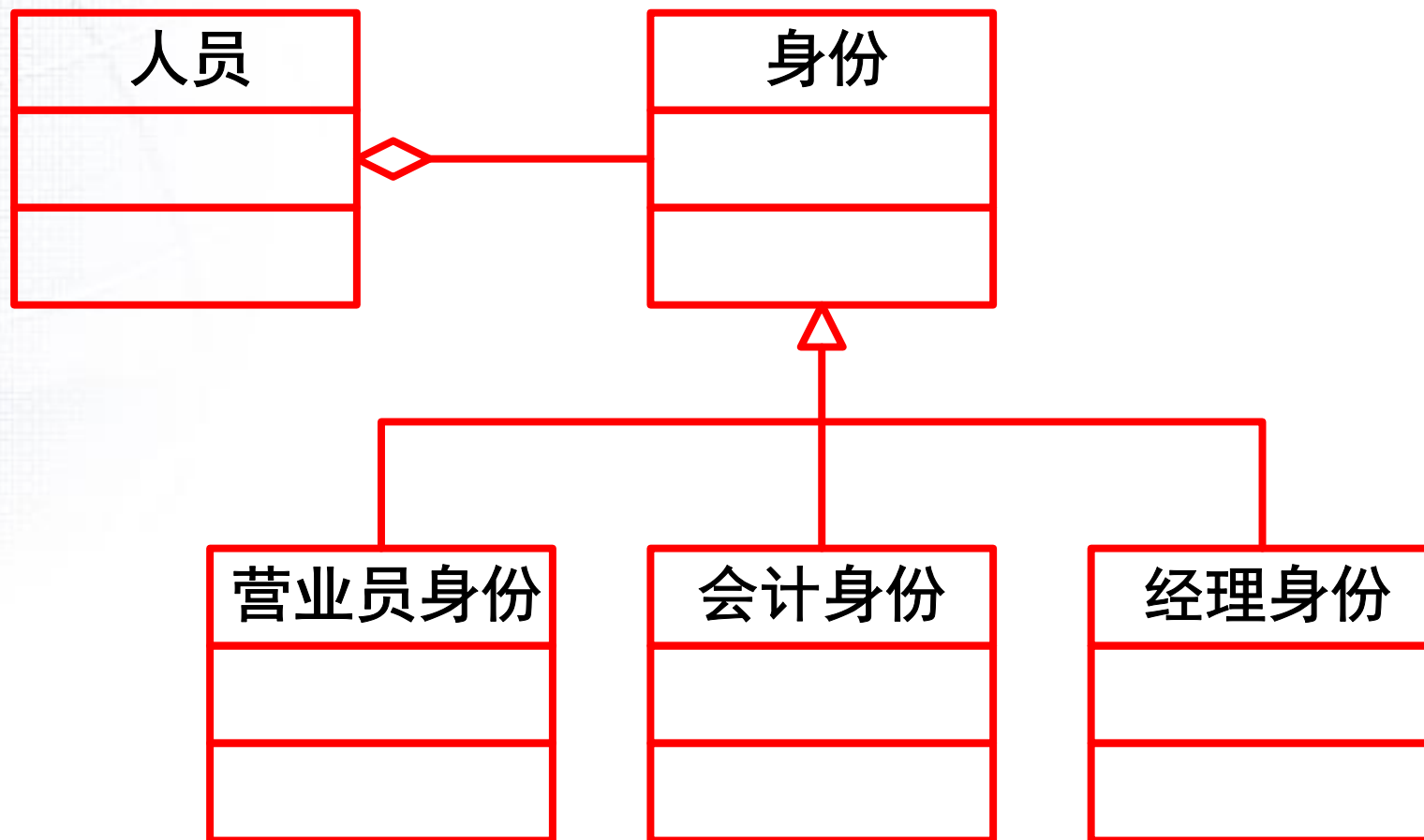
- 网上书店实体类图





6.3 抽象

- 利用聚集(人员身份问题解决方法2)

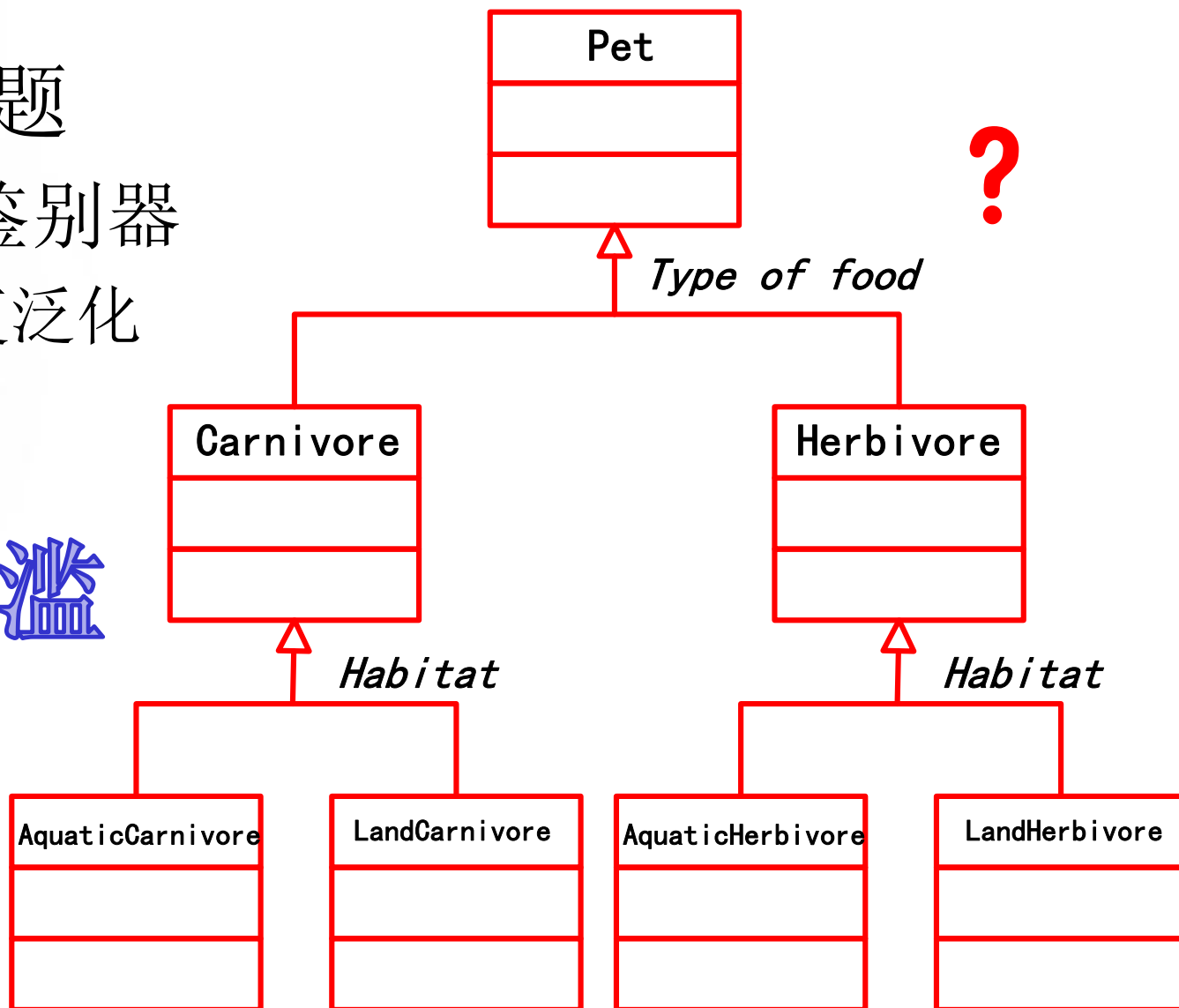




6.3 抽象

- 相关问题
 - 多重鉴别器
 - 重复泛化

泛化泛滥

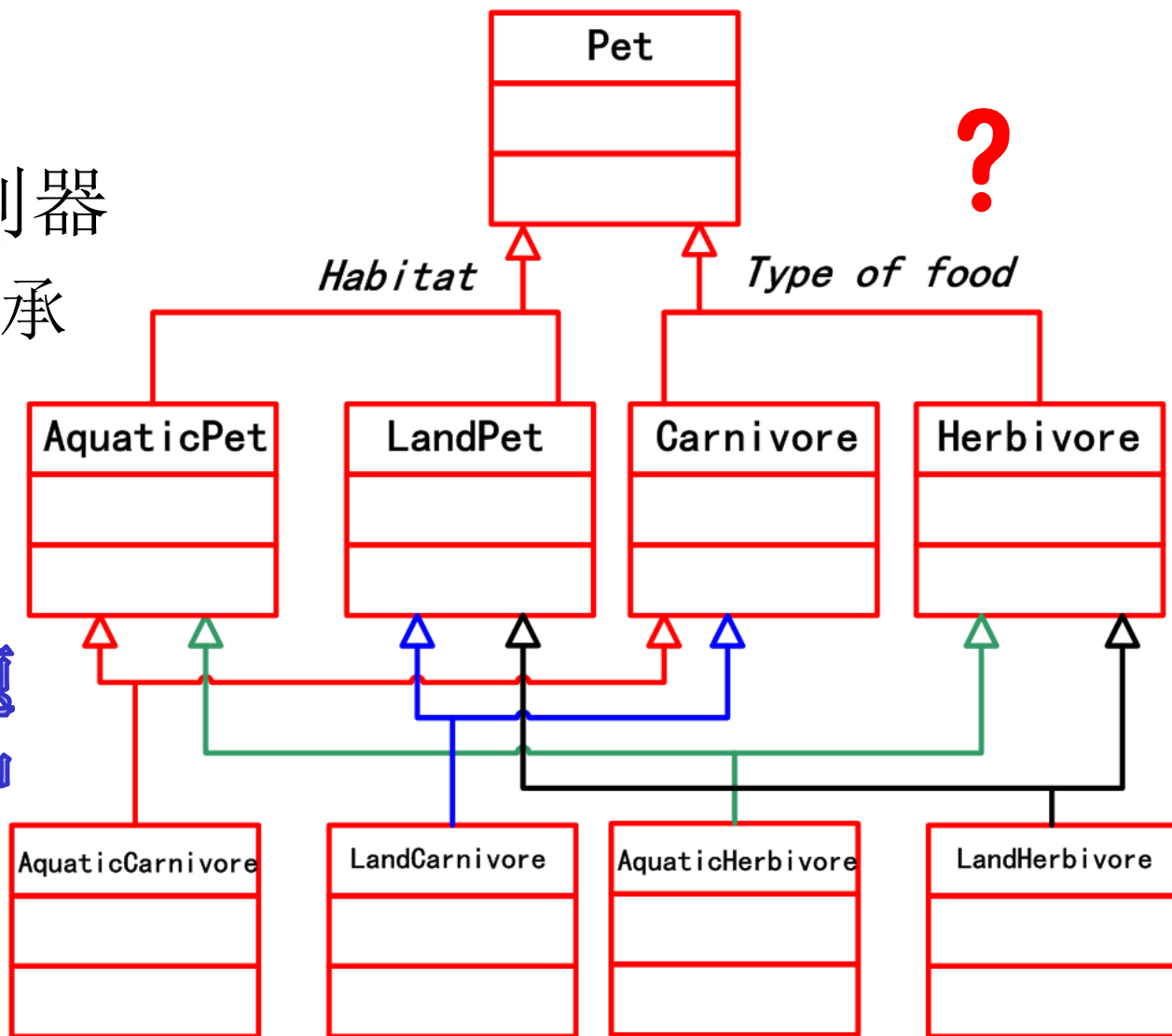




6.3 抽象

- 相关问题
 - 多重鉴别器
 - 多重继承

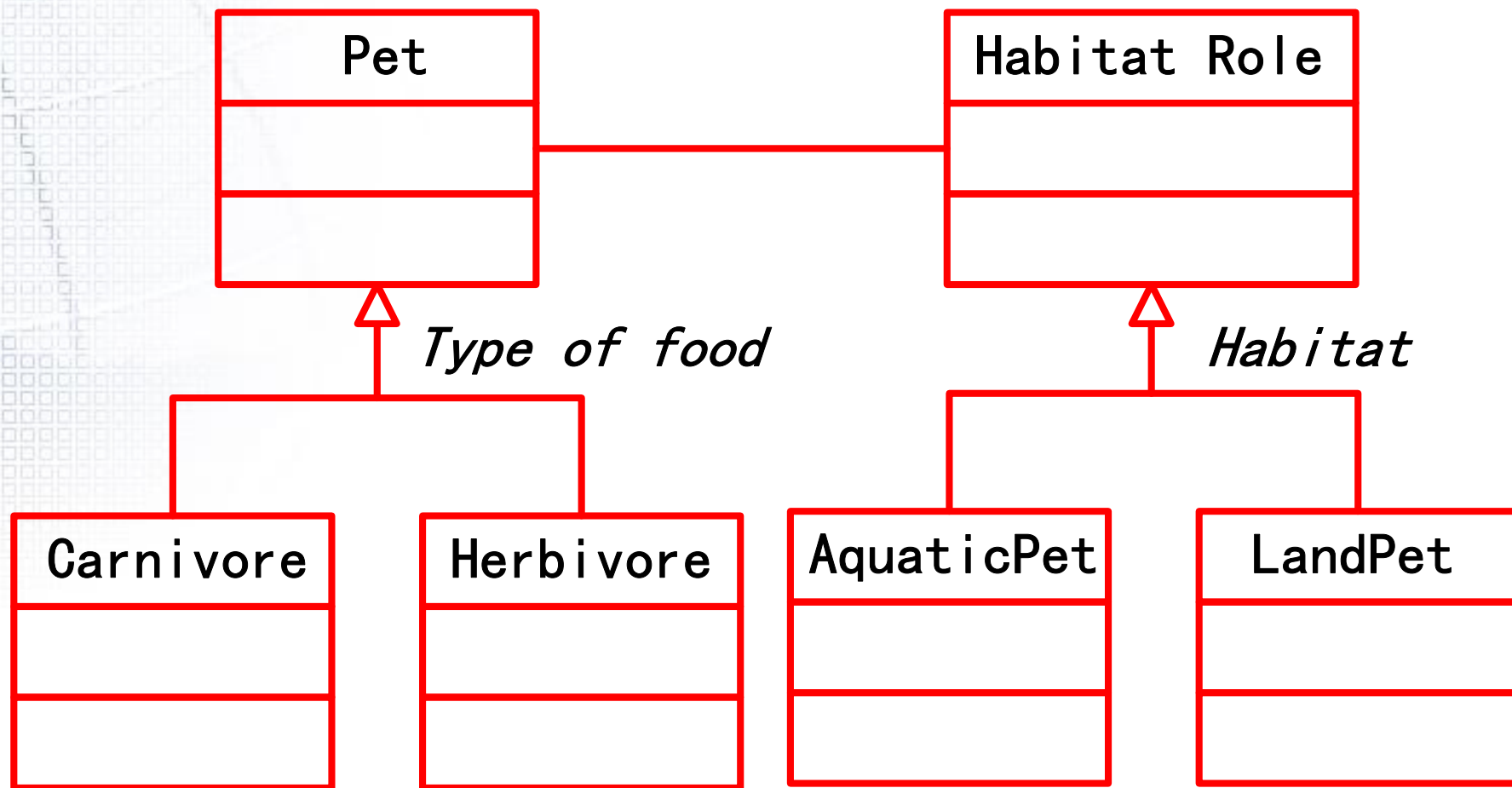
菱形继承问题
子类代码臃肿





6.3 抽象

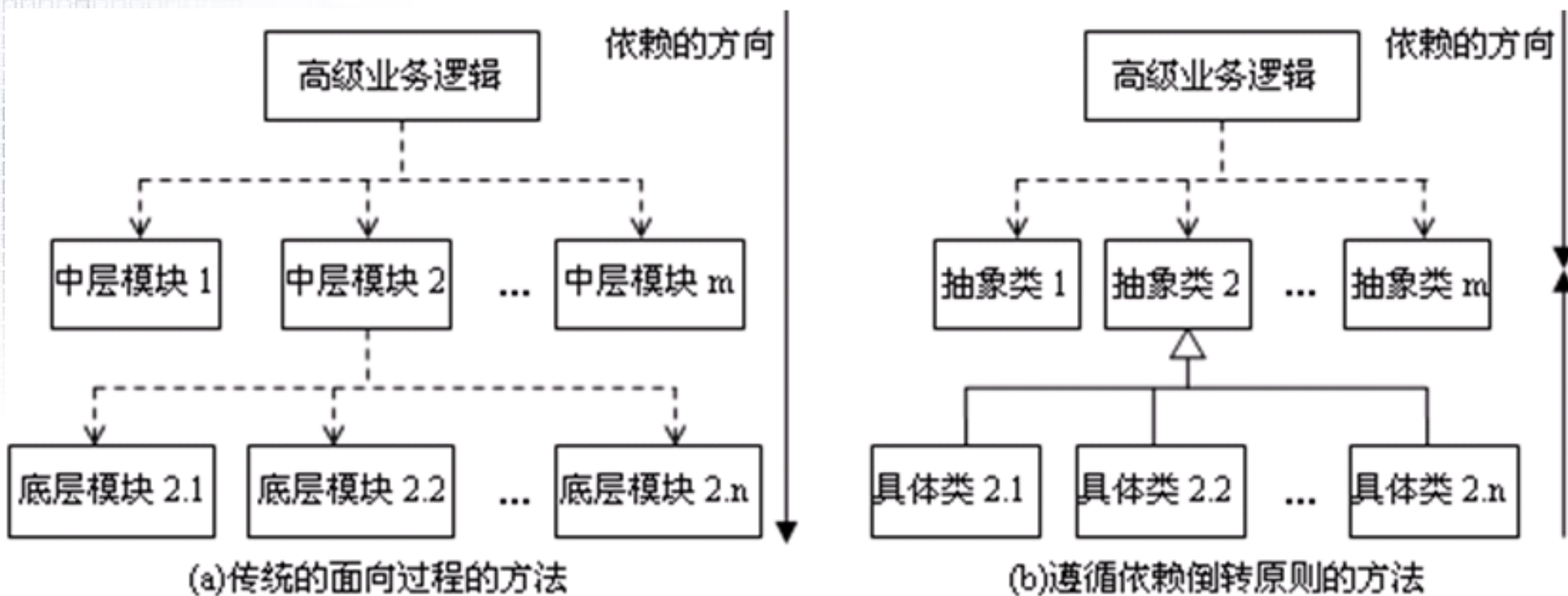
- 玩家-角色模式(多重鉴别器问题解决方法)





6.3 抽象

- 基于抽象的设计原则
 - 3) 依赖倒转原则(Dependence Inversion Principle)
 - 要依赖于抽象，不要依赖于具体类





- 基于抽象的设计原则

- 3) 依赖倒转原则(续)

- 要依赖于抽象，不要依赖于具体类

- 支付方式

- » 支付宝、微信、银行卡...

- 商品展示模块

- » 按类别展示...

- 发送消息模块

- » 邮件、短信

- 游戏中动作模块、场景模块.....

- » 攻击、移动、跳跃...

- » 城镇、乡村...



6.3 抽象

- 传统软件工程中的抽象

- 信息隐蔽

- 指一个模块内的数据和模块的实现细节对于该模块的客户即调用者模块有不可见的性质

- 数据局部化

- 缺点

- 仅对模块细节的封装，没有继承的概念，虽然可以“到处复用”，却也需要“到处修改”，维护成本大



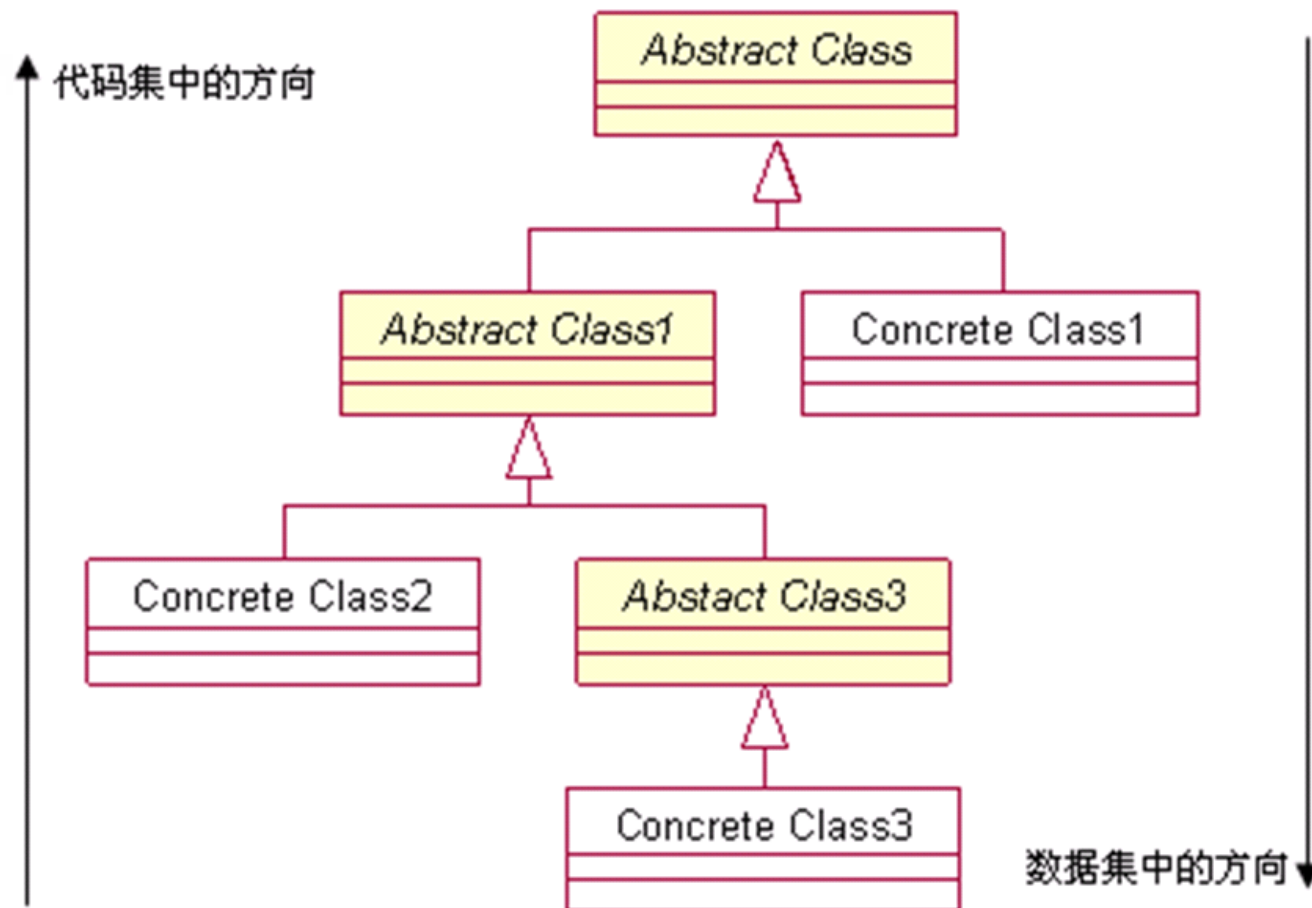
6.3 抽象

- 面向对象软件工程中的抽象
 - 继承传统
 - 私有类
 - 发展深化
 - 抽象类
 - 接口



6.3 抽象

- 面向对象软件工程中的抽象
 - 抽象类





6.3 抽象

- 基于抽象的设计原则

- 接口（端口）

- 同一接口，不同类可以实现完全不一样的行为



- 3) 接口隔离原则(Interface Segregation Principle)

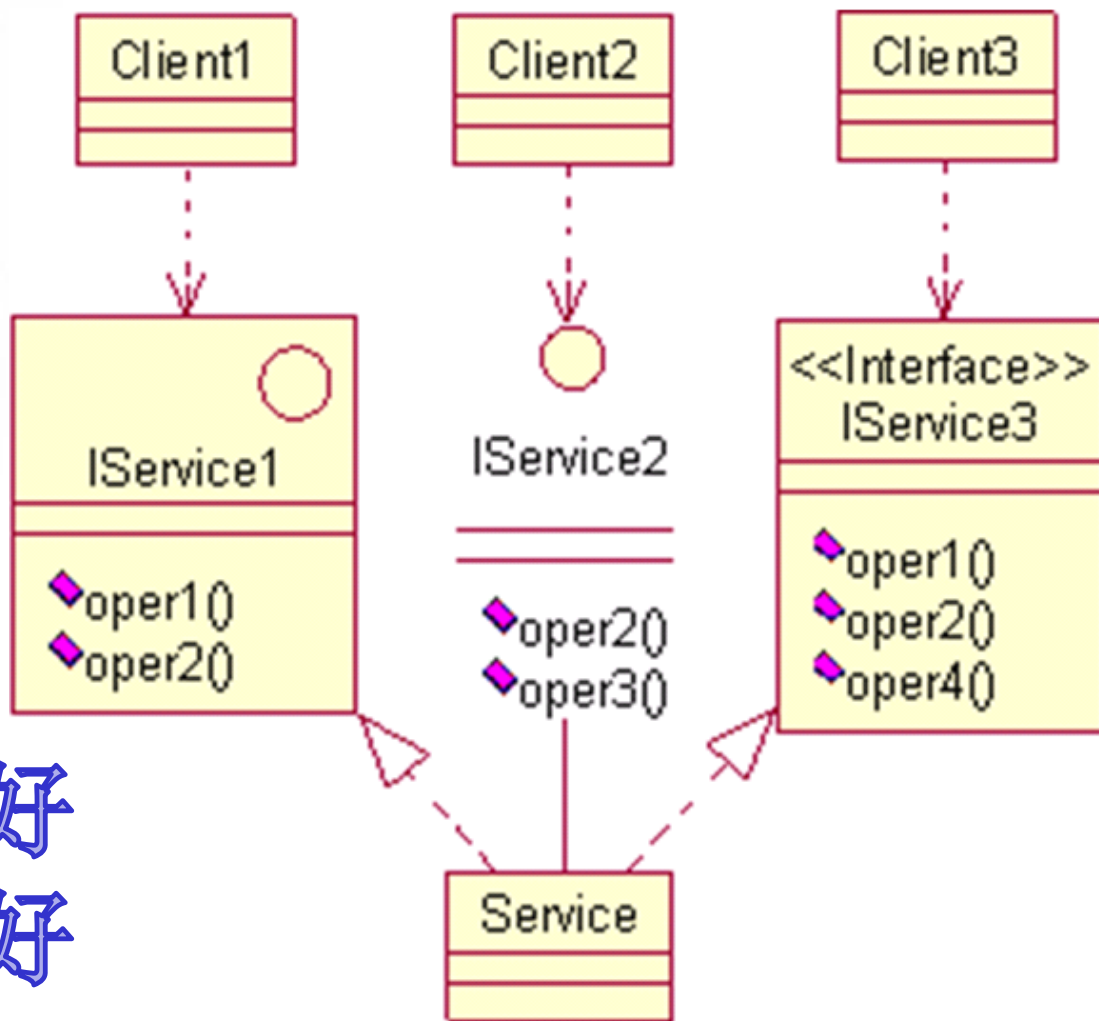
- 使用多个专门接口比使用单一总接口要好得多



6.3 抽象

- 面向对象软件工程中的抽象

— 接口



复用性好
扩展性好

练习



小结-SOLID

一个类只做一件事
好理解、可读性好

Single
Responsibility



O

Open-Closed
Principle

对扩展开放，对修改关闭
降低维护的风险

要基于行为继承
防止继承泛滥

Liskov
Substitution

L

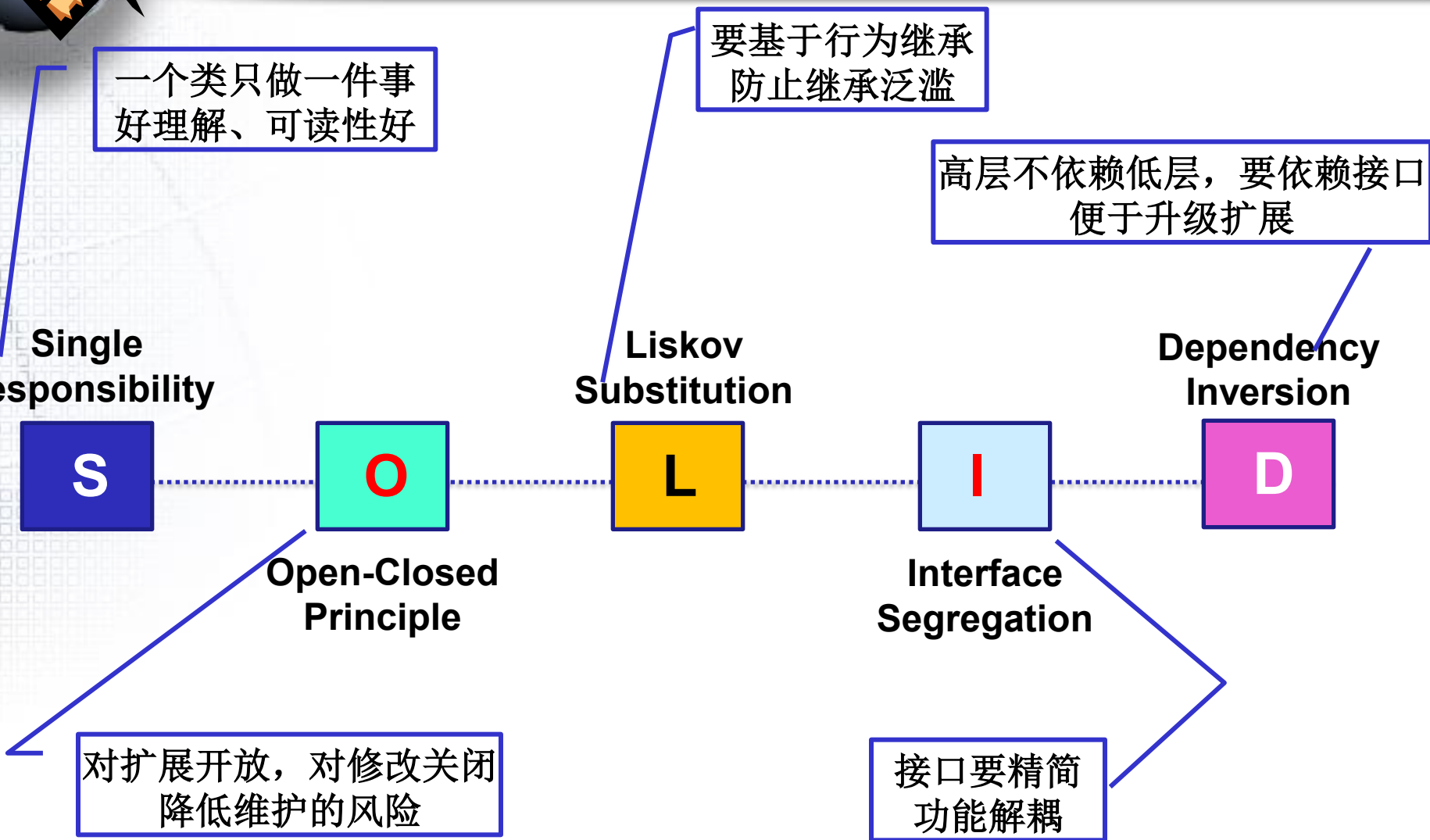
高层不依赖低层，要依赖接口
便于升级扩展

Dependency
Inversion

D

Interface
Segregation

接口要精简
功能解耦





6.4 内聚和耦合

- 内聚

- 是一个模块内部各部件之间联系紧密程度的度量
- 强调分解时将相关的内容放到一起
- 一个模块内的各个部件联系越紧越好

- 耦合

- 是模块间相互联系强弱的度量
- 耦合的强弱取决于模块间传递数据的方式、接口复杂情况以及传递数据的类型
- 各模块之间的耦合越松散越好



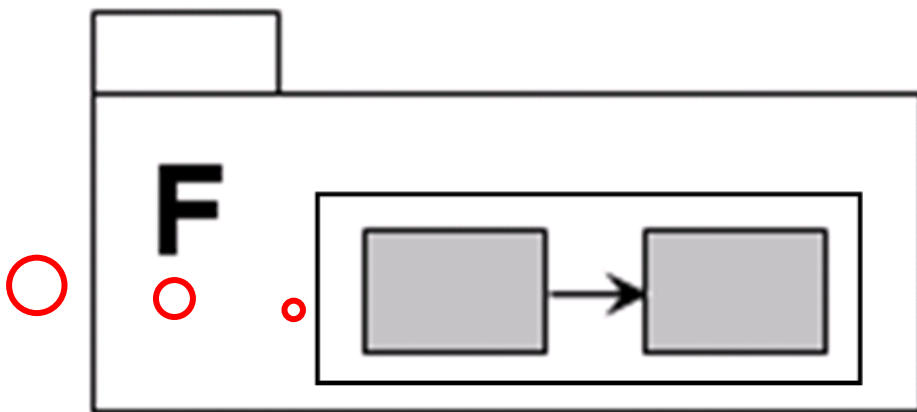
6.4.1 内聚

• 内聚的分类

— 1) 功能内聚 (Functional Cohesion)

- 所有部件处理同一组数据，共同完成单一的功能
- 易于理解、易于诊断、易于维护、易于替换、易于复用

计算三角形
面积、数学
函数、商品
总额统计等



- 仅仅是个计算结果，数据不变
- 无副作用，理想的内聚



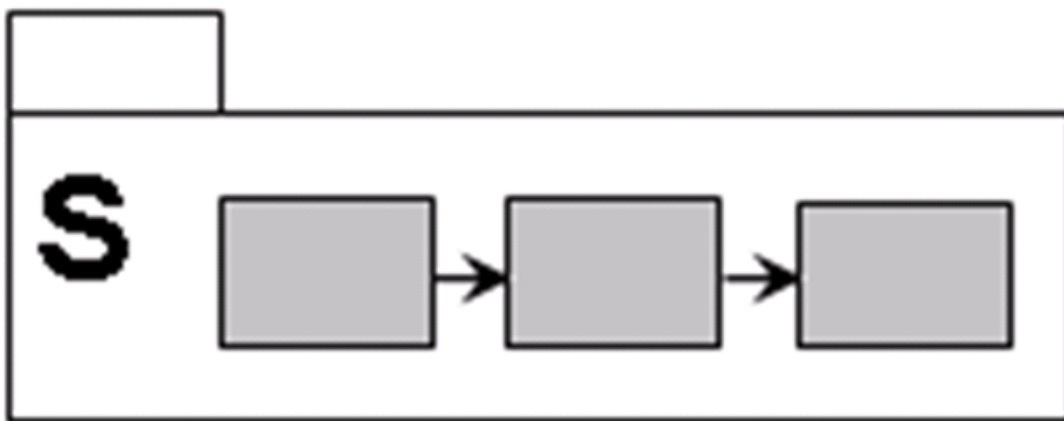
6.4.1 内聚

- 内聚的分类

- 2) 顺序内聚 (Sequential Cohesion)

- 各部件之间既有数据联系又有控制联系
 - 功能不单一，较为理想

读数据、降
噪消除回声、
压缩声音、
传输数据等



光学字符识别 (OCR) 模块

预处理

字符切分

字符识别

字符组装

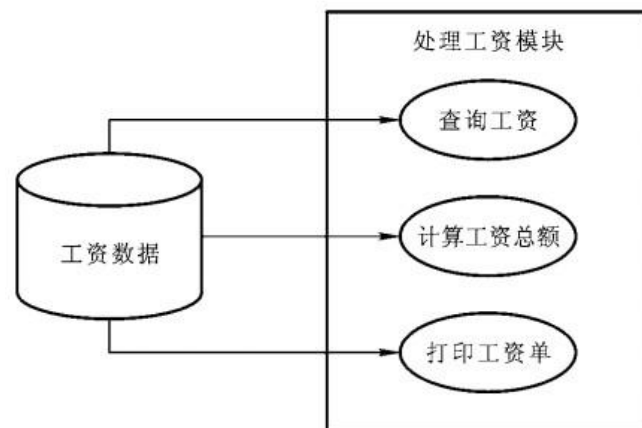
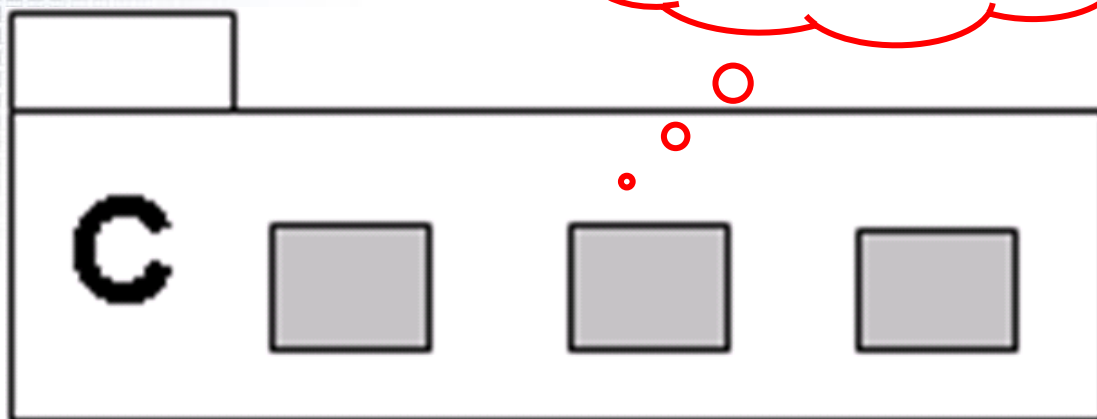
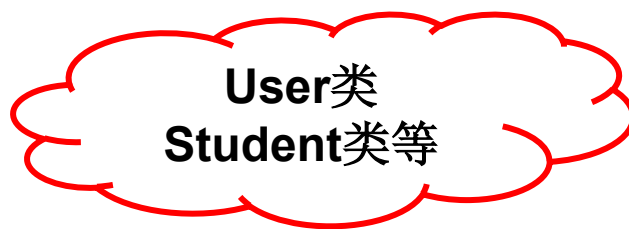


6.4.1 内聚

内聚的分类

– 3) 通信内聚 (Communicational Cohesion)

- 所有的部件都访问同一组数据，各部件之间只有数据关系，没有控制关系
- 较为理想





6.4.1 内聚

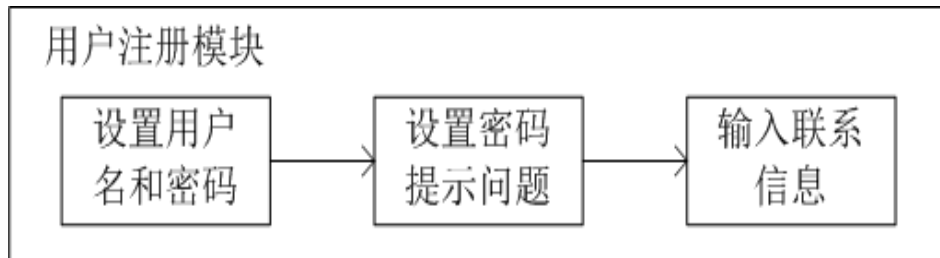
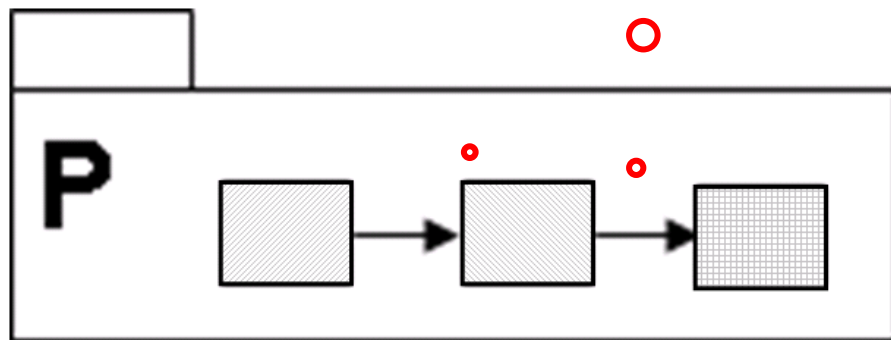
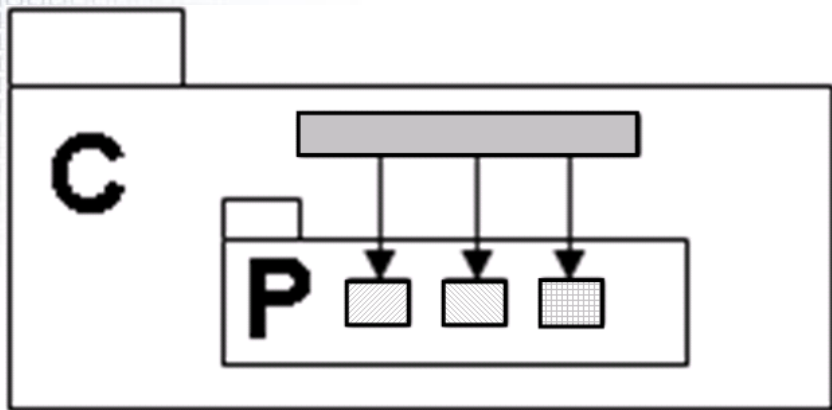
登录：判断用户、
验证码、记录日志

下订单：计算订单、
确定用户余额、顾
客确认、通知送货

• 内聚的分类

–4) 过程内聚 (Procedural Cohesion)

- 各部件之间只有控制联系，而没有数据联系
- 较弱的内聚
- 直接或间接控制





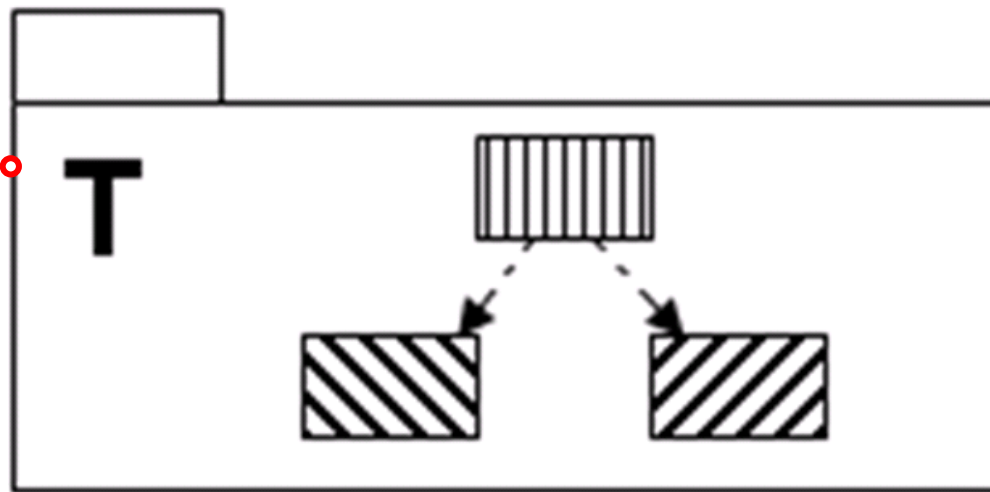
6.4.1 内聚

- 内聚的分类

- 5) 时间内聚 (Temporal Cohesion)

- 所有部件之间既无数据联系，也无控制联系，但是部件之间具有时间关系，即把在执行过程中同一阶段内完成执行任务的部件放到一起，而排除其它部件
 - 内聚更弱

程序初始化、
程序退出、
系统启动等





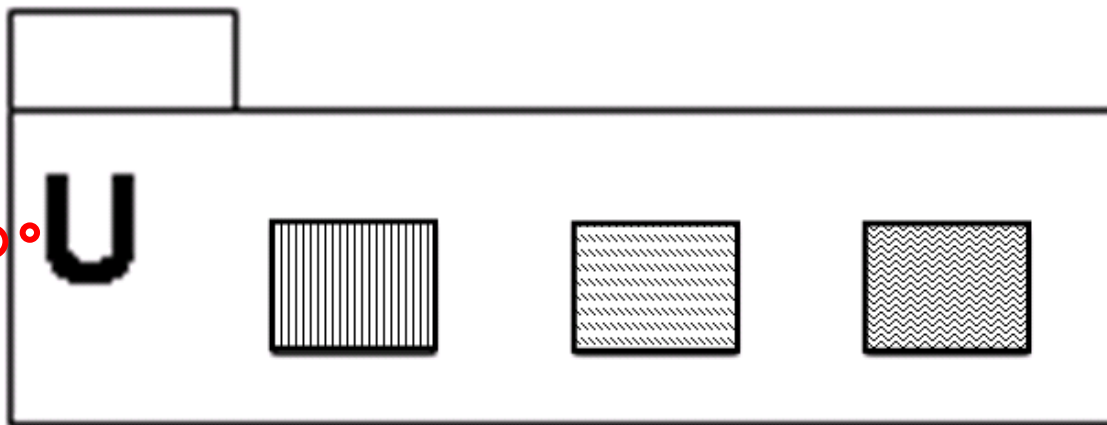
6.4.1 内聚

- 内聚的分类

- 6) 实用程序内聚 (Utility Cohesion)

- 部件常常是一些相关的、可重用的实用程序
 - 解决同一类问题
 - 内聚更弱

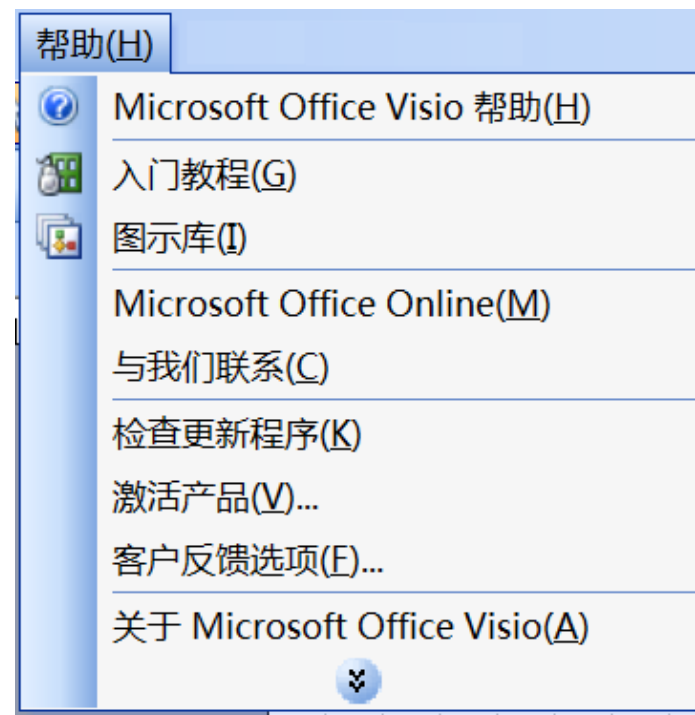
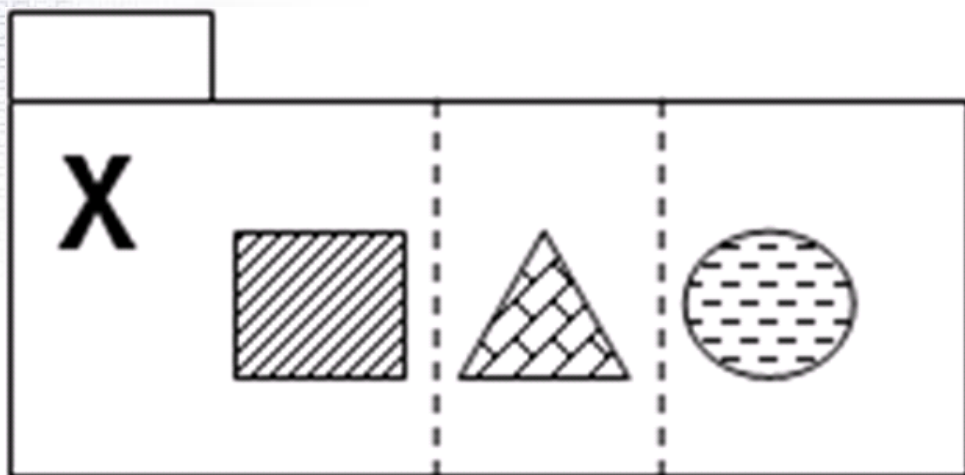
string.h、
stdio.h等





6.4.1 内聚

- 内聚的分类
 - 7)偶然内聚 (Coincidental Cohesion)
 - 模块内部的各部件之间没有任何关系
 - 最不好的一种内聚类型





6.4.1 内聚

- 内聚的分类

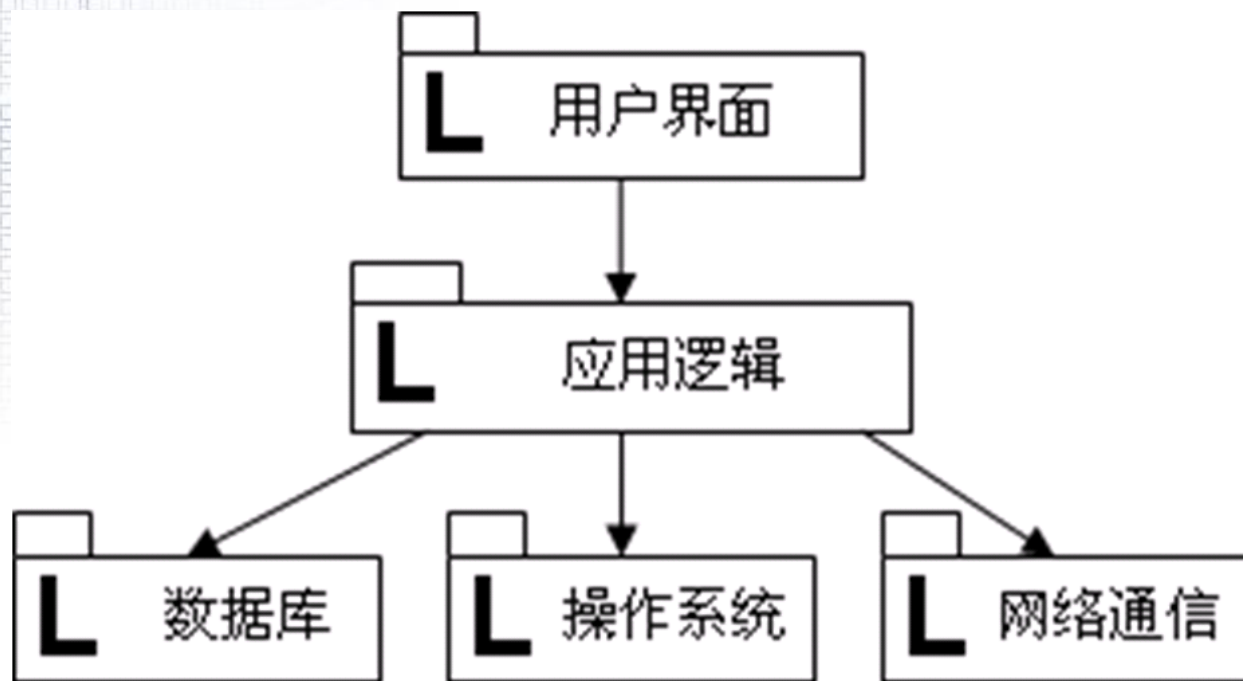
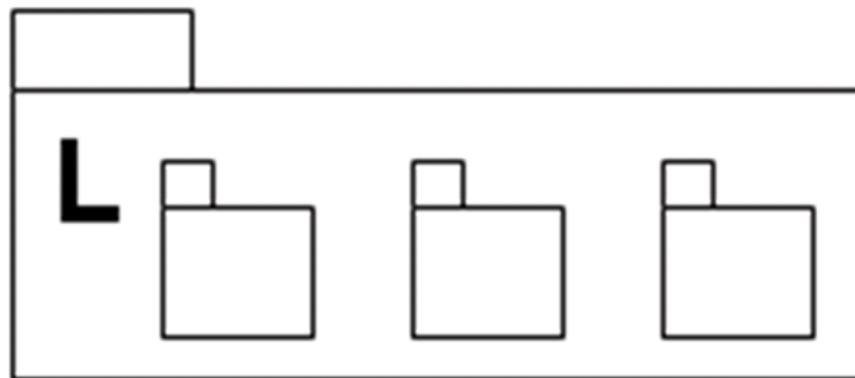
- 8) 层内聚 (Layer Cohesion)

- 将相关服务的功能放到一起，成为一层，而将其它内容排出在外
 - 优点
 - 可替换性：替换低层时可以用等价层（实现原层所有的**API**）而不必修改高层；
 - 单向依赖：替换高层模块对低层模块没有影响（低层服务不访问高层服务）
 - 层内聚被广泛地应用在软件的分层架构上



6.4.1 内聚

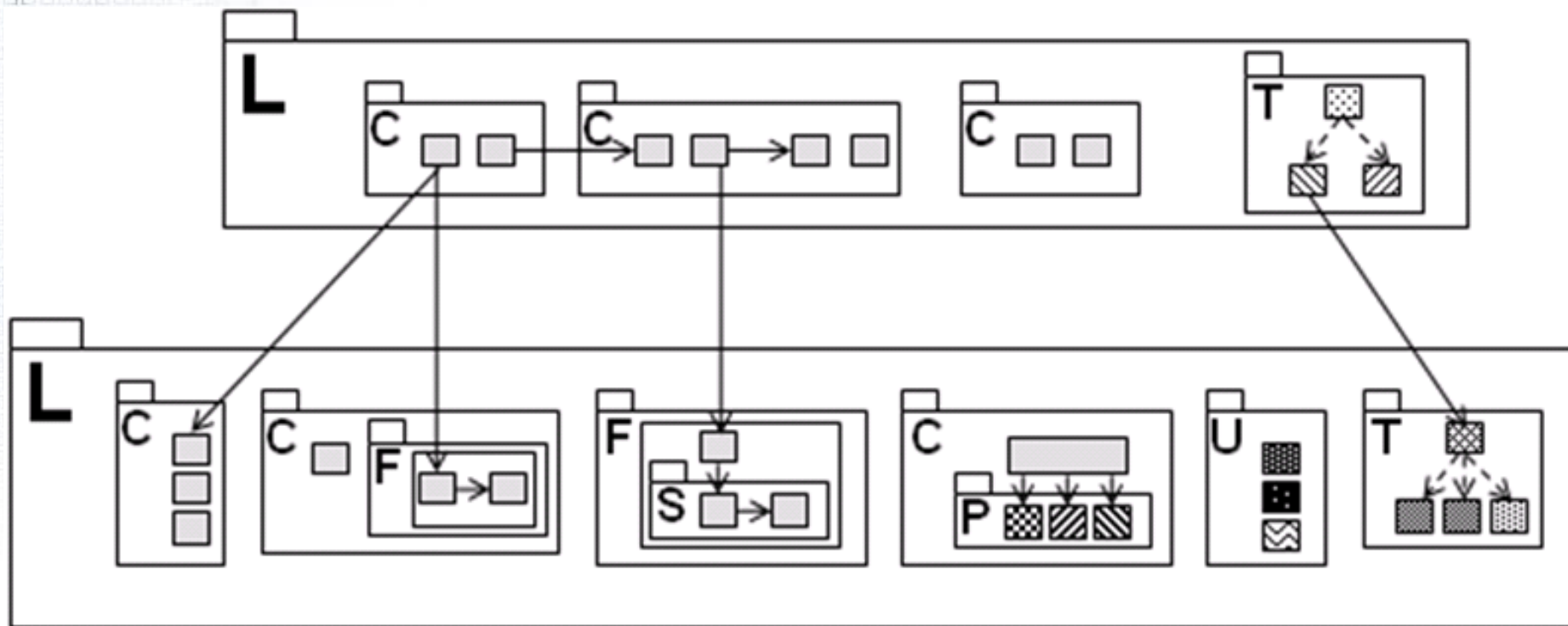
- 内聚的分类
 - 8) 层内聚





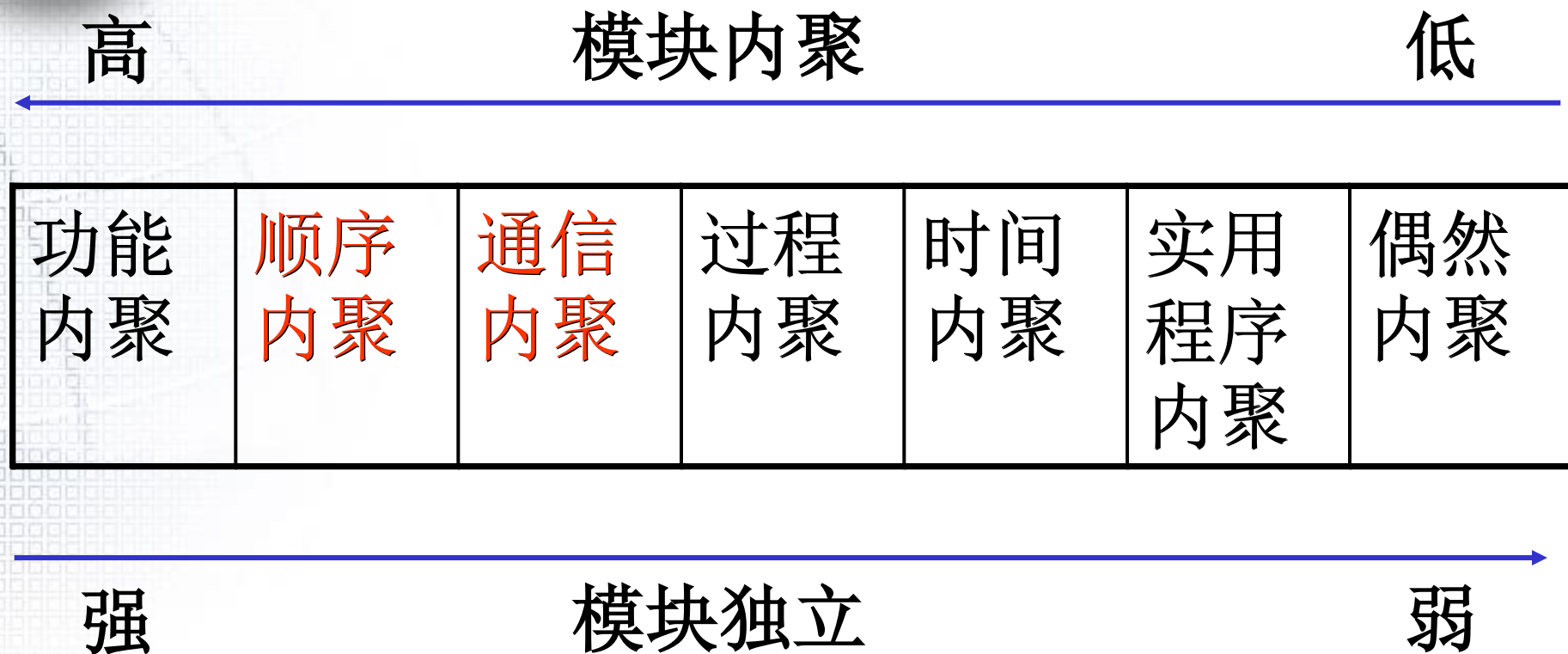
6.4.1 内聚

- 相互嵌套的内聚模块





6.4.1 内聚总结

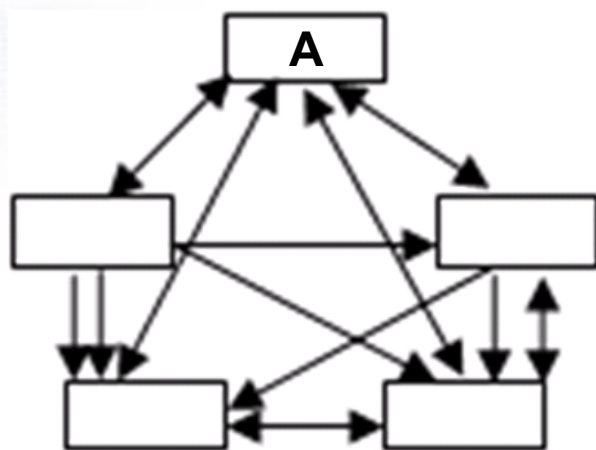


内聚练习

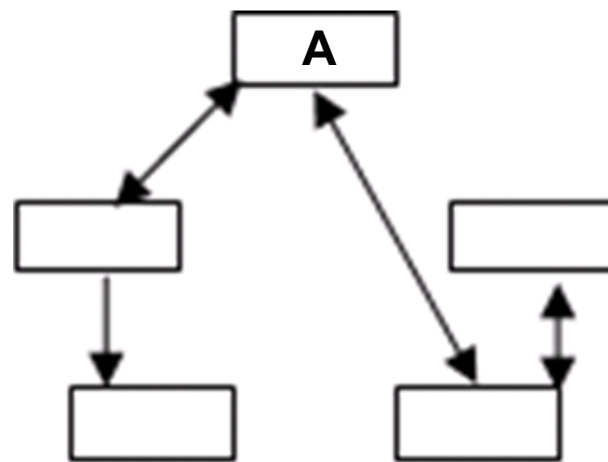


6.4.2 耦合

- 解耦的原因
 - 牵一发而动全身
 - 很难快速地理解紧耦合的模块
 - 独立性较差



(a) 紧耦合系统



(b) 松耦合系统



6.4.2 耦合

- 松散耦合——W3C对耦合及松耦合的定义
 - 耦合：是互相交互的系统彼此间的依赖。这种依赖又分为：
 - 真依赖：系统从其他系统消费的要素或服务的集合。真依赖总是存在，无法简化
 - 假依赖：系统为获得其他系统提供的要素或服务而不得不服从的因素。典型的假依赖包括语言依赖、平台依赖、**API**依赖等。假依赖总是存在，但可减少或降低其代价
 - 松耦合：理想状态的松耦合体现在系统间仅存在真依赖关系。真实情况下，松耦合是一个相对概念，体现在某系统的假依赖已经最小化



6.4.2 耦合

- 耦合的分类

- 1) 内容耦合 (Content coupling)

- 一个模块直接进入另一个模块中存取其数据
 - 一个模块修改了另一个模块的代码
 - 一个模块内的分支转移到了另一个模块中
 - 内容耦合的情形，被访问模块的任何变更，都可能造成程序出错，给系统造成很大的副作用，应避免出现
 - 最不好的一种耦合

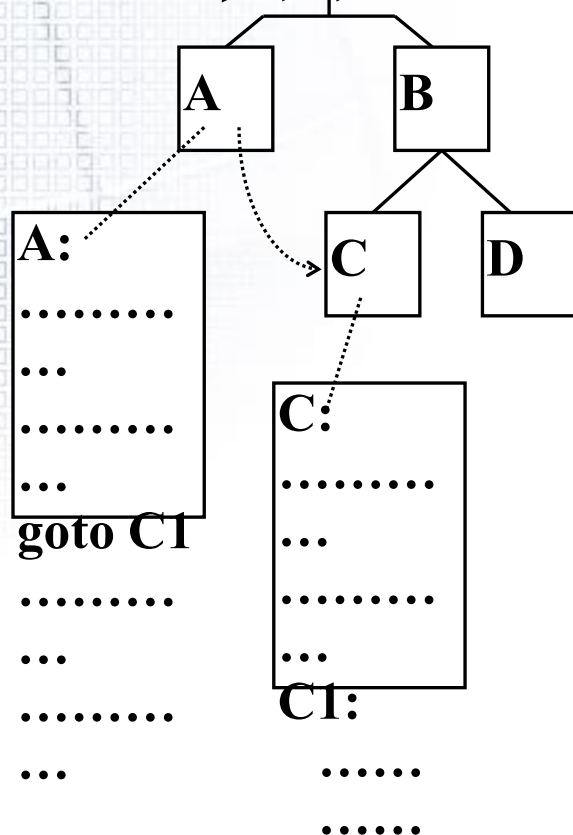


6.4.2 耦合

- 耦合的分类

- 1) 内容耦合 (Content coupling)

- 举例..



```
public class Product
{
    public float unitPrice;
    .....
}
public class order
{
    private Product myProduct=new Product();
    public void setItem()
    {
        myProduct.unitPrice=-100;
        .....
    }
}
```

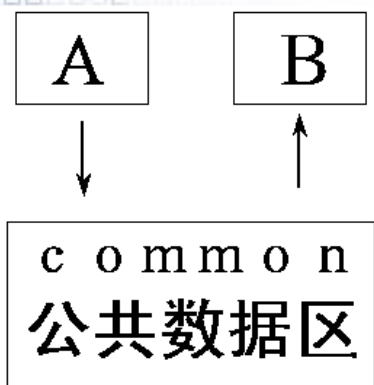


6.4.2 耦合

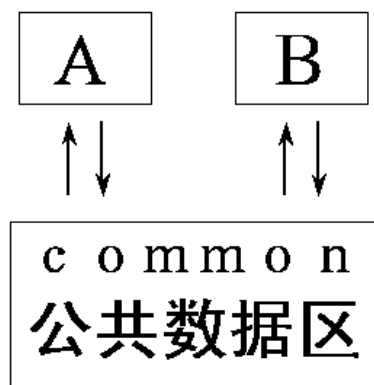
- 耦合的分类

- 2) 公共耦合 (Common coupling)

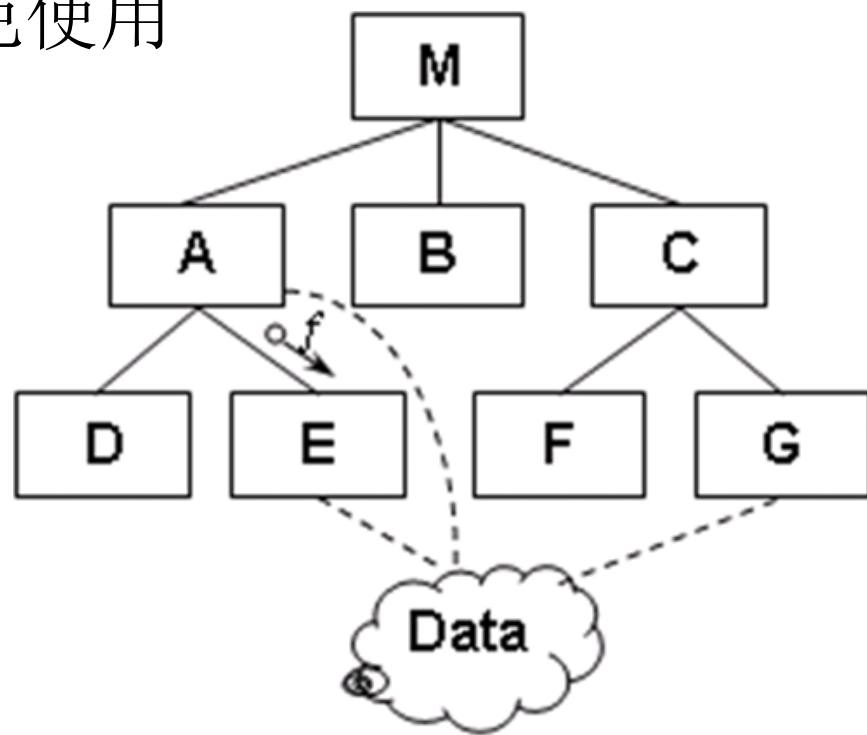
- 两个模块间通过一个公共环境进行数据交换
 - 较差的耦合，应该避免使用



松散的公共耦合



紧密的公共耦合





6.4.2 耦合

- 耦合的分类

- 3) 外部耦合 (External coupling)

- 模块对外部系统（如软件操作系统、数据库、硬件设备，以及可复用的组件库等）有依赖关系
 - 有时无法避免，应该尽量减少

可以通过分层降耦，如**ODBC、JDBC、JVM**

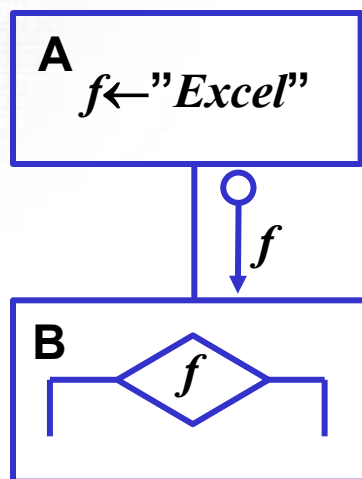


6.4.2 耦合

- 耦合的分类

- 4) 控制耦合 (Control coupling)

- 两个模块之间通过接口的参数表交换开关数据，旨在控制另一个模块的执行逻辑
 - 较差



```
public void B(string f)
{
    if(f=="excel")
        saveAsExcel();
    else
        saveAsWord();
}
```

降耦方法:

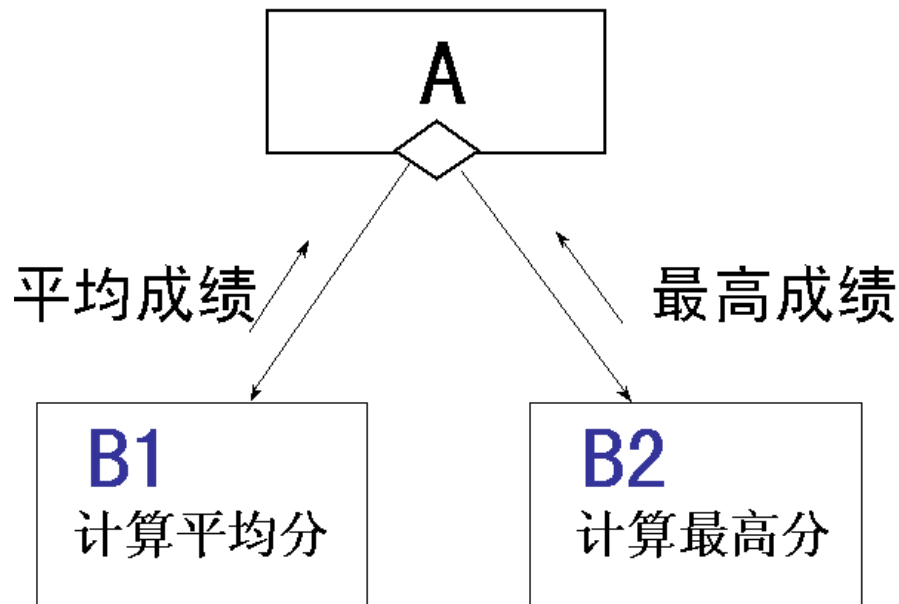
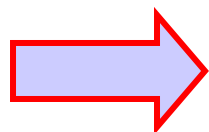
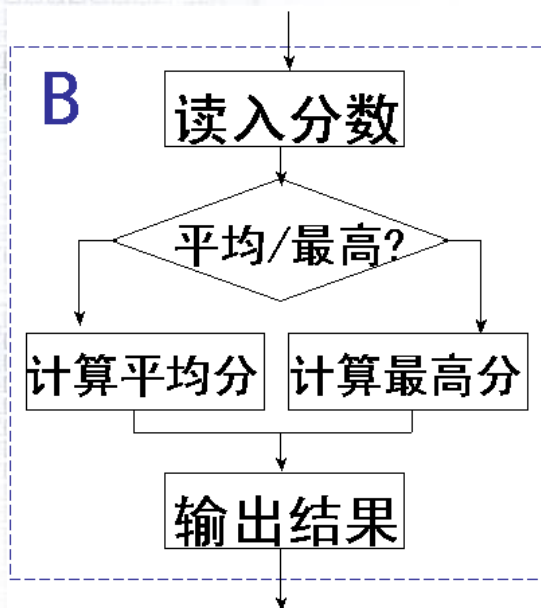
- 1、if判断上移到调用方法中;
- 2、采用多态操作，每个有自己的save方法



6.4.2 耦合

- 耦合的分类

- 4) 控制耦合 (Control coupling)



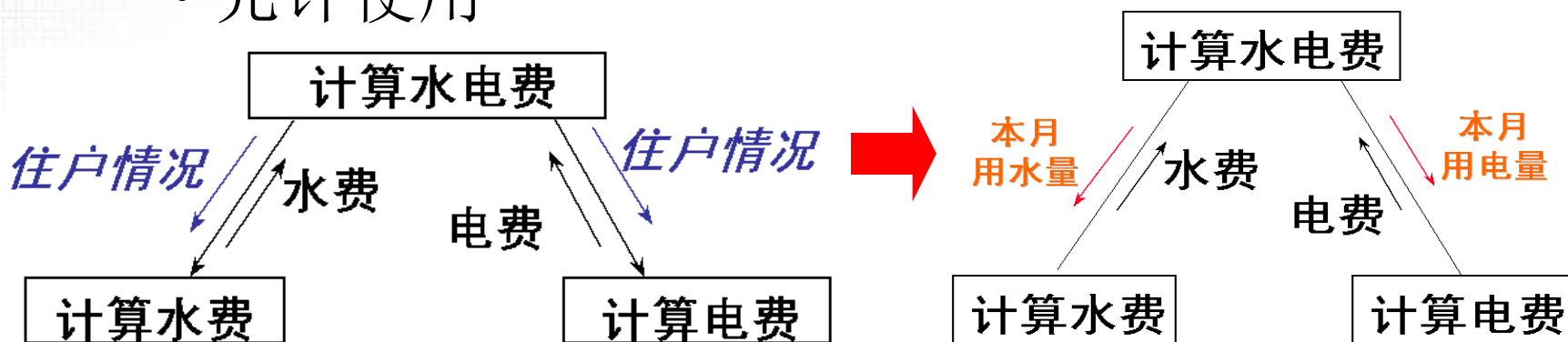


6.4.2 耦合

• 耦合的分类

– 5) 印记耦合 (Stamp coupling)

- Pressman: 指两个模块之间通过参数交换方式产生耦合，并且交换的是数据结构而不是数据元素
- Timothy C. Lethbridge: 当应用程序的类被声明为方法参数类型时，就会产生印记耦合
- 允许使用





6.4.2 耦合

采用接口隔离原则进行降耦



```
public class Order {  
    public float calcTotalMoney(User user) {  
        int userLevel = user.getLevel();  
        int userConsumeScore = user.getConsumeScore();  
        ... .. //计算订单总金额  
    }  
    ... ..  
}
```



该印记耦合如何降耦？

```
public interface IConsumeInfo {  
    int getLevel();  
    int getConsumeScore();  
}  
public class User Implements IConsumeInfo {  
    public int getLevel() {  
        ... .. //获得用户等级代码  
    }  
    public int getLevel() {  
        ... .. //获得用户等级代码  
    }  
    ... ..  
}  
public class Order {  
    public float calcTotalMoney(IConsumeInfo cinfo)  
    {  
        int userLevel = cinfo.getLevel();  
        int userConsumeScore = cinfo.getConsumeScore();  
        ... .. //计算订单总金额  
    }  
    ... ..  
}
```

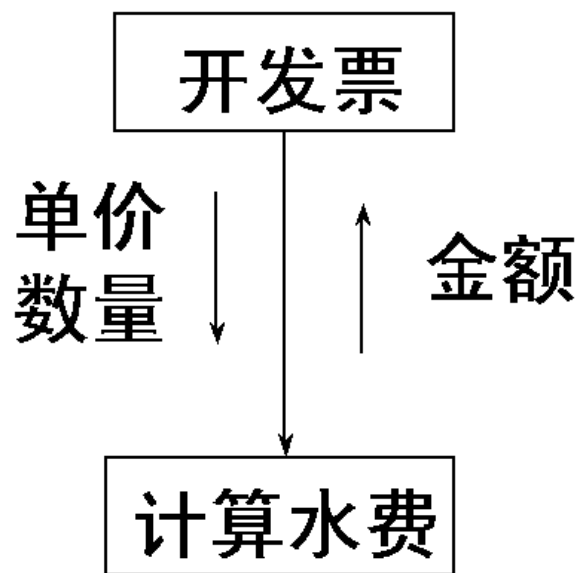


6.4.2 耦合

- 耦合的分类

- 6) 数据耦合 (Data coupling)

- 两个模块之间通过接口的参数表交换信息数据，并且这些信息数据的类型是基本数据类型
 - 需要传递的参数较少时，数据耦合较好
 - 需要传递的参数较多时，印记耦合要更好





6.4.2 耦合

- 耦合的分类
 - 6) 数据耦合 (Data coupling)
 - 降耦方法-采用降为数据耦合方法降耦

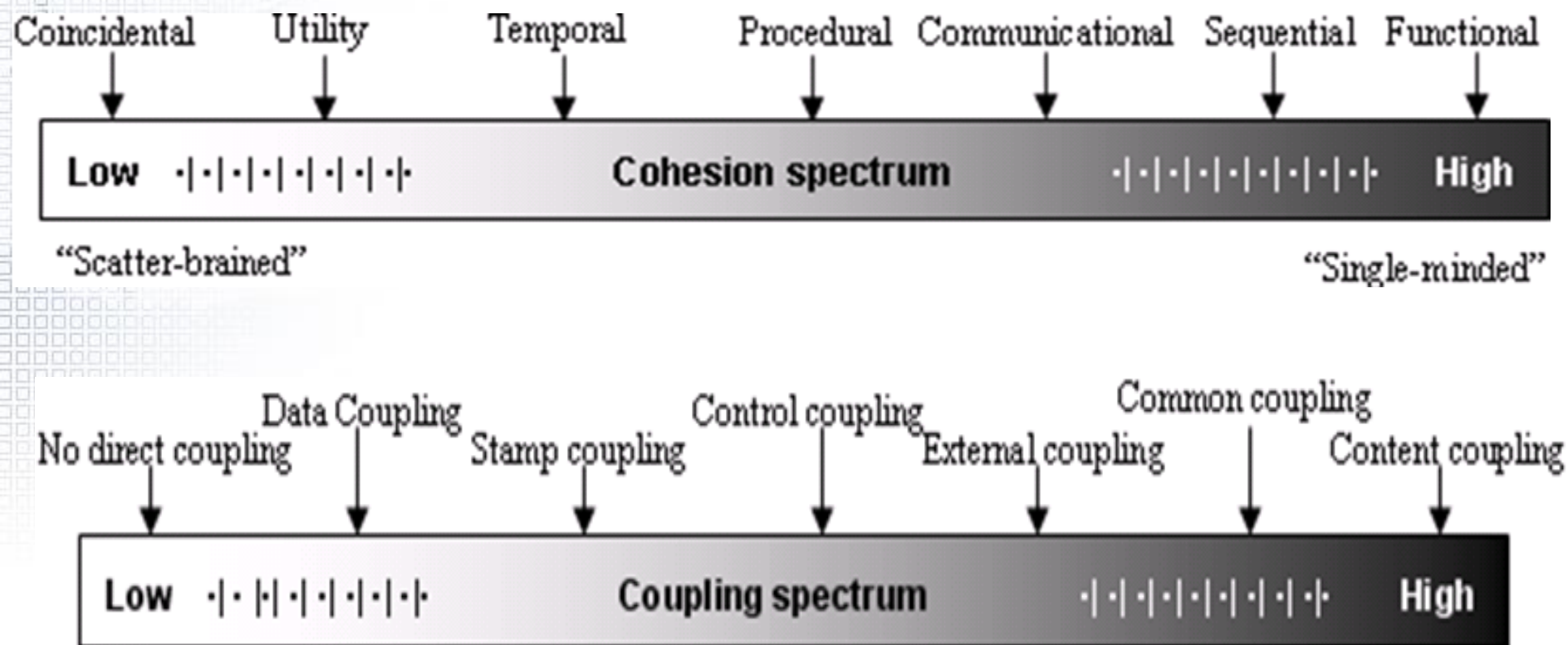
```
public class Order    {  
    public float calcTotalMoney(int userLevel,  
                                int  consumeScore)  
    {  
        ... .. //计算订单总金额  
    }  
    ... ..  
}
```

耦合练习



6.4 内聚和耦合

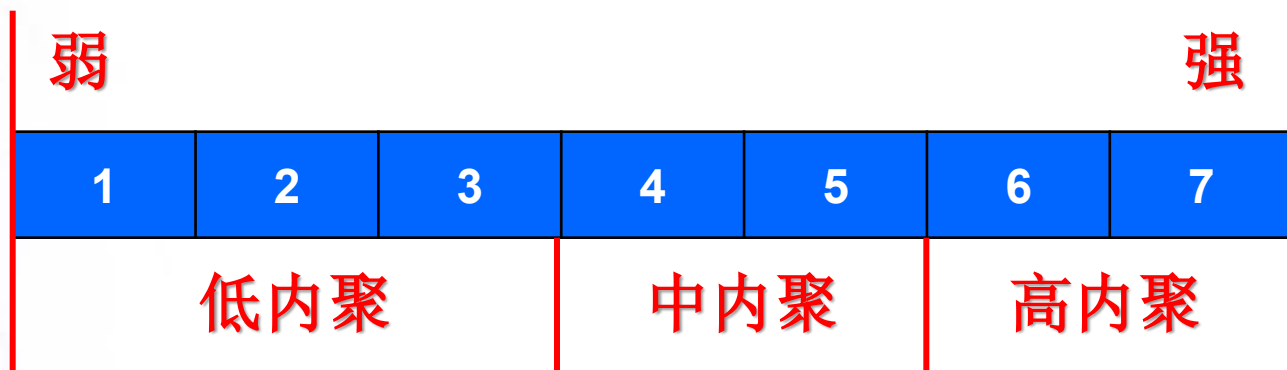
- 内聚谱系和耦合谱系





6.4 内聚

- 内聚

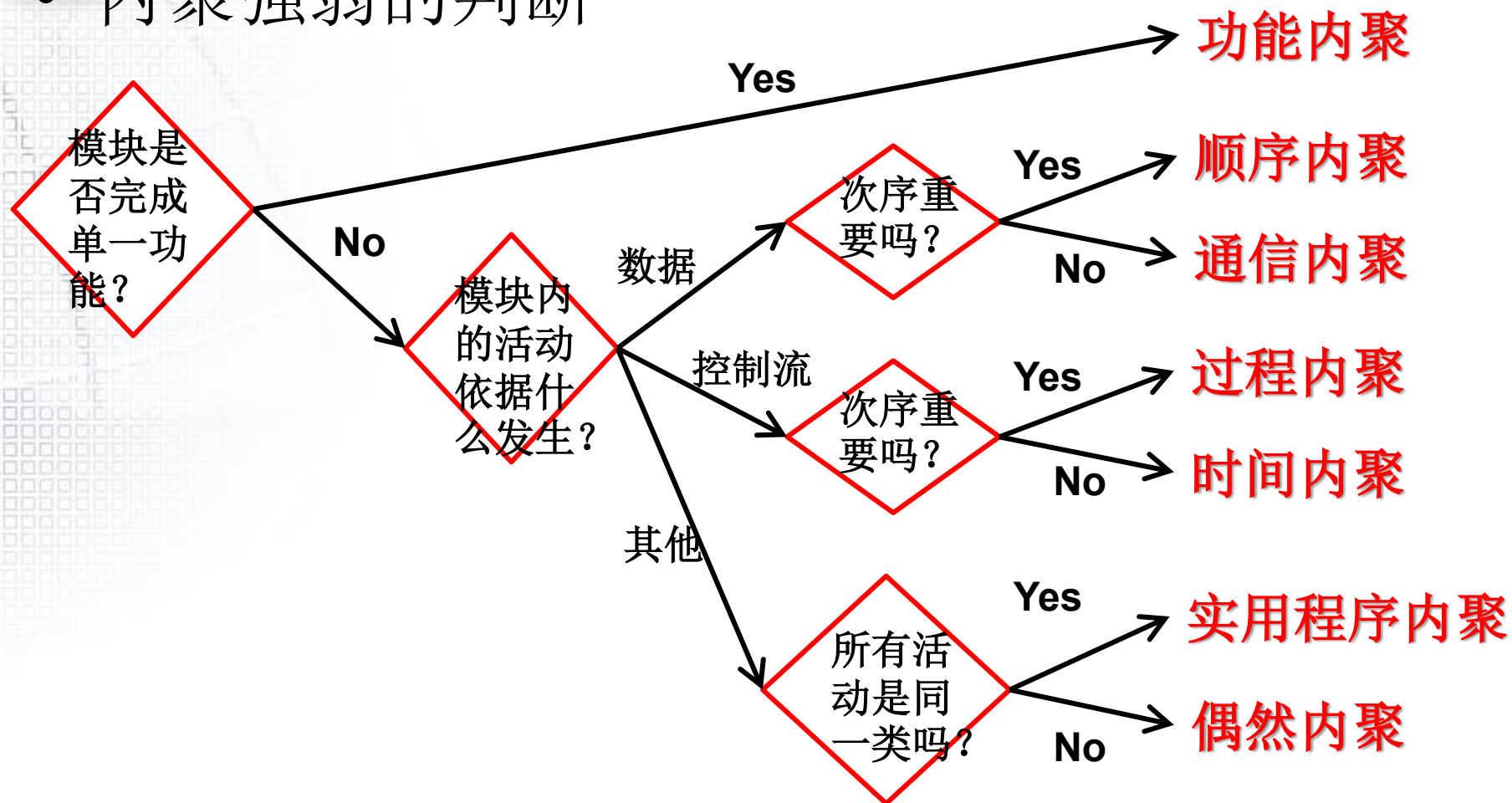


1	偶然内聚（ Coincidental Cohesion ）
2	实用程序内聚（ Utility Cohesion ）
3	时间内聚（ Temporal Cohesion ）
4	过程内聚（ Procedural Cohesion ）
5	通信内聚（ Communicational Cohesion ）
6	顺序内聚（ Sequential Cohesion ）
7	功能内聚（ Functional Cohesion ）



6.4 内聚

• 内聚强弱的判断





6.4 耦合

- 耦合

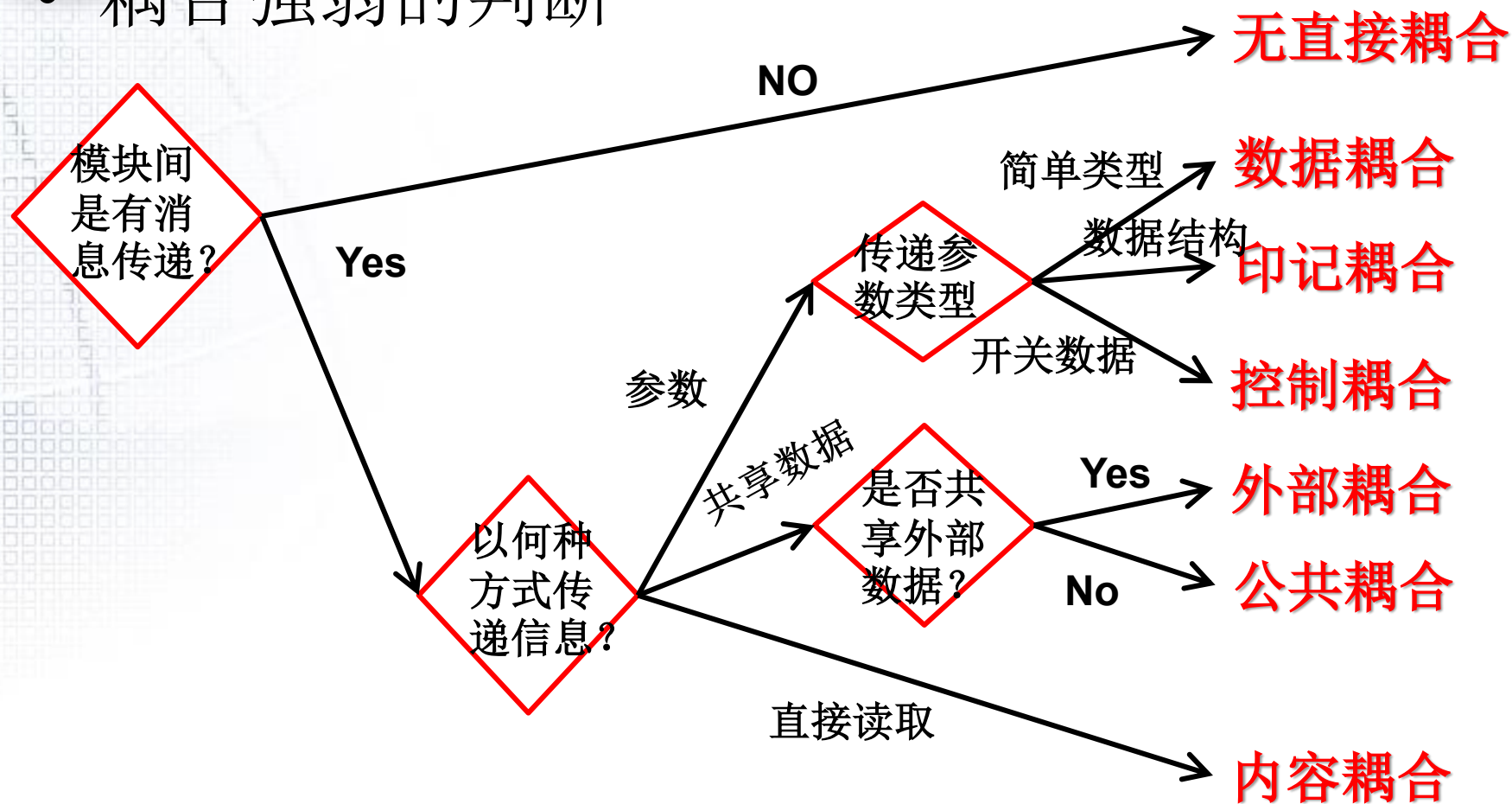
弱			强			
1	2	3	4	5	6	7
弱耦合			中耦合	强耦合		最强耦合

1	无直接耦合（No Direct Coupling）
2	数据耦合（Data coupling）
3	印记耦合（Stamp coupling）
4	控制耦合（Control coupling）
5	外部耦合（External coupling）
6	公共耦合（Common coupling）
7	内容耦合（Content coupling）



6.4 耦合

• 耦合强弱的判断





6.5 复用

- 提高软件生产率的办法
 - 改进过程
 - 改善需求分析方法
 - 改进设计方法
 - 改进工具
 -
 - 然而.....
 - 复用——最有效率的方法之一





优势展望

- 软件行业相关资料（摘录）
- 软件复用技术：
 - 软件产品开发周期缩短为原来的 $1/2 \sim 4/5$
 - 软件产品的缺陷密度减少至原来的 $10\% \sim 50\%$
 - 软件开发费用能减少 $15\% \sim 75\%$
 - 维护费用也大大降低



优势展望

- 软件复用程度的相关数据
 - 美国电报电信公司开发的电信支持系统起软件复用率达到**40%~92%**
 - 摩托罗拉公司在为编译器编写测试包时，复用率高达**85%**
 - **1984**年惠普**HP**公司就开发了可复用的组件；**1987**年建立了自己的软件复用库；软件复用率达到**10%~50%**
 - **1991**年的统计报告显示：日本软件公司其软件复用率达到**50%**



软件工程的一个新的发展方向-复用

- 应用系统的开发不再采用一切从“零”开始的模式，可以充分利用过去应用系统开发中积累的知识和经验
- 优点：
 - 提升软件开发速度和效率
 - 缩短开发周期
 - 提升软件产品质量
 - 降低开发成本



6.5.1 复用的概念

- 复用的定义
 - Biggerstaff和Ritcher:
 - 在新的开发项目中使用以前已获得的概念和对象
 - Tracz:
 - 特别为复用目的而设计的软件的过程
 - Freeman:
 - 是一种通过使用以前开发工作的某些东西来生产（或帮助生产）一个系统的过程，其关键问题是复用什么、什么是导致成功复用的过程



可复用的软件资产

- (1) 项目计划：软件项目计划的基本结构和许多内容，如**SQA**计划，均可以跨项目复用。
- (2) 可行性报告：相关类似领域项目的可行性报告可以参考。
- (3) 过程模型：当你发现问题背景、特点、工期、人员差不多类似时，可以参考已有的过程模型。
- (4) 成本估计：由于不同项目中常包含类似的功能，所以有可能在极少修改或不修改的情况下，复用对该功能的成本估计。
- (5) 需求模型和规格说明：数据流图、类模型等均可以复用。
- (6) 设计：系统和对象设计等是常见的复用成份。
- (7) 体系结构：即使应用领域千差万别，但程序和数据的体系结构很少有截然不同的情形。因此，有可能创建一组类似的体系结构模板，如事务处理结构，将这些模板作为可复用涉及的框架。



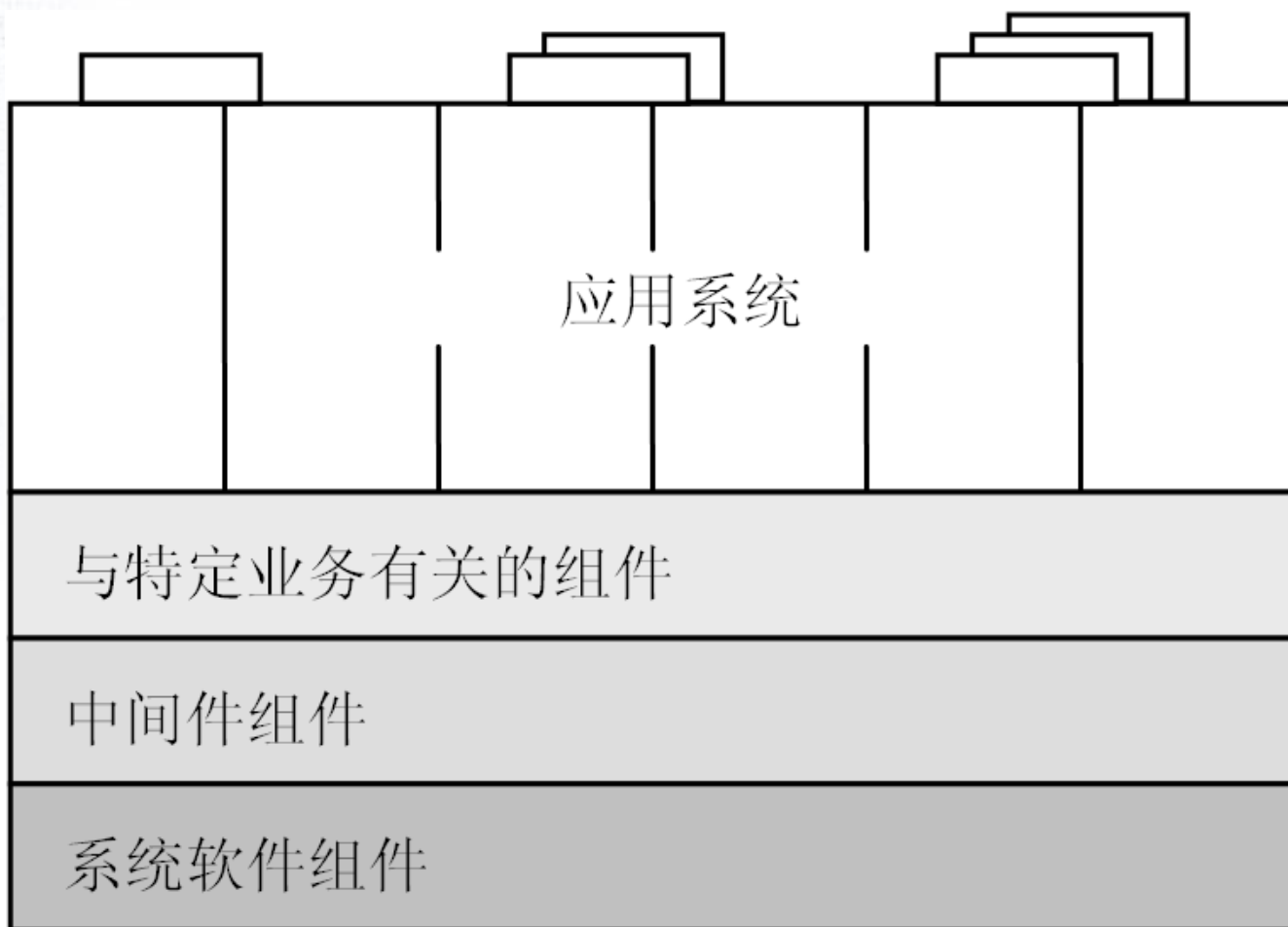
可复用的软件资产

- (8) 源代码
- (9) 数据：在大多数经常被复用的软件制品中，数据包括：内部表、列表和记录结构，以及文件和完整的数据库。
- (10) 用户界面：用户界面可能是最广泛地被复用的软件制品。由于它可能占一个应用程序的60%的代码量，所以复用的效果最明显。
- (11) 用户文档和技术文档：即使特定的应用不同，也有可能复用用户文档和技术文档中的大部分内容。
- (12) 领域知识：教育、金融、医疗、电子商务领域等有自己的特点。



6.5.1 复用的概念

- 一个典型的复用分层体系





6.5.1 复用的概念

- 涉及的内容
 - 复用的管理问题
 - 软件组织
 - 资产
 - 复用过程
 - 领域工程
 - 技术(可复用构件)
 - 复用经济学（兼容性、创造性、升级）
 -



6.5.1 复用的概念

- 复用的分类
 - 根据对可复用信息进行复用的方式
 - 黑盒复用和白盒复用
 - 按照复用的规模
 - 小规模复用与大规模复用
 - 按照复用方法
 - 组合式复用和生成式复用
 - 按照软件复用所应用的领域范围
 - 横向复用和纵向复用
 - 按复用的构件的物理形式
 - 子程序、类、组件、Web服务组件等



一代码公开(版权)

一 复杂

- 改动，而不是继承(版本)

— ● ● ● ● ● ■

一、复用记录

```

try
    OMR_GetProcHandle();
    N := OMR_Installed(OMR_GetDeviceNum('自动检测'));
except
end;
if N = 0 then
begin
    Result := false;
    Application.MessageBox('OMR机型设置不正确或没有打开电源，OMR驱动未安装。', 'OMR驱动未安装', MB_OK);
end
else
begin
    InitOMRFlag := true;
    //调用格式文件
    OMR_clear;
    if FileExists(Is_OMRFileName) then
    begin
        wRet:=OMR_LoadForm(pchar(Is_OMRFileName));
        if wRet<>0 then
        begin
            case wRet of
                2: MessageDlg('格式缓冲区已满',mtInformation,[mbOK],0);
                3: MessageDlg('格式文件名不正确',mtInformation,[mbOK],0);
                4: MessageDlg('文件格式不正确',mtInformation,[mbOK],0);
            end ;
        end;
    end;
end;
3: MessageDlg('格式文件名不正确',mtInformation,[mbOK],0);
4: MessageDlg('文件格式不正确',mtInformation,[mbOK],0);
end ;

```

try	begin
Omr_Ge	In
N := 0	//
except	Om
end;	if
	be

```

try
    Omr_Get
    N := 0
except
end;
if N = 0
begin
    Res
    App
end

```

```

try
    Omr_Ge
    N := 0
except
end;
if N = 0
begin
    Res
    App
end
else
begin
    Ini
    //if
    Omr
    if
    beg

```

```
begin
case wRet of
2: MessageDlg('格式缓冲区已满',mtInformation,[mbOK],0);
3: MessageDlg('格式文件名不正确',mtInformation,[mbOK],0);
4: MessageDlg('文件格式不正确',mtInformation,[mbOK],0);
end ;
```

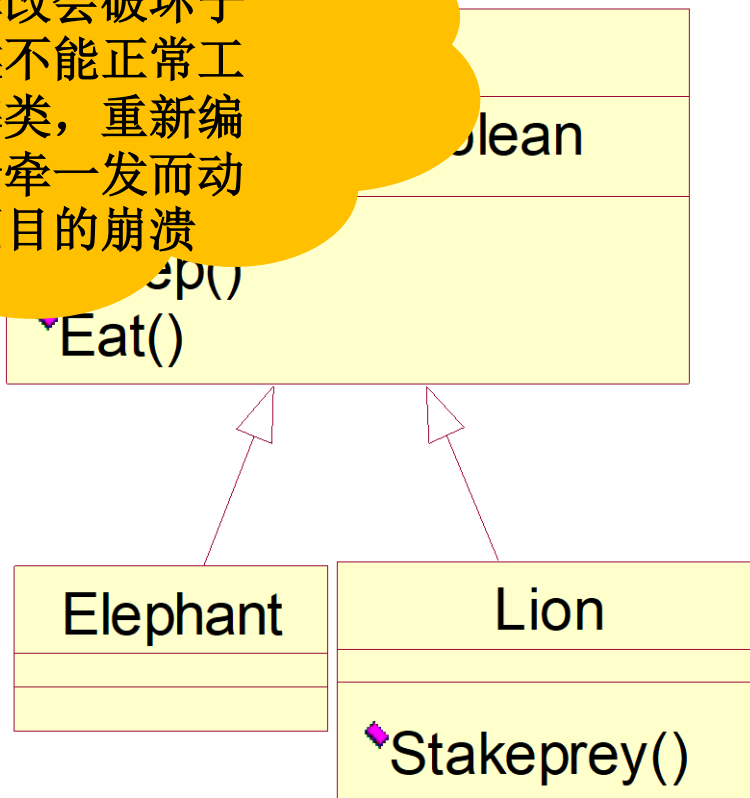


6.5.2 复用技术

- 面向对象的复用：继承、封装

- 源码复用
- 依赖语言
- 粒度较小
- 脆弱的基类
- 不可独立部署
- 第三方市场不够繁荣

基类很小的修改会破坏子类，导致子类不能正常工作，即更改基类，重新编译再发布，会牵一发而动全身，导致项目的崩溃





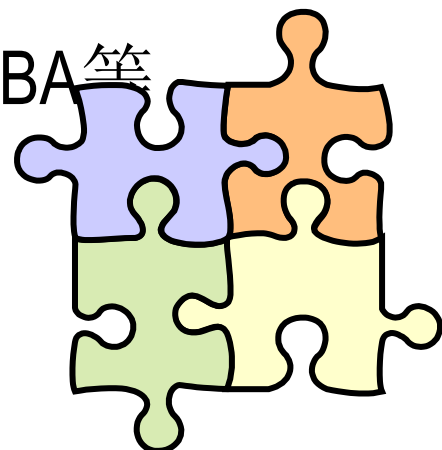
6.5.2 复用技术

- 基于组件的复用(概念)

- 组件概念:

基于接口的程序设计

- 组件(Component)是指有定义完备接口的、明确规定了上下文依赖关系的合成单元，它可以被第三方用不同语言开发，并且能够被独立的部署，它具有自包含的属性，其内部构造和特征不可见
 - 典型代表：EJB、COM、.NET和CORBA^等是主要的组件模型





6.5.2 复用技术

- 基于组件的复用(不足)
 - Internet(协议、防火墙)
 - 跨平台
 - 组件环境
 - 部署和升级
 - 远程服务需要实例化





6.5.2 复用技术

- 基于Web服务的复用

- Web服务概念

- 平台独立、松耦合、自包含、可编程的Web应用程序（通过Web接口在网络上提供的某个功能的**组件**）
 - 基于开放的XML标准描述、发布、发现、协调和配置这些应用程序，用于开发分布式互操作的应用程序
 - 术语：HTTP，SOAP，WSDL，UDDI，XML.....

- 开发商：应用程序越来越变得松散耦合

- 被分解为多个组件
 - 被分布在多个计算机上

- Web服务提供商：服务变得更专业



6.5.2 复用技术

- 基于Web服务的功能复用
 - 优点
 - 避免了组件的不足(前页)
 - 更简单
 - 使用者，而非所有者
 - 充分利用第三方的技能和经验
 - 版权和版本
 - 不足
 - 速度
 - 开放问题
 - Web服务太多，如何发现？选取？



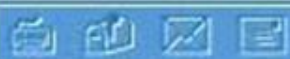


6.5.3 案例

- 宠物商店系统
 - 各种角色（如顾客、管理员、商店经理等）要登录到系统中，需要填写用户名、口令
 - 为了防止非法入侵，还要填写验证码。验证码的相关代码相对复杂，涉及到验证码图片的生成和验证码的校验。很多网站都有类似的功能。



PetShop



用户名

密码

验证码



登录

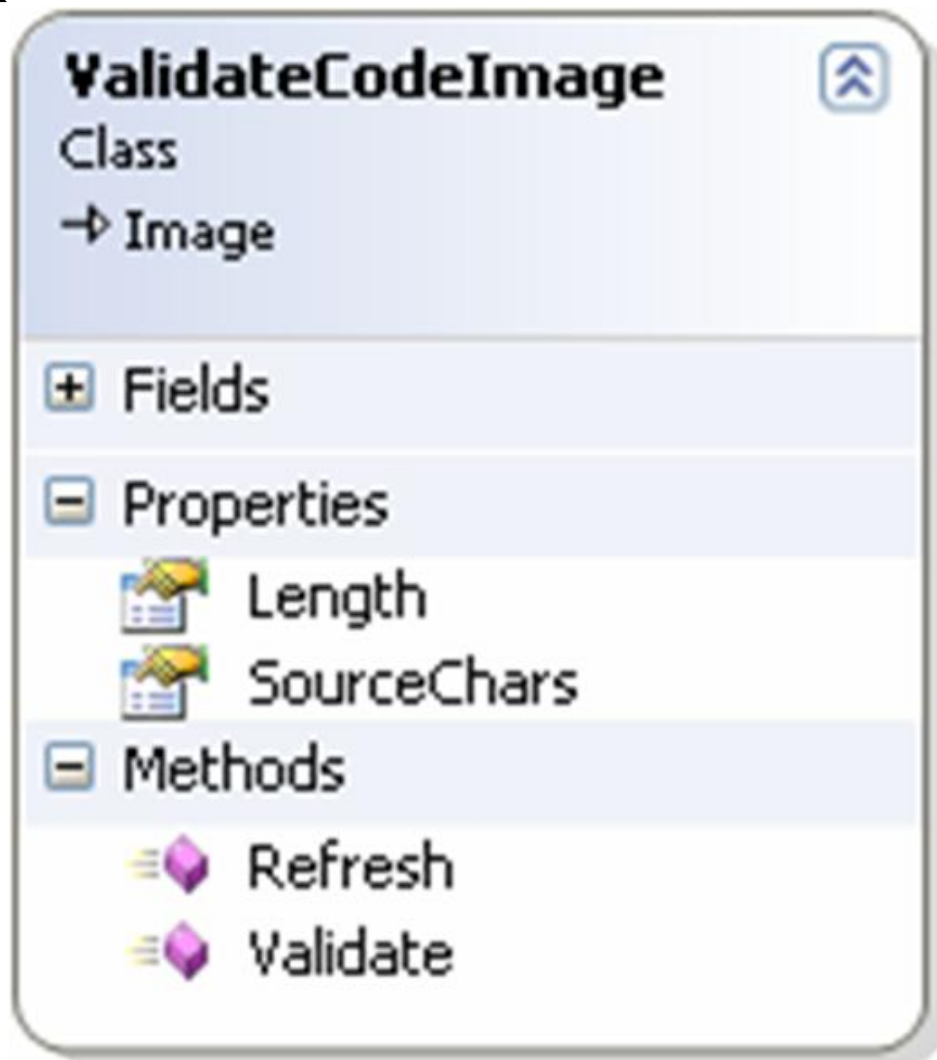
注册新用户

忘记密码?



6.5.3 案例 (3/9)

- 验证码控件





6.5.3 案例

- 验证码控件

Length	5
SkinID	
SourceChars	0123456789
TabIndex	0
ToolTip	
Visible	True
Width	209px

用户名

密码

验证码





6.5.3 案例

- 验证码控件

Length	6
SkinID	
SourceChars	GuoJun123
TabIndex	0
ToolTip	
Visible	True
Width	209px

用户名

密码

验证码





6.5.3 案例

- 判断验证码是否正确

```
if (!ValidateCodeImage1.Validate(textValidateCode.Text))  
{  
    lblErr.Text = "验证码错误， 请重新填写";  
    .....  
}
```



6.5.3 案例

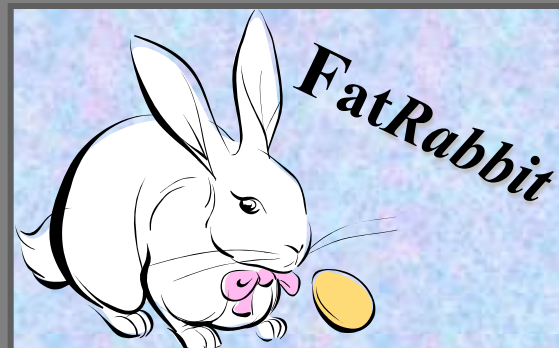
ValidateCodeService

Class

→ WebService

Methods

- CreateDomainGlossaryPic
- CreateGiantPic
- CreateNumberPic
- CreatePoemPic
- CreateRandomPic
- CreateSongPic
- CreateSpecialPic
- SetDomainGlossary
- Validate



.....



6.5.3 案例

- 调用

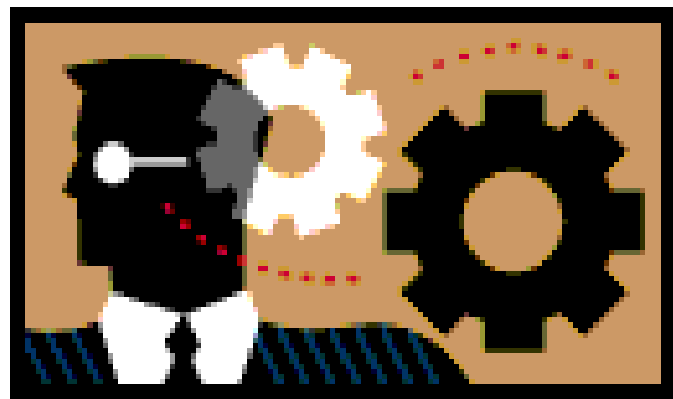
```
public void Refresh()
{
    //产生一个验证码字符串ValidateCode
    ....
    //调用Web服务，生成图片
    MemoryStream strmBuffer = new
        MemoryStream(PicService.CreateSpecialPic(ValidateCode));
    Bitmap obj= new Bitmap(strmBuffer);
    obj.Save(FileName,ImageFormat.Jpeg);
    ...
}
```



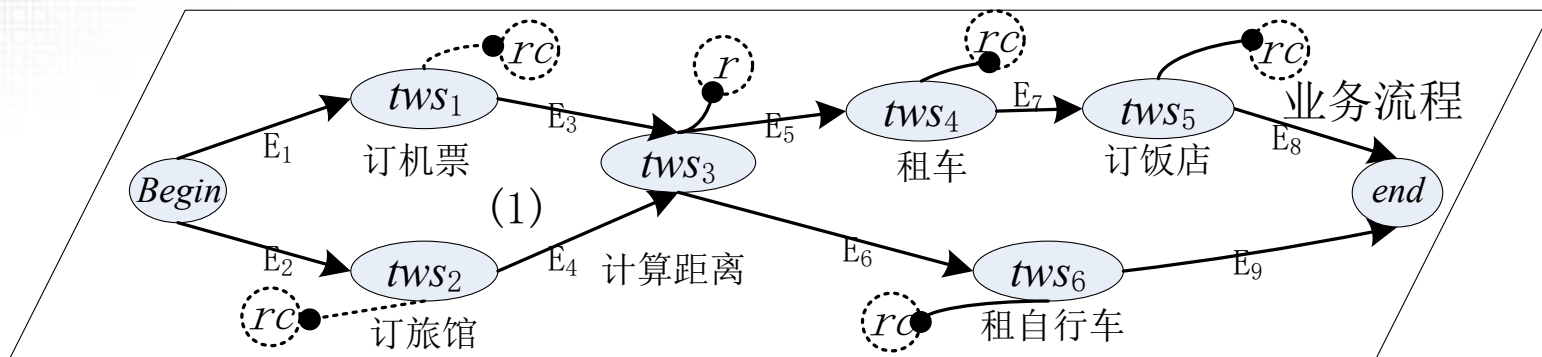
6.5.3 案例

扩展思维

- 站在服务提供商的角度
 - 考虑图像Web服务
- 站在服务使用者的角度
 - 考虑面向服务的架构(SOA)
- 站在复用工程师的角度
 - 考虑可复用构件的管理



—





6.6 设计描述方法

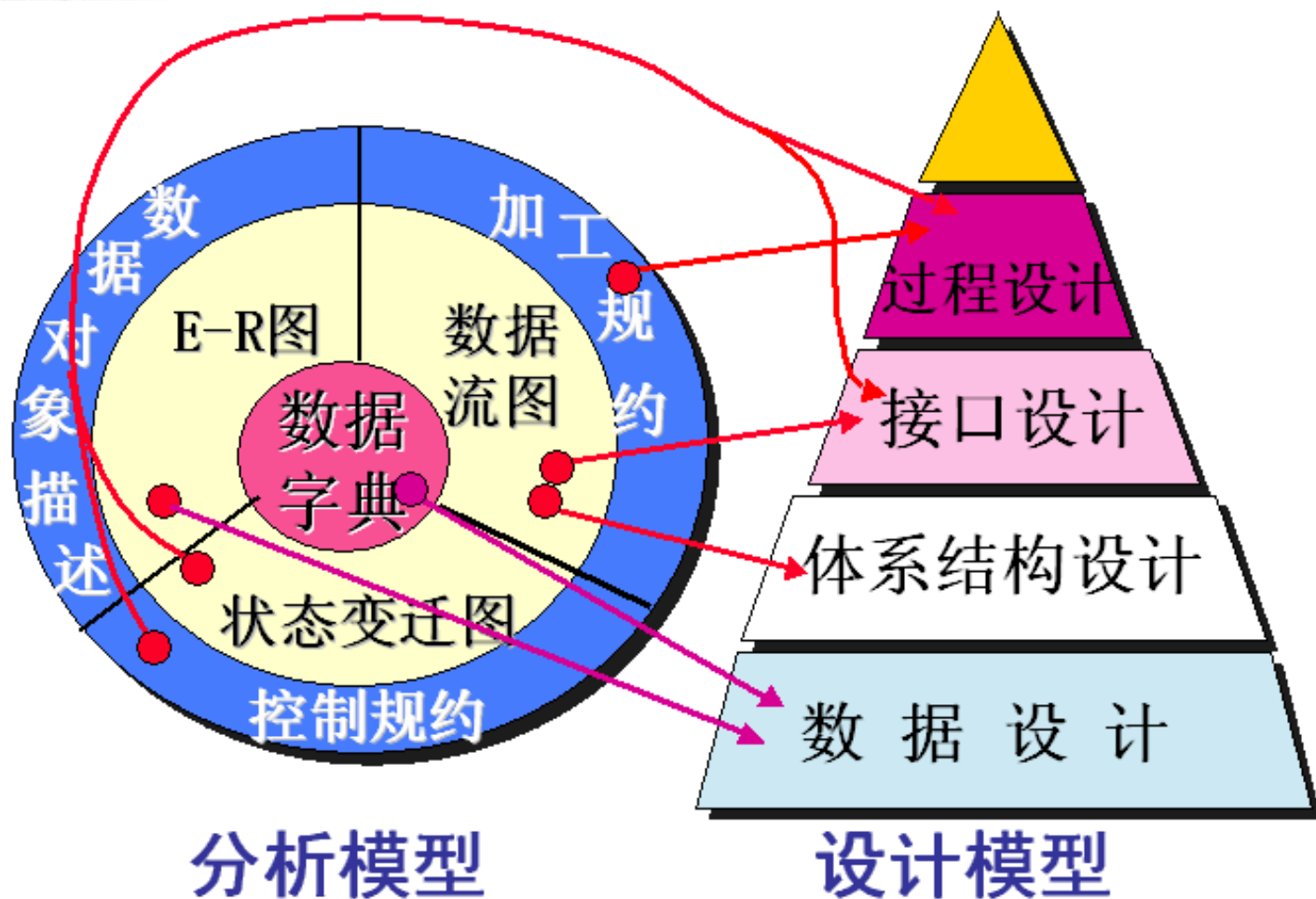
- 设计模型

- 原因：完全使用文字描述的方法来表达软件复杂的设计结果，往往会造成语义模糊、过于冗长、不够抽象等问题
- 模型更易于被快速的编档、理解和执行
- 学术界和企业界
 - 传统：层次图、结构图、HIPO(Hierarchical Input Process Output)图、ER图、程序流程图、盒式图、PAD(Problem Analysis Diagram)图、PDL
 - OO：包图、类图、构件图、部署图



6.6 设计描述方法

- 分析模型到设计模型的转换





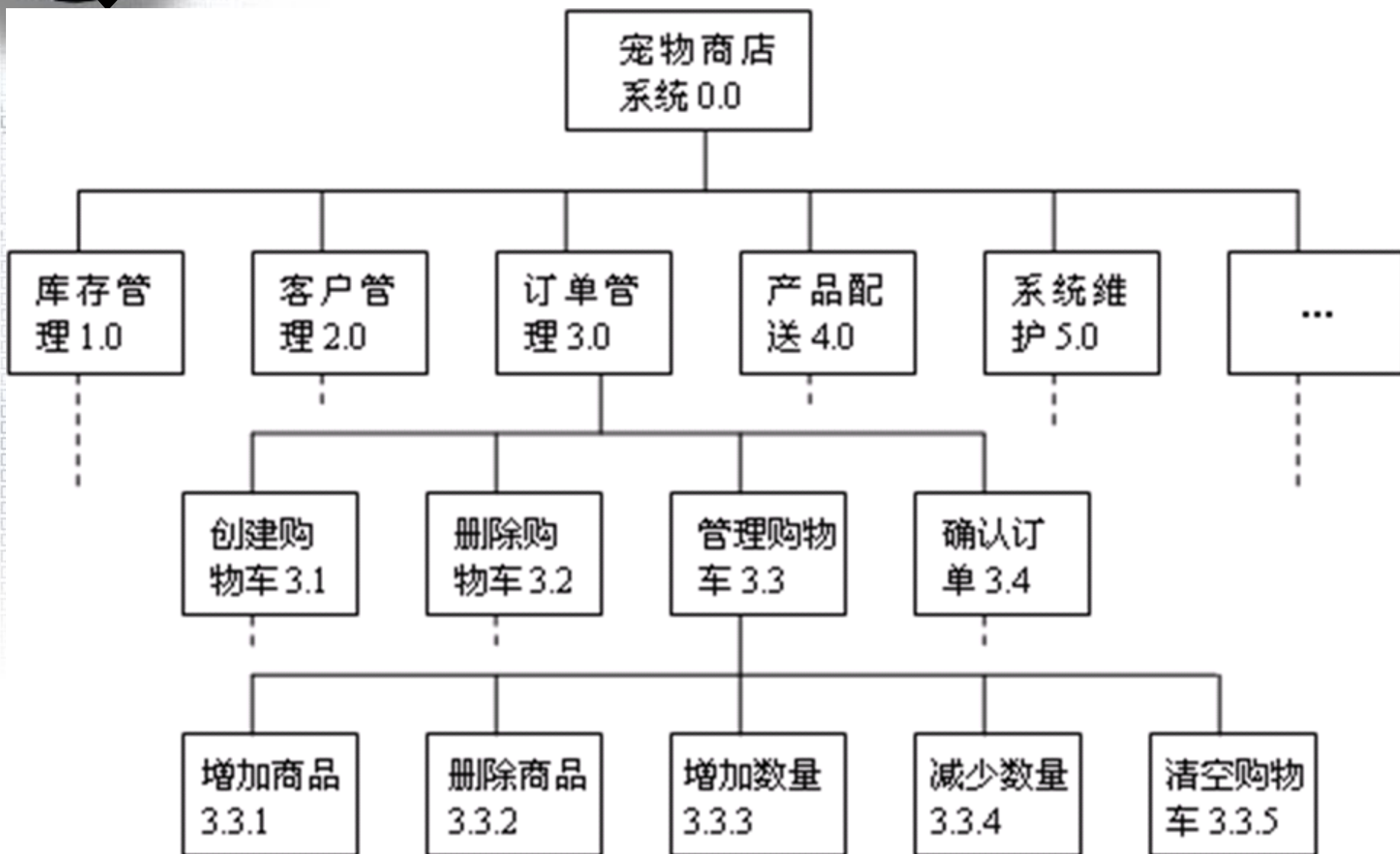
6.6.1 传统的设计描述方法

- 1) 层次图

- 用于描述程序的组成，即组成系统的程序模块及其调用关系
- 能够清楚地表明系统的结构，并可用来粗略地估计系统的尺寸
- 广泛地应用 ✓



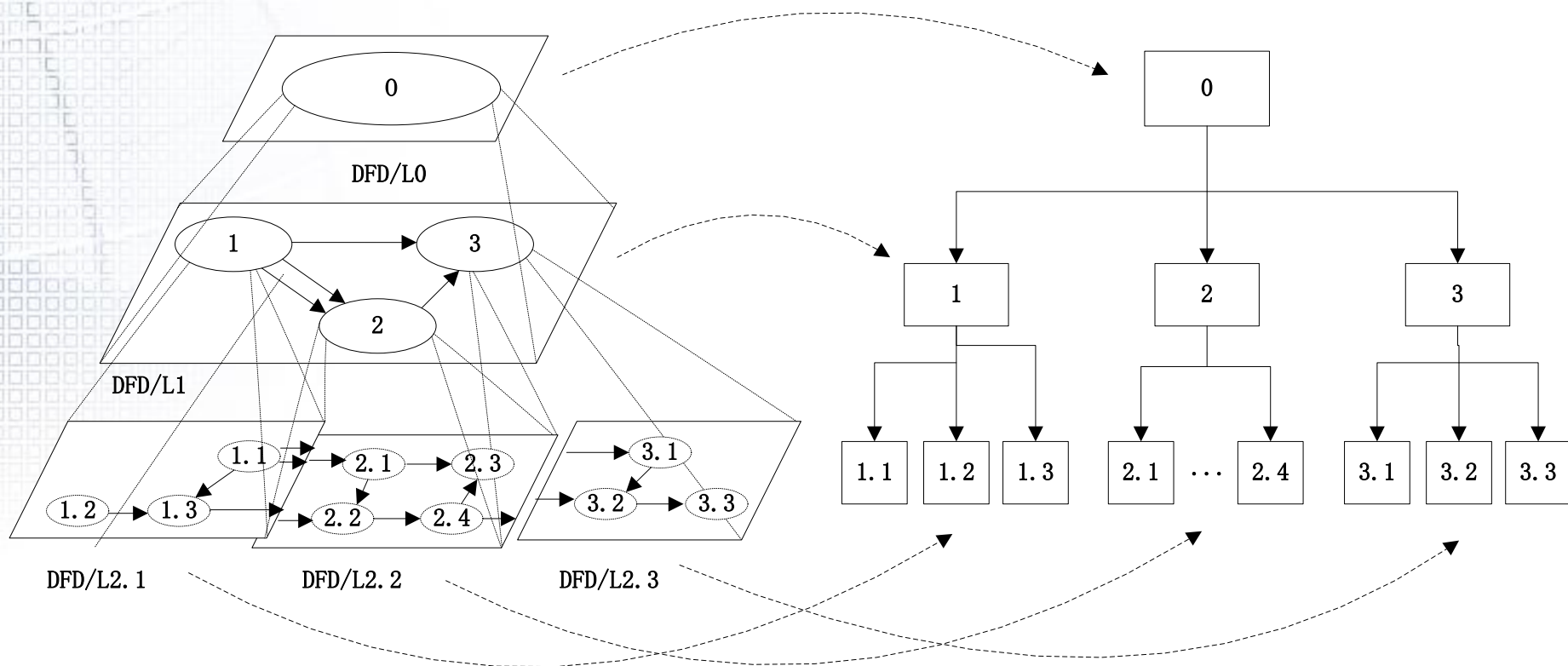
6.6.1 传统的设计描述方法





6.6.1 传统的设计描述方法

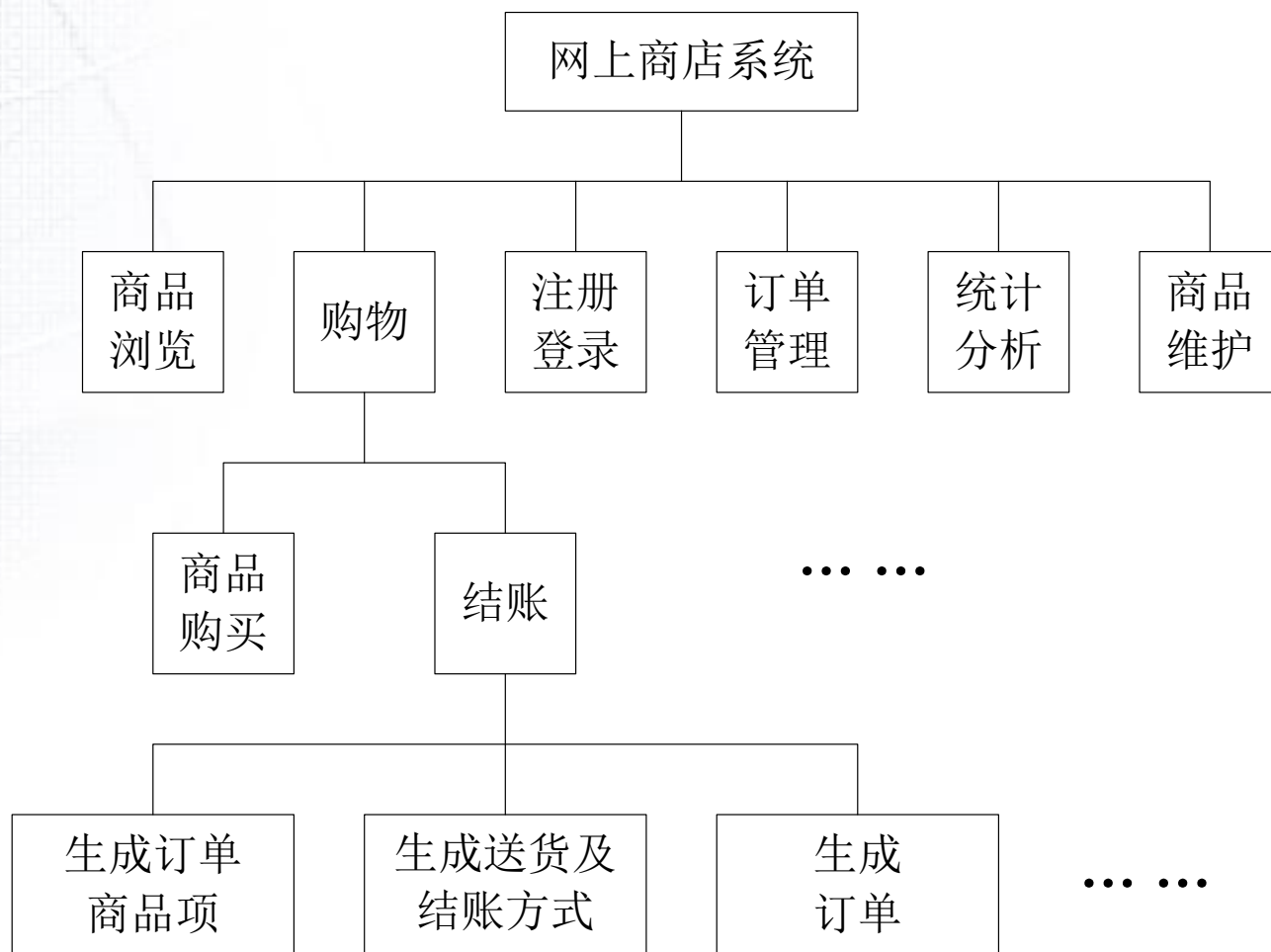
- 数据流程图转层次图——一一对应法





6.6.1 传统的设计描述方法

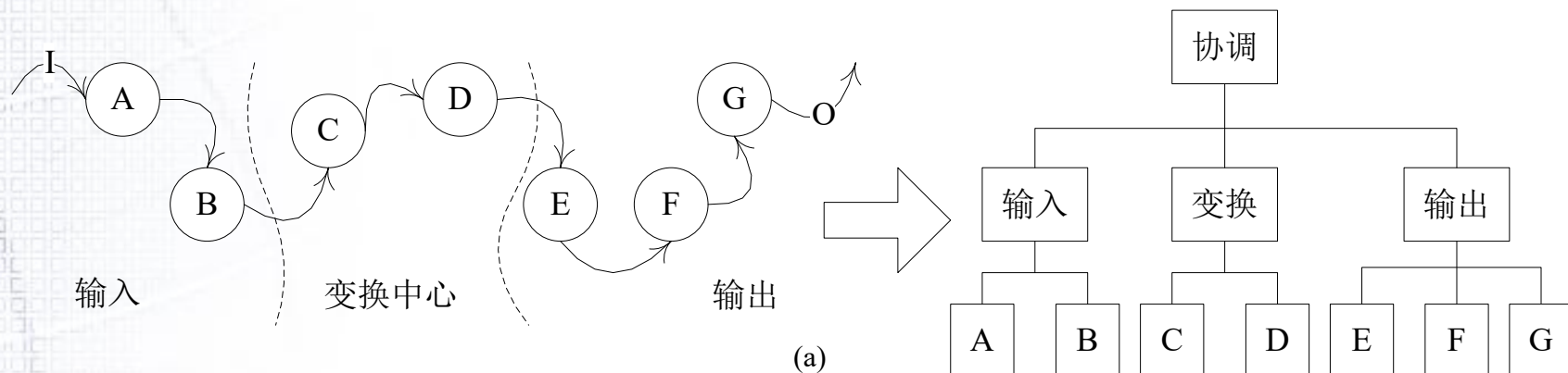
- 数据流程图转层次图——一一对应法



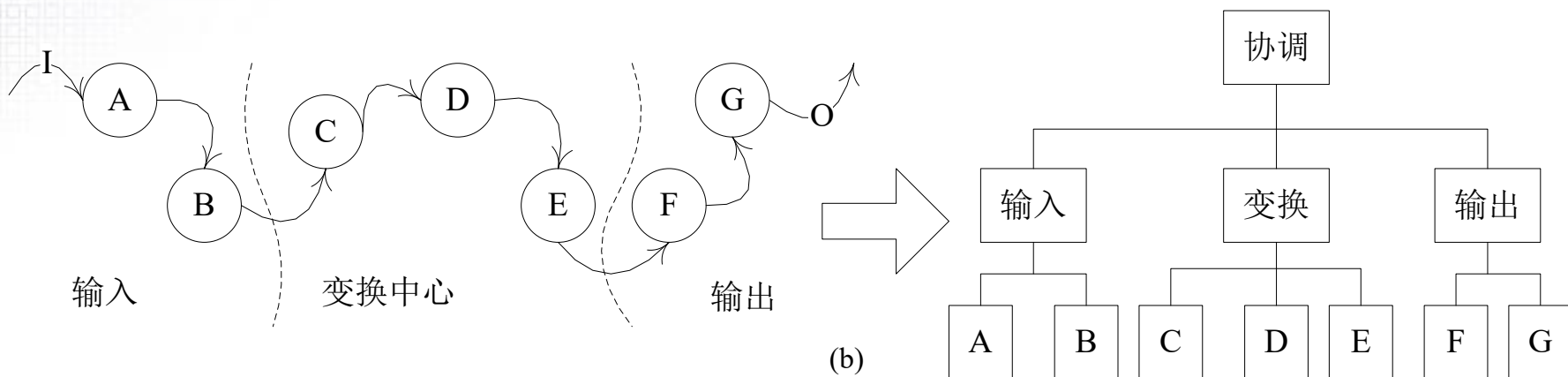


6.6.1 传统的设计描述方法

• 数据流程图转层次图——基于变换流法



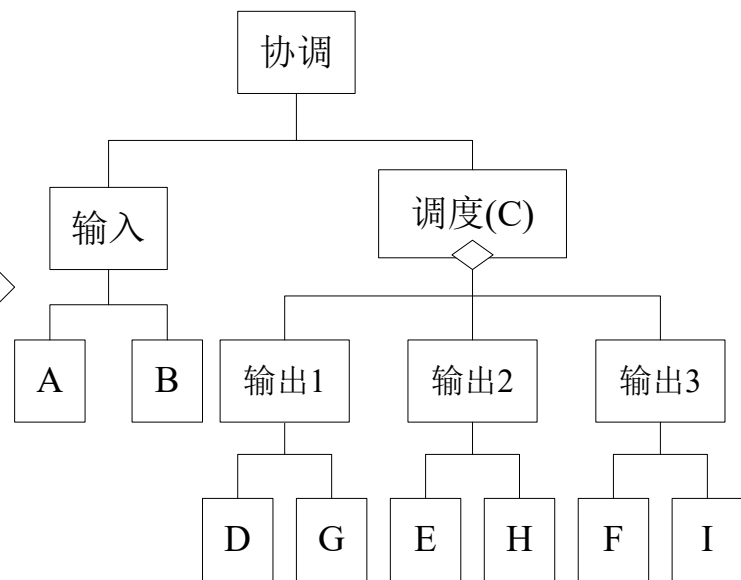
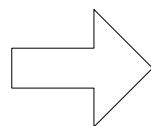
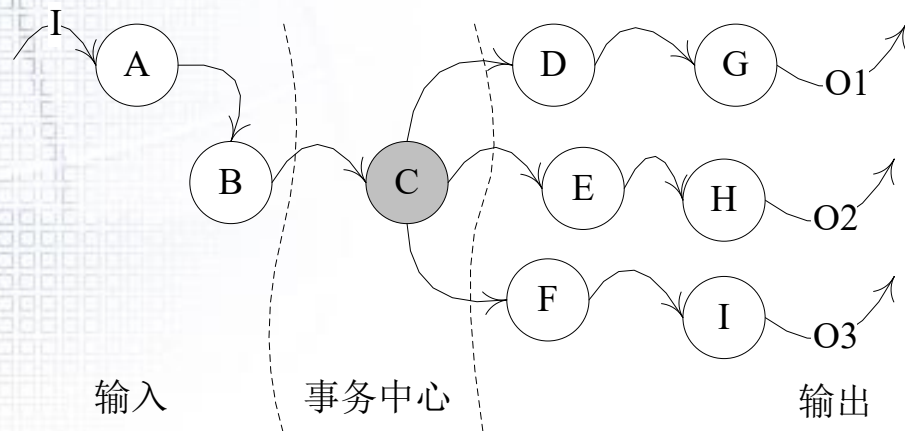
同样流程不同变换方法





6.6.1 传统的设计描述方法

- 数据流程图转层次图——基于事务流法

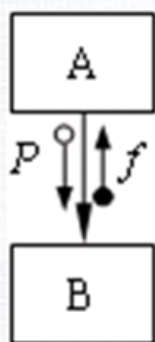




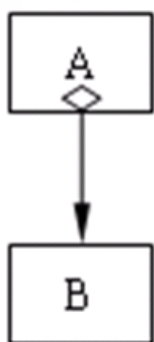
6.6.1 传统的设计描述方法

- 2) 结构图

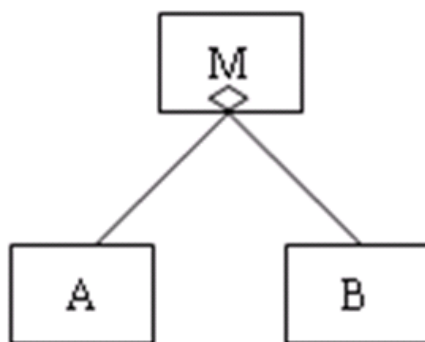
- 表明模块之间的相互作用关系



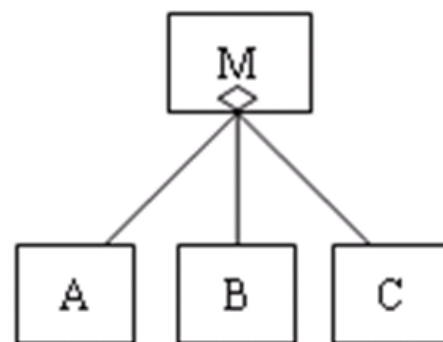
(a)



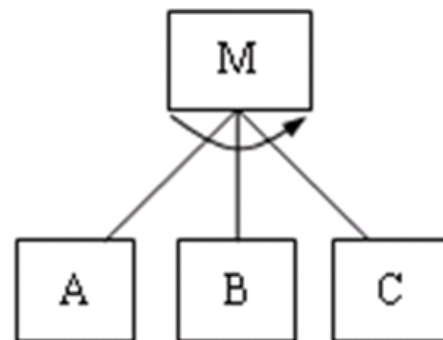
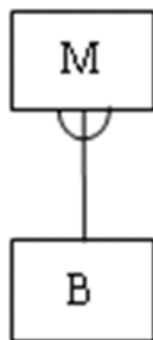
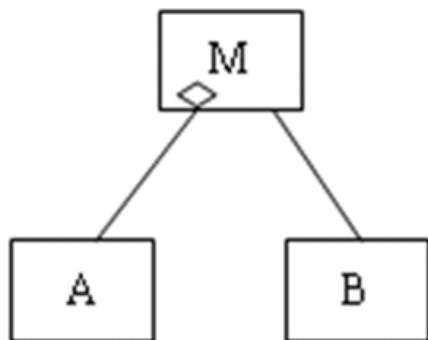
(b)



(c)



(d)

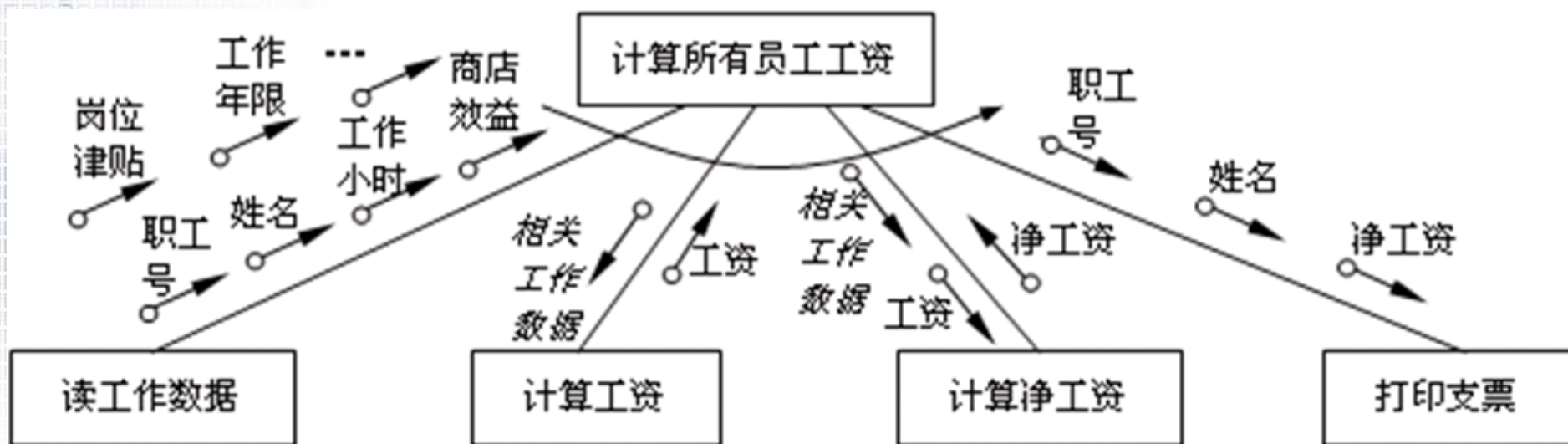




6.6.1 传统的设计描述方法

- 2) 结构图

- 简单的工资计算程序的结构图

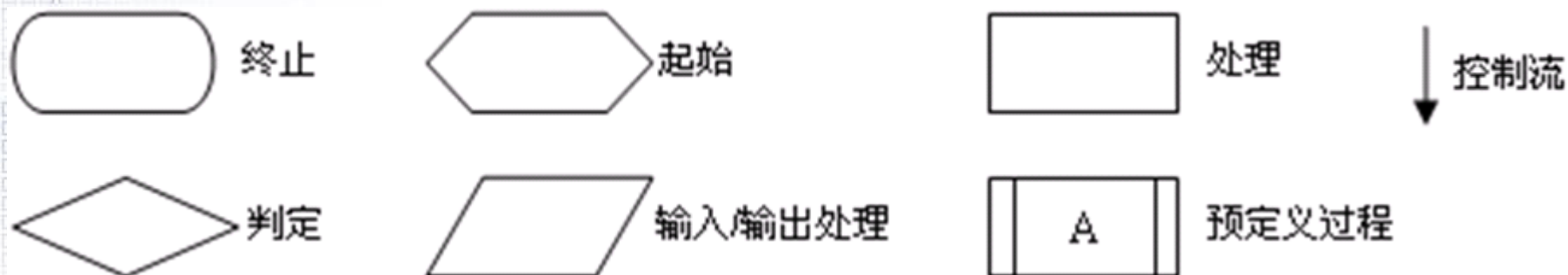




6.6.1 传统的设计描述方法

- 3) 程序流程图

- 又称程序框图，是详细设计中广泛使用的工具之一

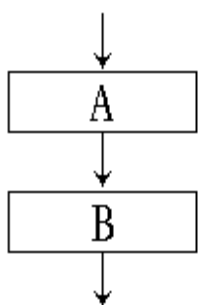


- 直观、形象、容易理解

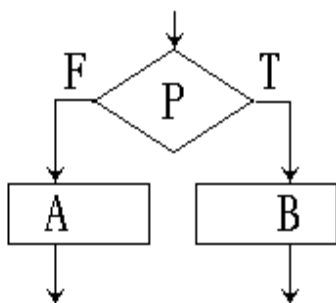


6.6.1 传统的设计描述方法

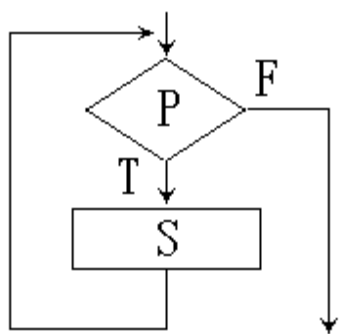
• 程序流程图(续)



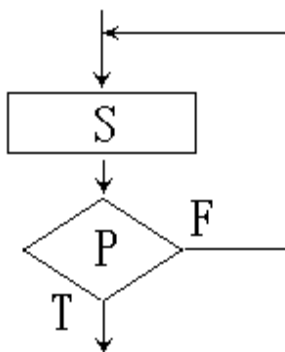
① 顺序型



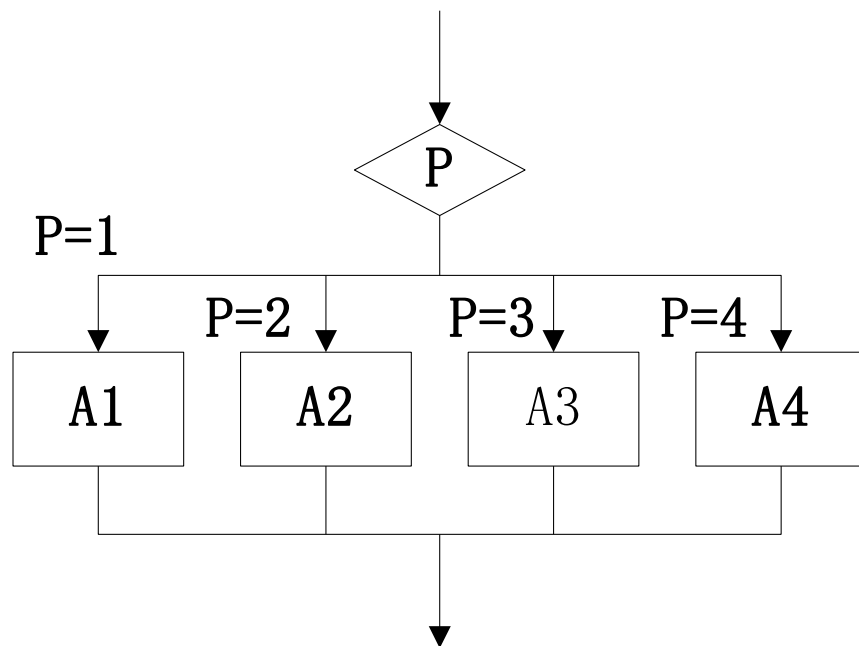
② 选择型



③ 先判定型循环
(DO-WHILE)



④ 后判定型循环
(DO-UNTIL)



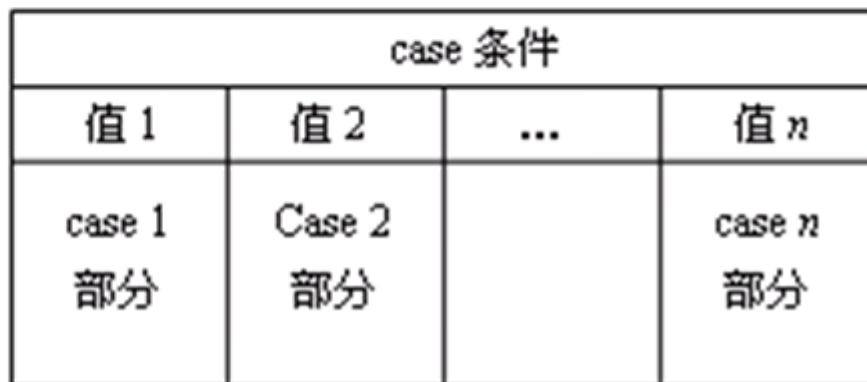
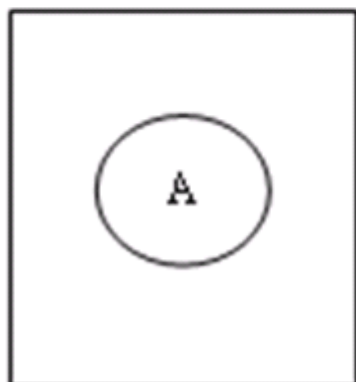
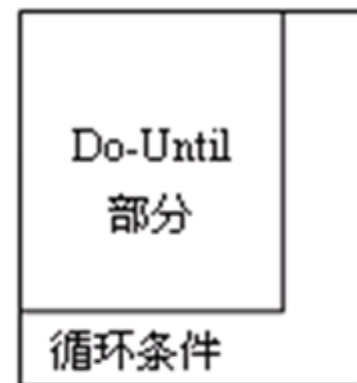
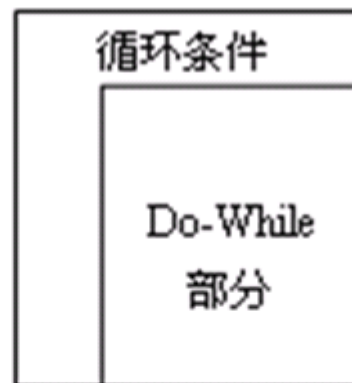
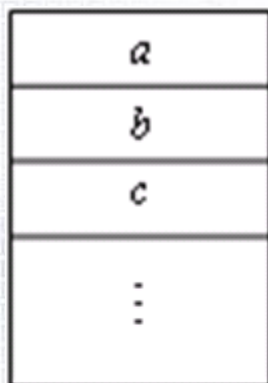
⑤ 多情况选择型
(CASE 型)



6.6.1 传统的设计描述方法

- 4) 盒式图

- Nassi和Shneiderman提出





6.6.1 传统的设计描述方法

- 5)PDL(Procedure Description Language)
 - 又称伪码，用英文if...then-else...end if和while...do...end do等表示控制结构的保留字及用设计者本土语言所构成的算法的描述

```
连接数据库；  
打开数据库的员工表；  
执行SQL语句：  
Select * from 员工表 while UserName=@username, Password=@password;  
if (员工表存在相关记录) then  
    转移到主页面；  
else  
    转移到报错页面，通知用户用户名或秘密错；  
.....
```



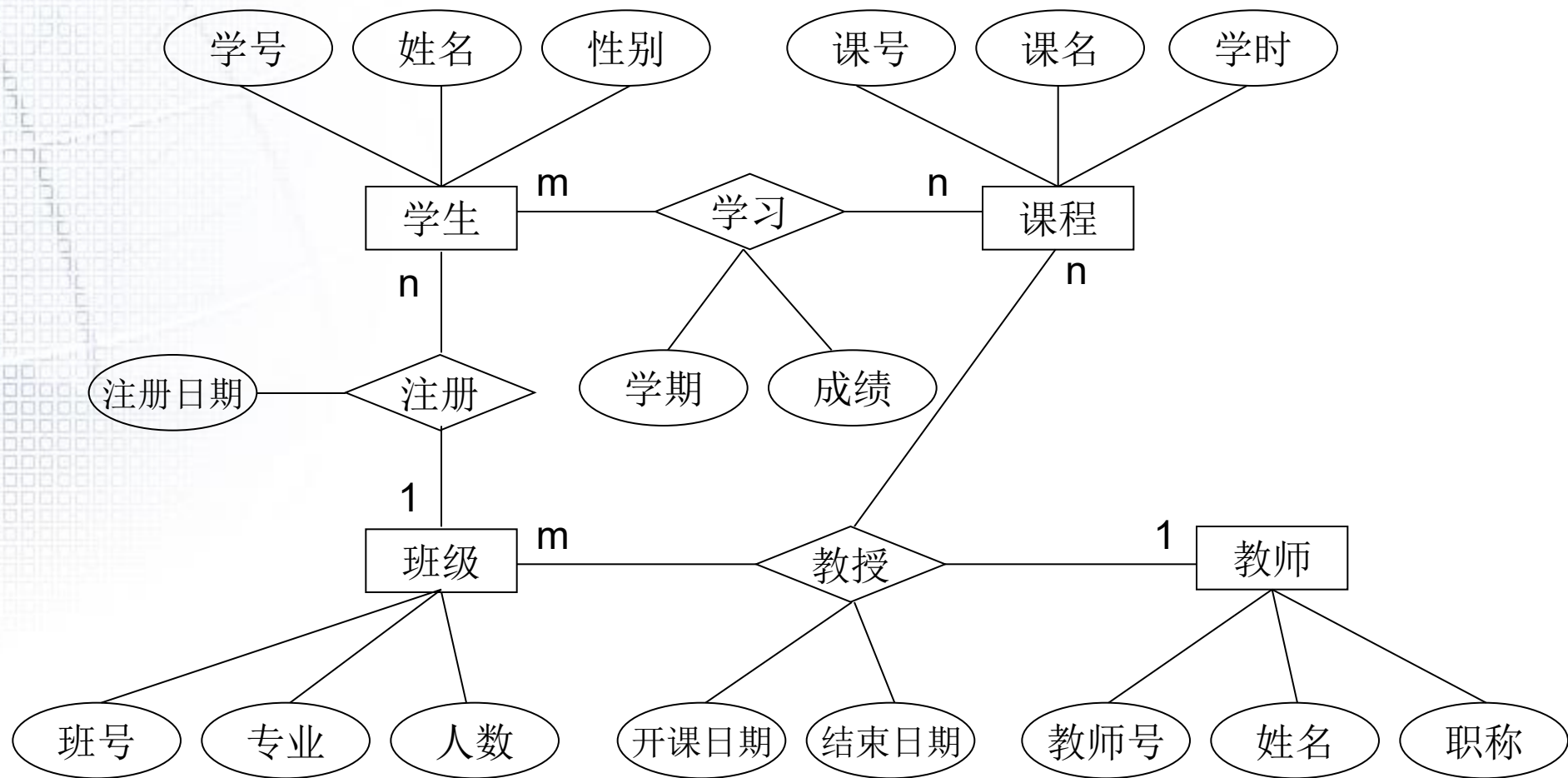
6.6.1 传统的设计描述方法

- 6)E-R图

- 实体联系图，用以表示应用领域中的实体及其相互关系
- 通过E-R图可以很快转成关系数据库
- 概念
 - 实体：一切事物，如操作员、顾客、宠物等
 - 联系：事物间关系
 - 属性：客观实体和联系实体所拥有的性质
- 广泛地应用✓



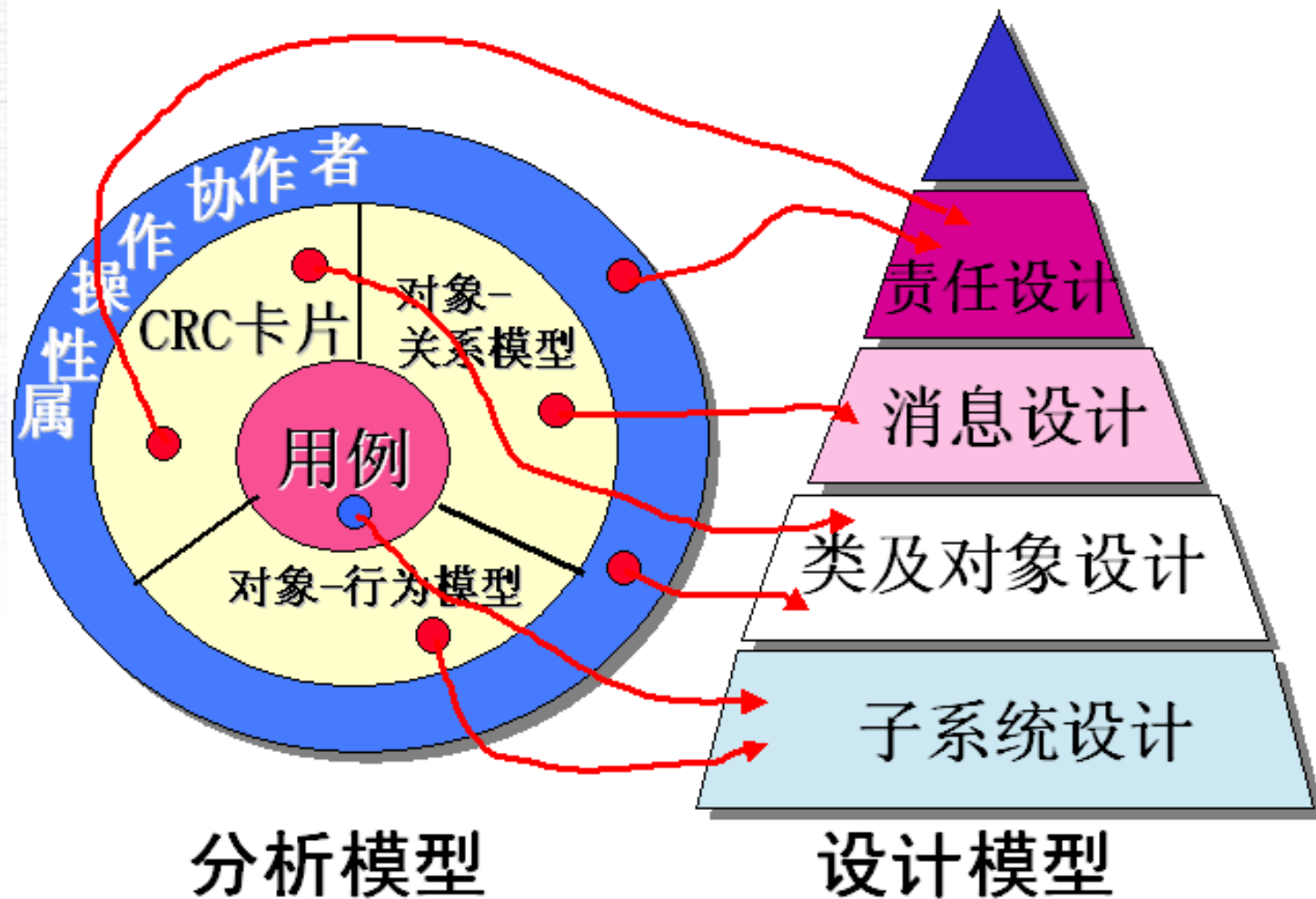
传统的设计描述方法





6.6.2 OO的设计描述方法

- 分析模型到设计模型的转换

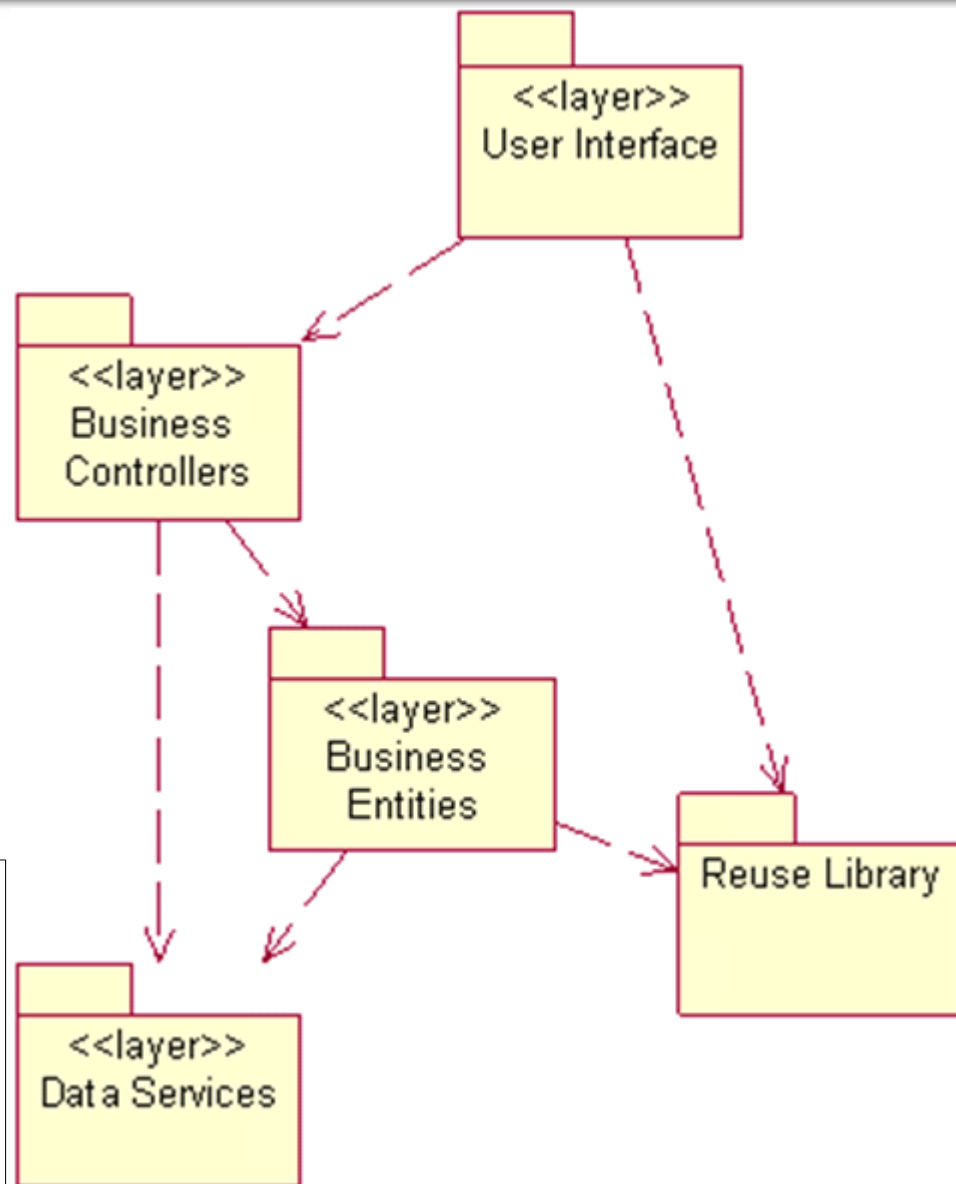
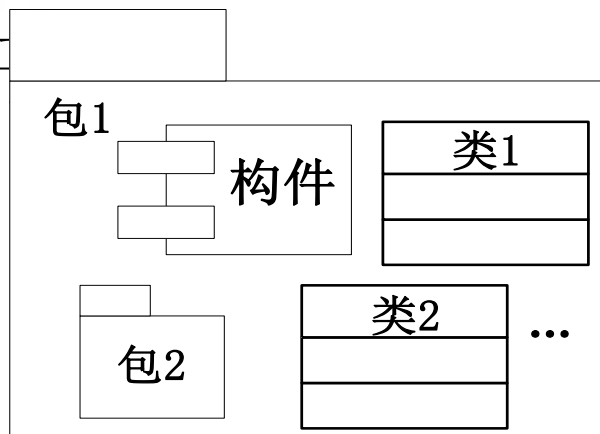




6.6.2 OO的设计描述方法

- 1)包

- 把元素组织成组的通用机制
- 元素包括?
- 为何分包?
- 表示方法?
- 例



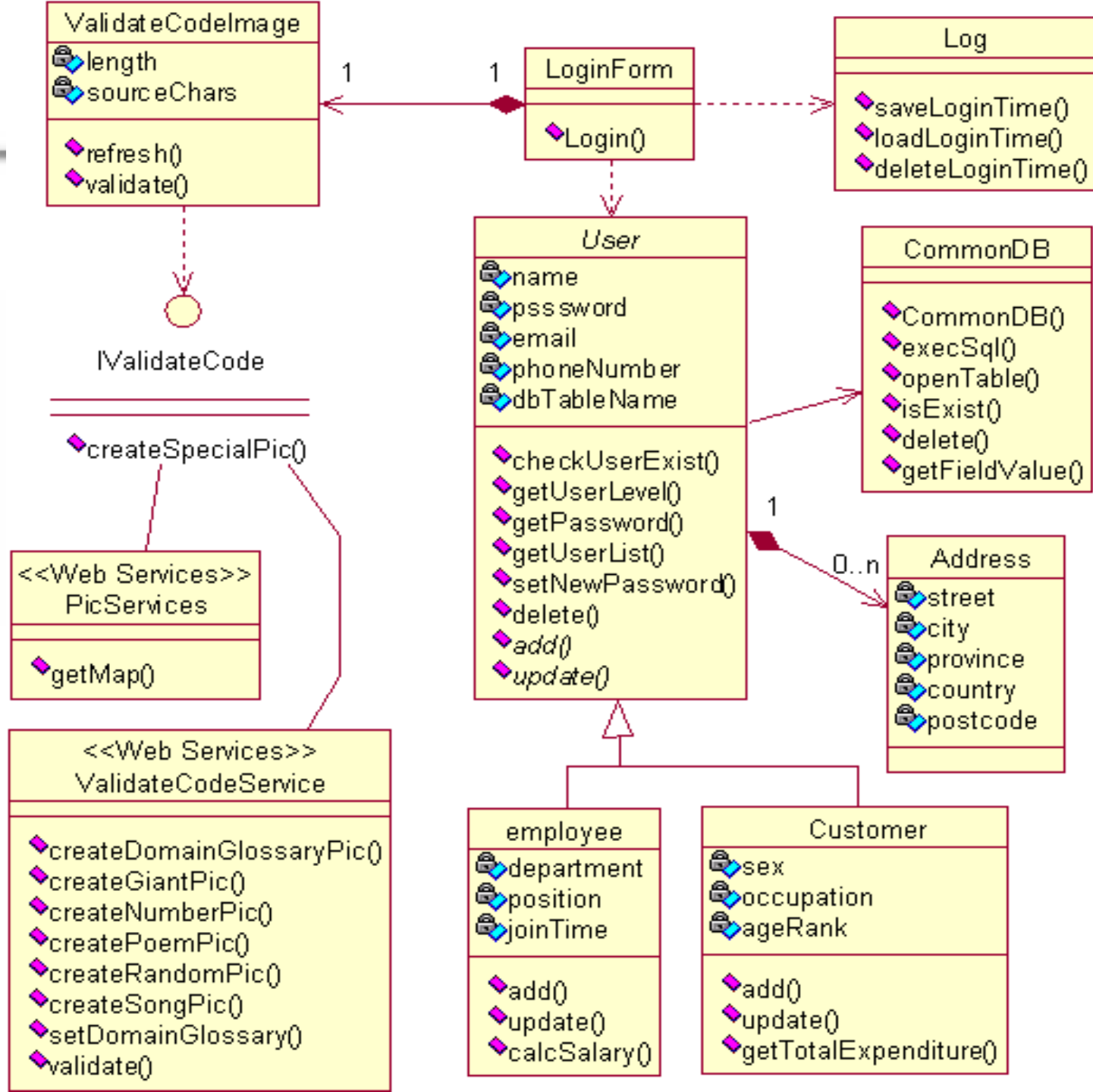


6.6.2 OO的设计描述方法

- 2)类图
 - 类加上它们之间的关系就构成了类图
 - 设计原则
 - 设计模式



• 2)类图 — 例子

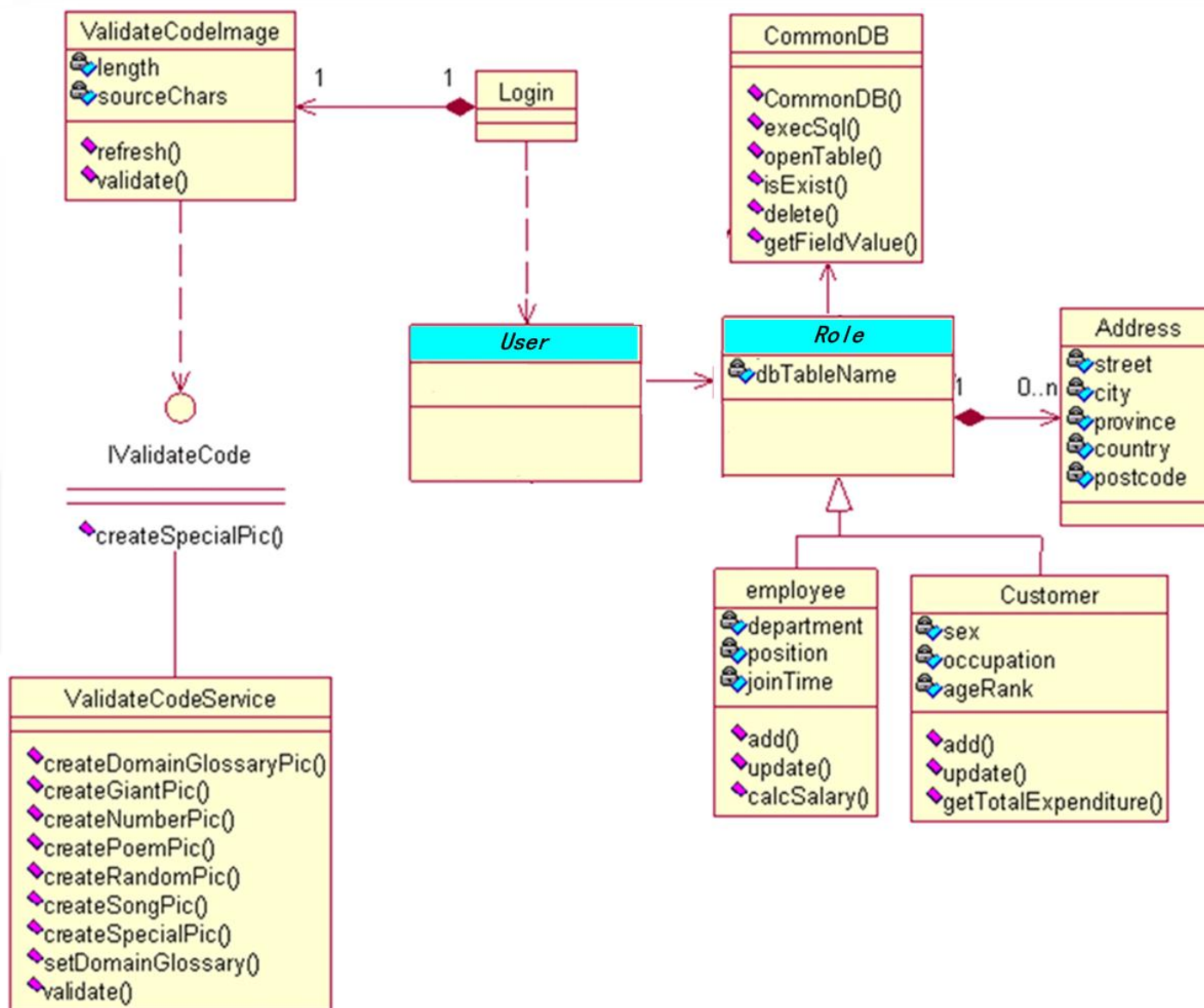




6.6.2 OO的设计描述方法

- 改进1
—应用开闭原则

见青色背景

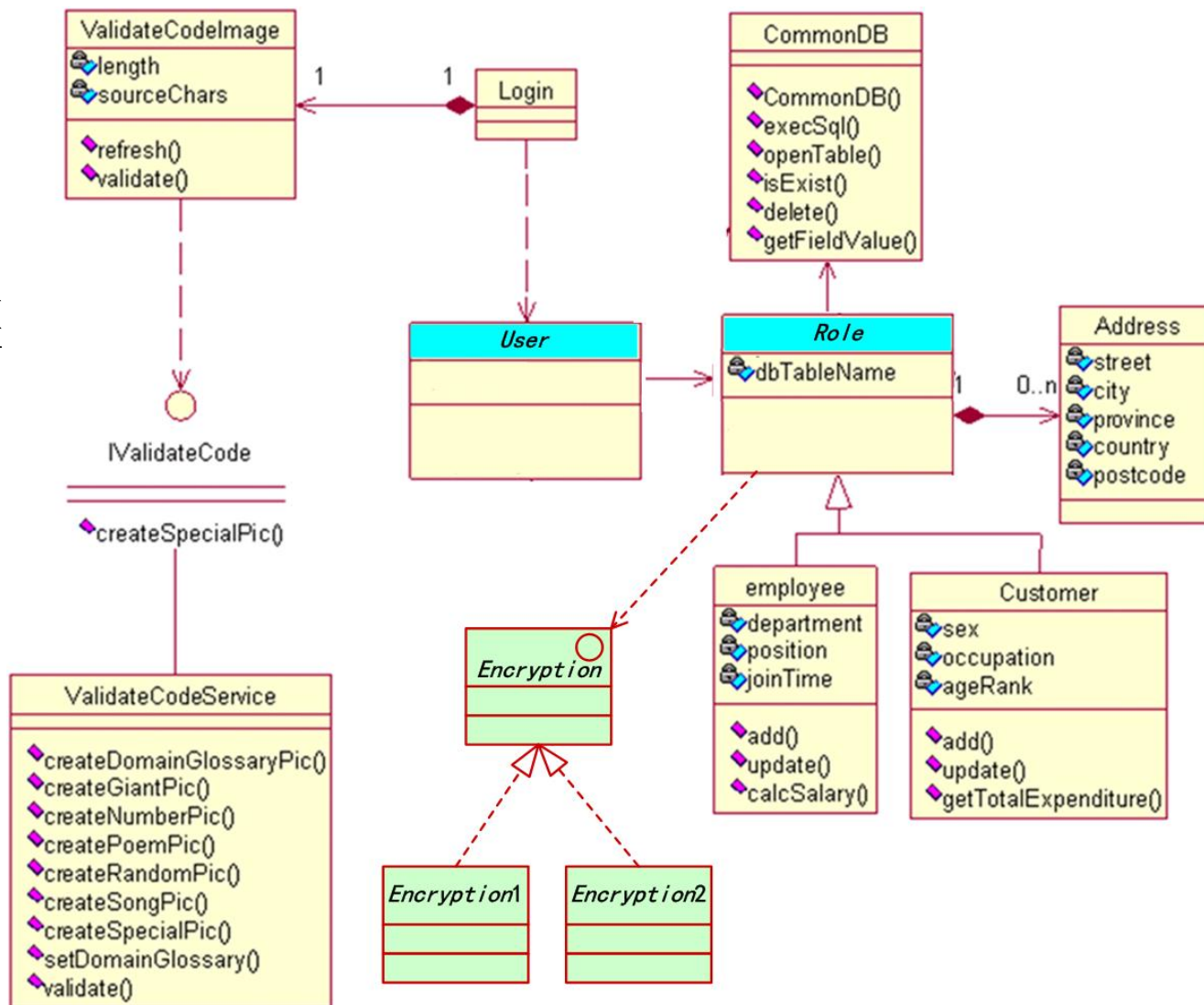




6.6.2 OO的设计描述方法

- 改进2
— 再添加
接口和类

见浅绿色
背景





6.6.2 OO的设计描述方法

- 再改进
— 头脑风暴?

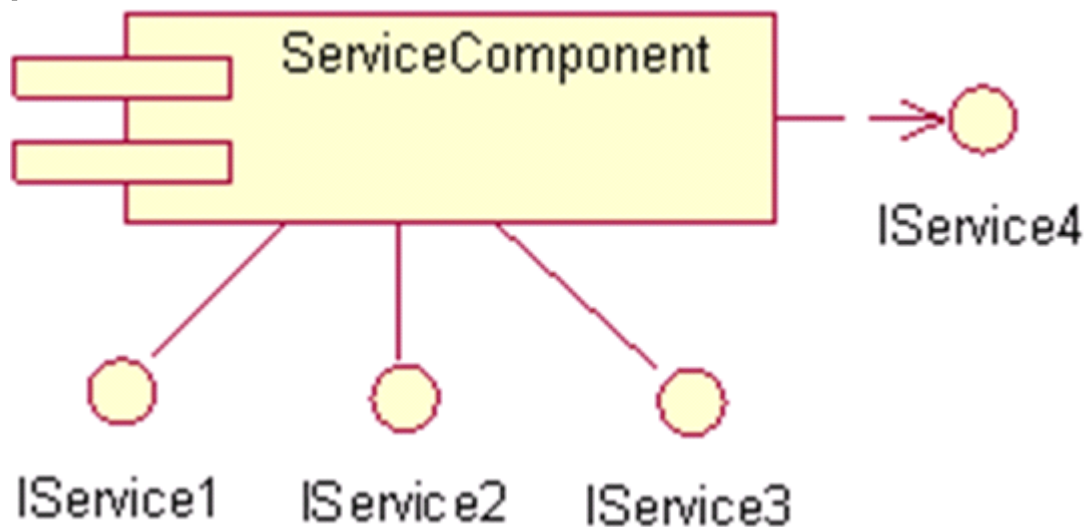




6.6.2 OO的设计描述方法

• 3) 构件图

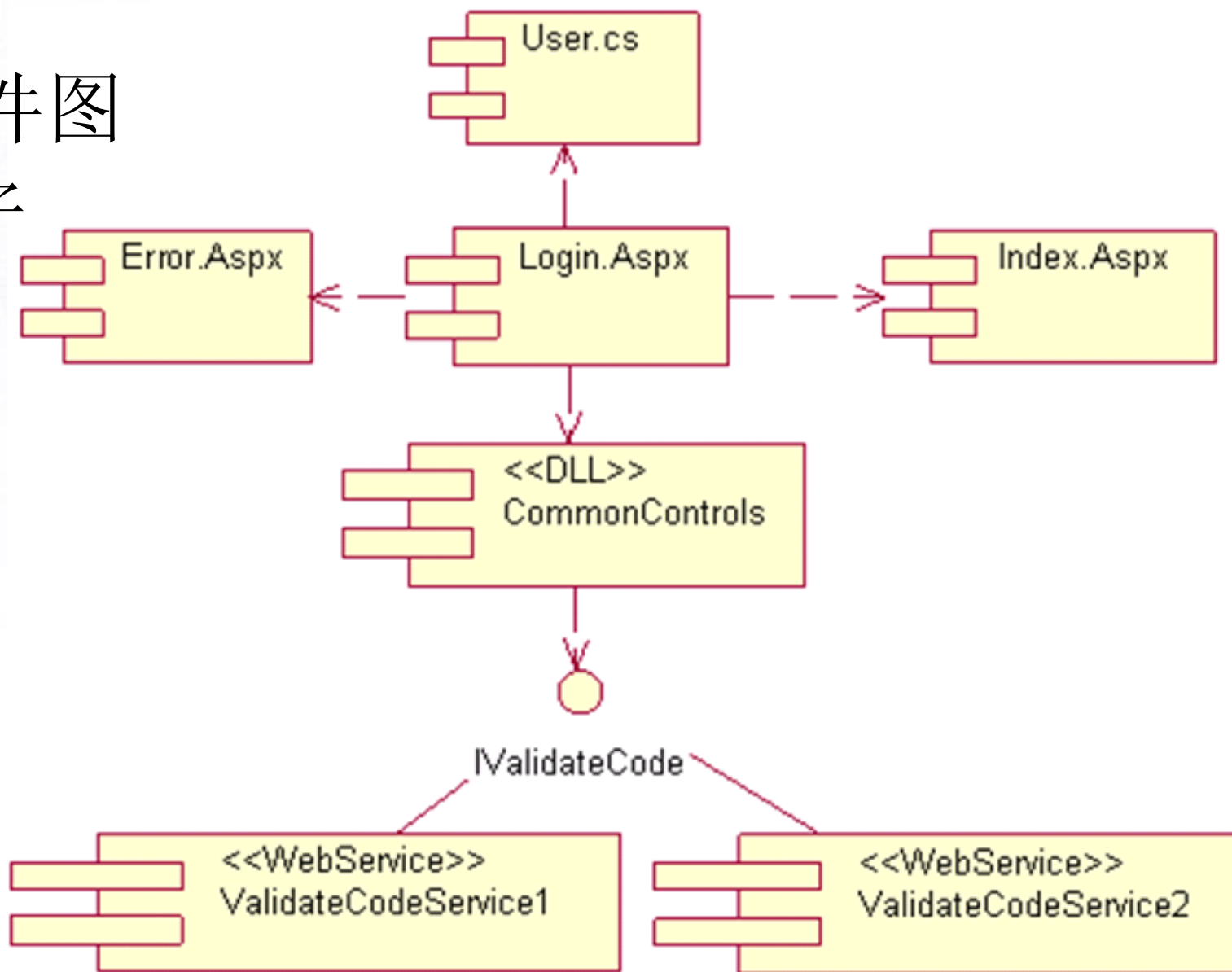
- 构件是系统中遵从一组接口且提供其实现的物理的、可替换的部分
- 输出接口(export interface)和引入接口(import interface)





6.6.2 OO的设计描述方法

- 3) 构件图
— 例子

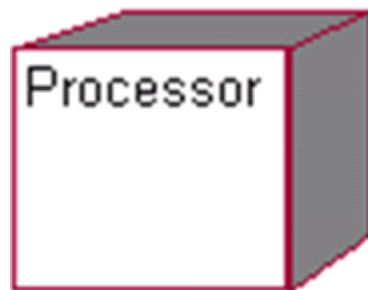




6.6.2 OO的设计描述方法

- 4)部署图

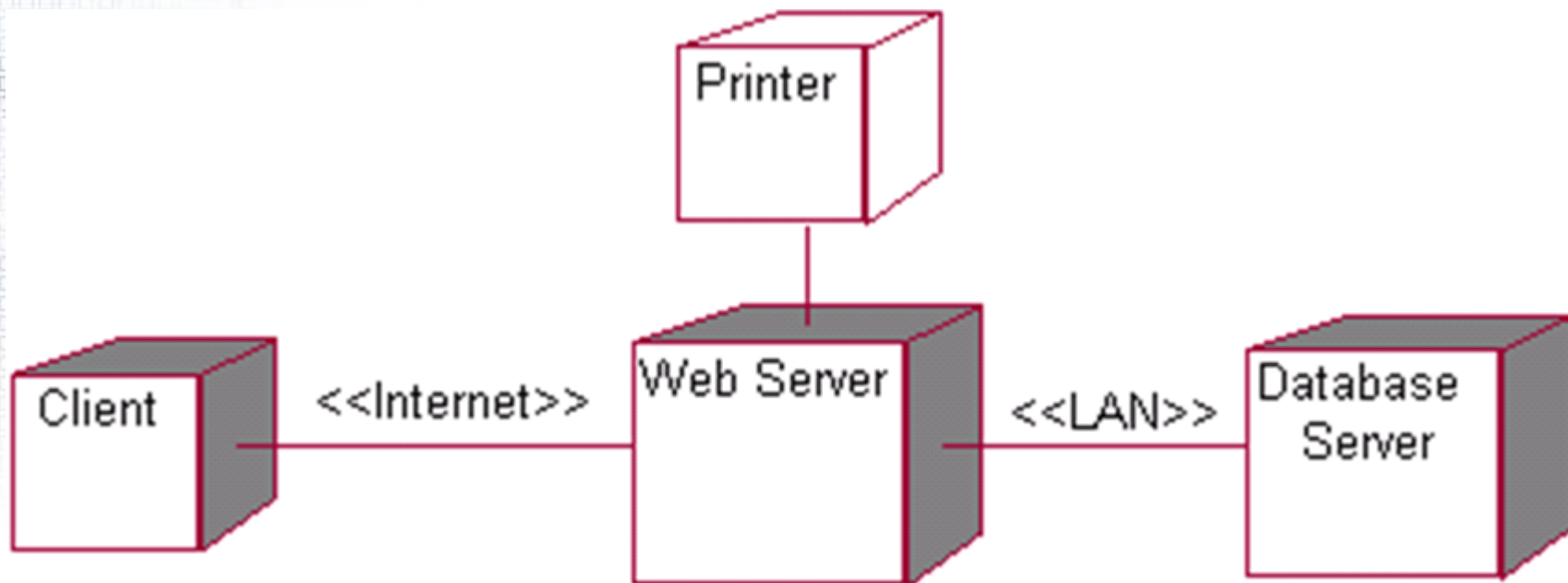
- 用来表示系统中的计算节点的拓扑结构和通信路径与节点上运行的软构件等
- 节点是存在于运行时的代表计算资源的物理元素，一般拥有处理能力和内存，如处理器或设备





6.6.2 OO的设计描述方法

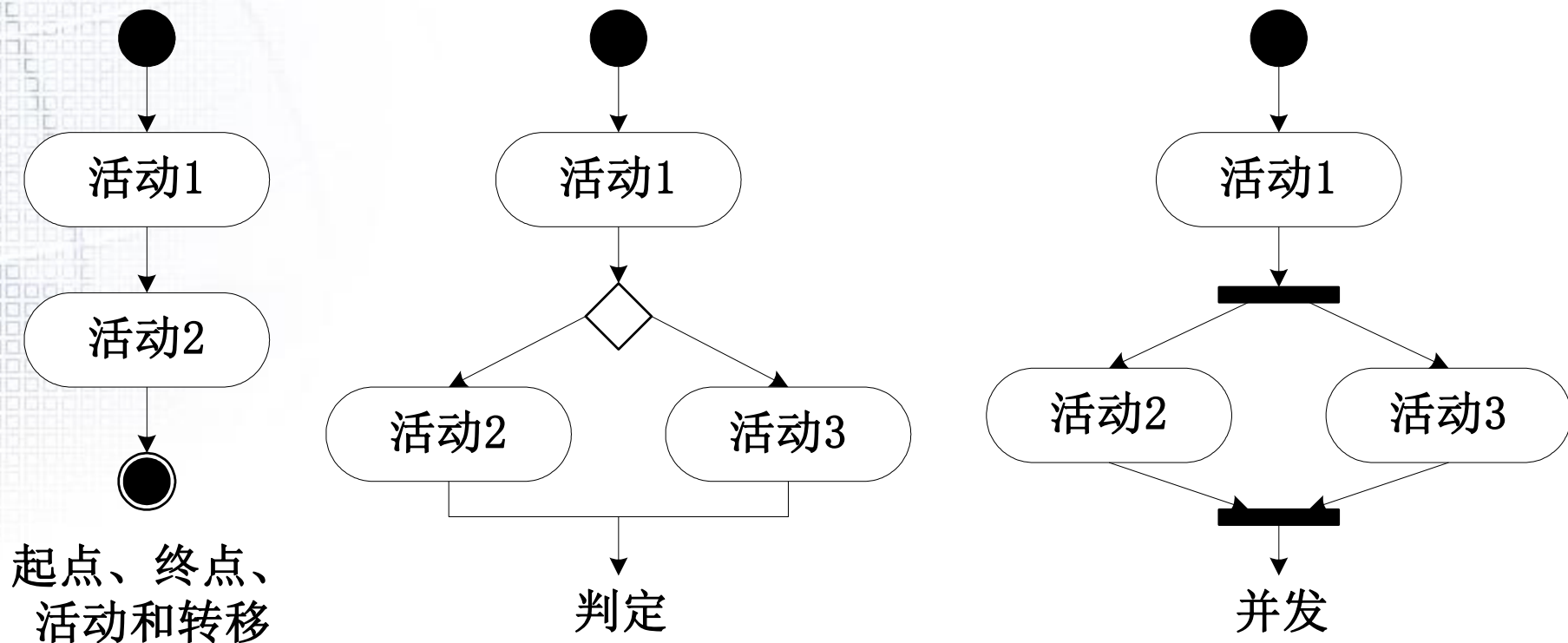
- 4)部署图
 - 例子





6.6.2 OO的设计描述方法

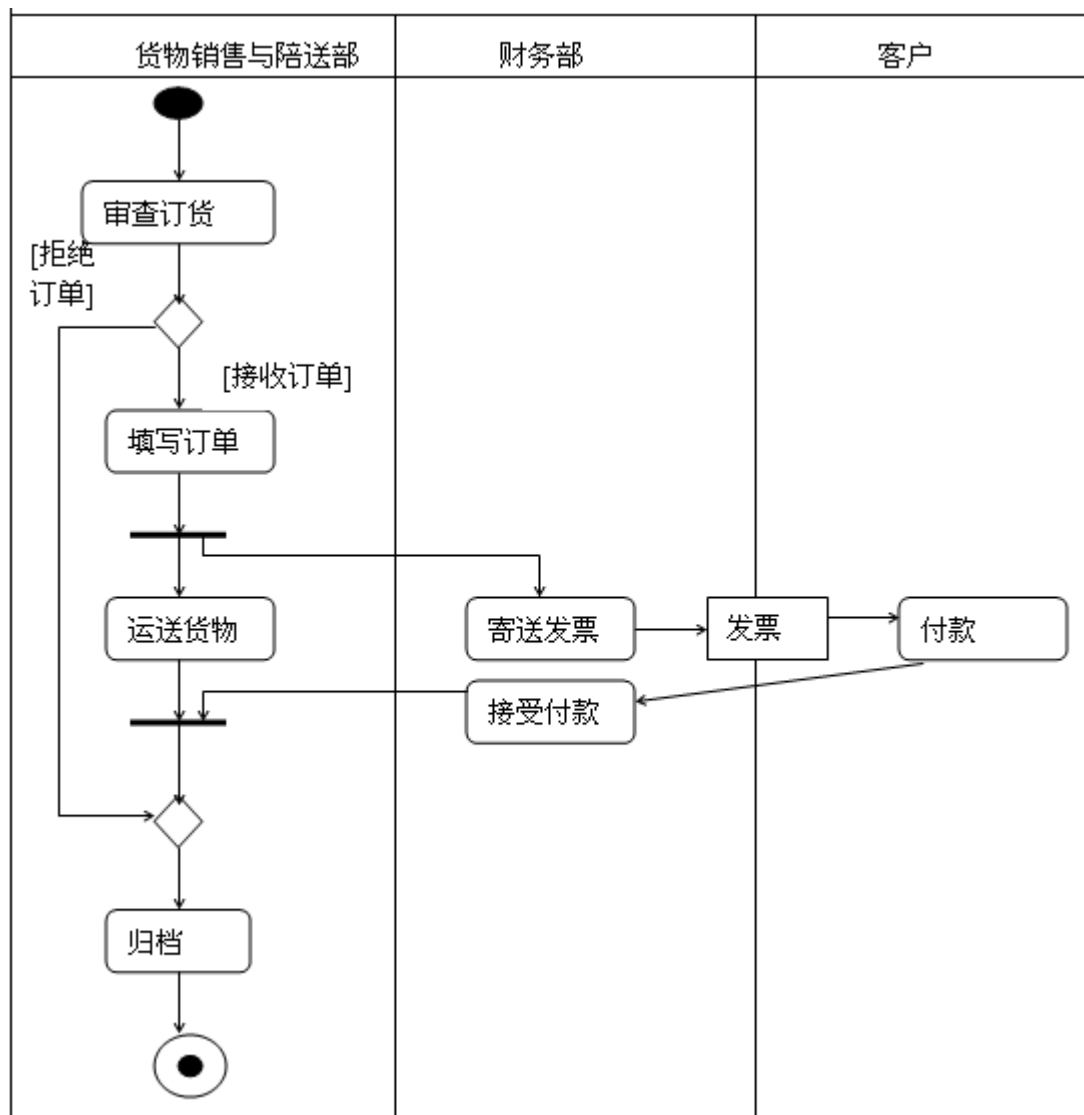
• 5)活动图





6.6.2 OO的设计描述方法

• 5)活动图(泳道)





6.7 人机交互界面设计

- 什么是人机交互（HCI）？
- 为什么要有HCI？
- HCI做什么？
 - 设计用户界面，即设计输入和输出。对每一次输入，开发者要考虑用户和计算机的交互，要设计界面来处理输入；同理，对用户的每一次输出（报表），开发者也要设计界面实现交互处理。又称对话设计。



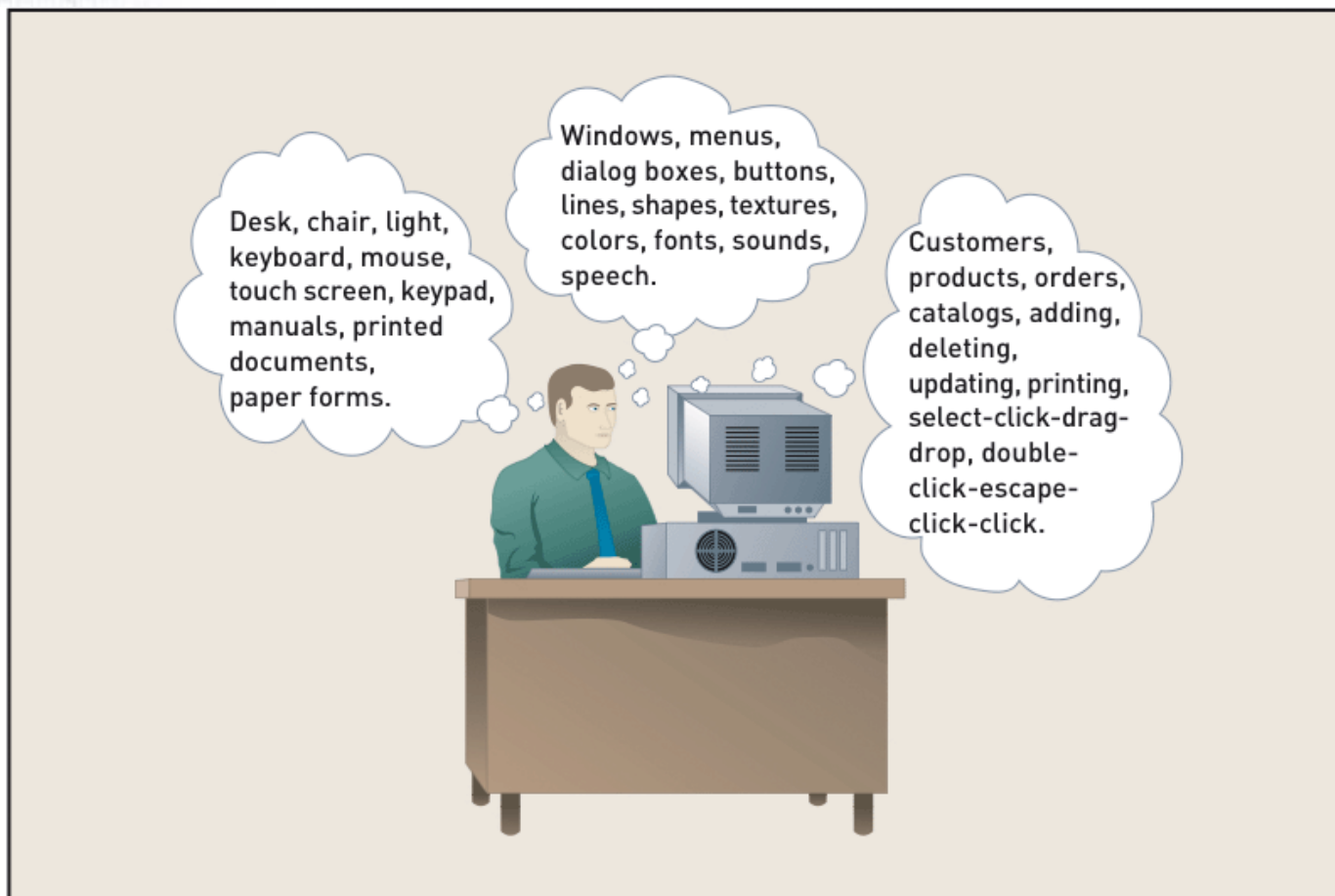
6.7 人机交互界面设计

- 输入和输出识别？
 - 早期阶段：
 - 需求分析阶段：
 - 传统：DFD中用数据流表示输入/出
 - OO：Actor与用例通过消息传递输入/出
- 系统界面/用户界面
 - 系统界面：较少的人工干预
 - 用户界面：需要用户干预



6.7 人机交互界面设计

- 用户使用系统时，所接触到的全部内容





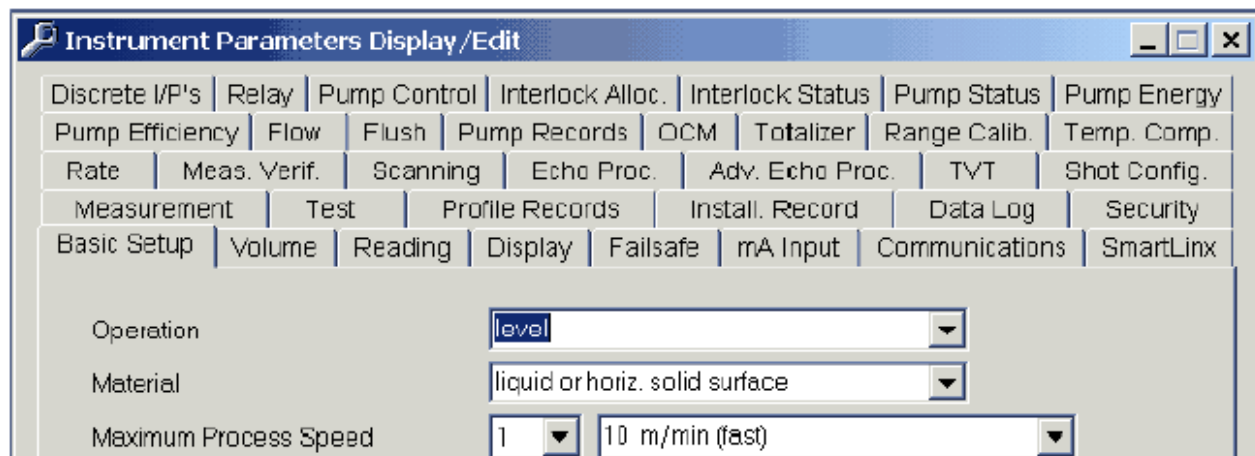
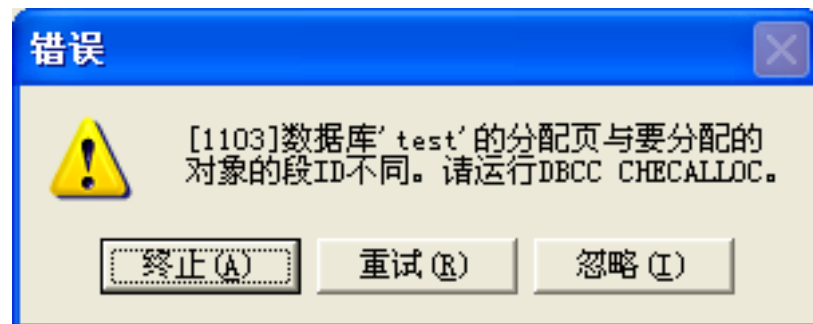
6.7 人机交互界面设计

- 大量的项目因为没有让用户充分参与界面的设计而失败
 - 市场部门的问题
 - 开发人员的问题
 - 用户的问题
 - 设计模型的问题



6.7 人机交互界面设计

- 一些典型的不良界面
 - 对用户的主观臆测
 - 不友好
 - 晦涩难懂
 - 行为不当
 - 界面复杂





6.7 人机交互界面设计

- 以用户为中心的设计(User-Centered Design,UCD)
 - 理解用户的特征
 - 理解用户的任务
 - 确保用户参与
 - 遵循良好的界面设计原则



6.7 人机交互界面设计

- 界面设计指导原则
 - 全部界面格式和风格应保持一致
 - 适当组织菜单层次和菜单项
 - 为不同的用户或功能提供不同的界面服务
 - 力求用户需要的输入量最少
 - 破坏性命令或功能选项应确认
 - 提供有效的系统保障能力
 - 布局合理
 - 简单易懂，整洁有序，条理清晰
 - 帮助功能
 - 一定的智能



6.7 人机交互界面设计

- 界面设计基础
 - 字符型界面和图形界面

```
命令提示符
2006-11-21 15:56 <DIR> WINNT
3 个文件 14,890 字节
8 个目录 11,196,870,656 可用字节

C:\>dir/p
驱动器 c 中的卷没有标签。
卷的序列号是 9CFA-8281

C:\ 的目录

2006-10-26 10:02 <DIR> ASFRoot
2006-10-26 10:56 25 cab.log
2006-10-26 10:38 <DIR> dell
2006-11-01 20:38 <DIR> Documents and Settings
2006-11-06 21:09 <DIR> download
2006-10-26 10:43 <DIR> dwtmp
2006-10-26 10:03 <DIR> Inetpub
2006-11-21 16:15 13,030 PDXUSRS.NET
2006-11-17 17:35 <DIR> Program Files
2006-11-08 16:42 1,035 UUSZ_MLog.txt
2006-11-21 15:56 <DIR> WINNT
3 个文件 14,890 字节
8 个目录 11,196,870,656 可用字节

C:\>
```





6.7 人机交互界面设计

- 界面设计基础

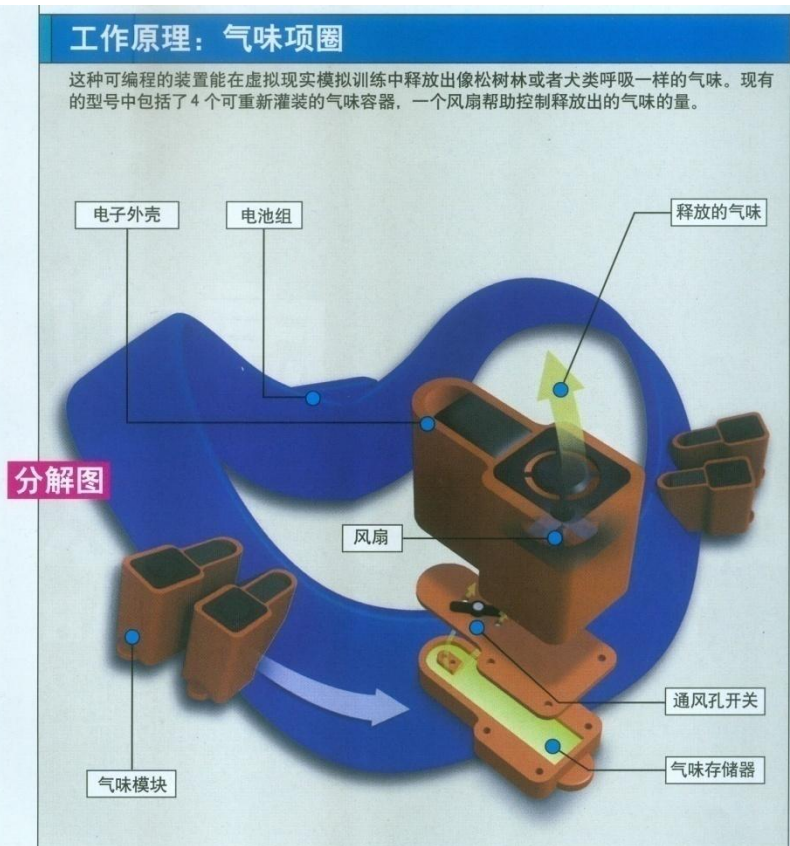
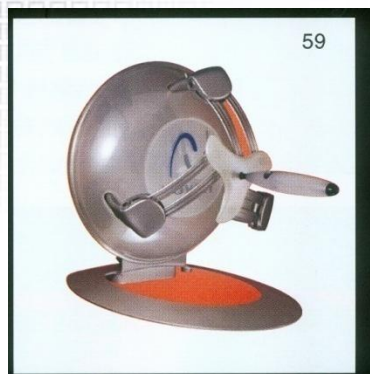
- 界面控件

- 菜单、命令控件、静态文本控件、文本控件、数据窗口控件、下拉列表框控件、单选按钮控件、复选框控件、树形结构控件、图形控件、微调框控件、滑动条控件.....
 - 自定义的控件



6.7 人机交互界面设计

- 多通道人机交互
 - 强调计算机输入信息的自然性和多样性
 - 语音交互
 - 虚拟现实高级交互





6.7 人机交互界面设计-supl1

- 信息输入/输出界面
 - I/O用户界面是系统重要的组成部分。
 - 界面设计的好坏直接影响用户使用系统的效率。
 - 特别信息量大时，考虑提高速度，减少出错率。
- 信息输入界面设计
- 信息输出界面设计



6.7 人机交互界面设计

- 信息输入界面设计

- 尽量减少工作量

- 相同内容输入设置默认值,
- 列表选择或单击选择, 不需输入
- 模拟方式输入
- 语音输入

- 有交互

- 确定输入、确定删除、提示信息
- 注意语气

Select a Date and Time

Year: < 2008 **2009** 2010 >

Month: Jan Feb Mar Apr **May** Jun Jul Aug Sep Oct Nov Dec

Day of Month:

Sun	Mon	Tue	Wed	Thu	Fri	Sat
					1	2
3	4	5	6	7	8	9
10	11	12	13	14	15	16
17	18	19	20	21	22	23
24	25	26	27	28	29	30
31						

Hour: 12am 12pm 1am 1pm 2am 2pm 3am 3pm 4am **4pm** 5am 5pm 6am 6pm 7am 7pm 8am 8pm 9am 9pm 10am 10pm 11am 11pm

Minute: 0 0 1 1 2 2 3 3 4 4 **5 5** 6 6 7 7 8 8 9 9

录音控制

选项(P) 帮助(H)

Mono Mix	Stereo Mix	线路输入	麦克风
平衡:	平衡:	平衡:	平衡:
音量:	音量:	音量:	音量:
☐ 选择(S)	☒ 选择(S)	☐ 选择(S)	☐ 选择(S)

高级(A)

Avance AC'97 Audio

成功添加到询价清单

确定

您确定要删除吗?

确定 取消

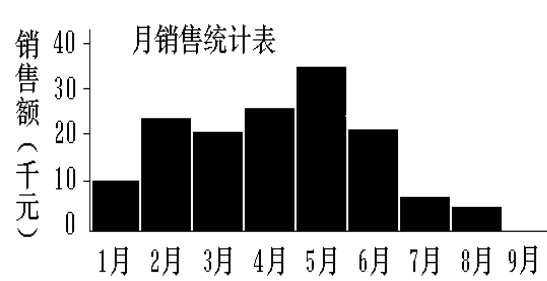


6.7 人机交互界面设计-sup13

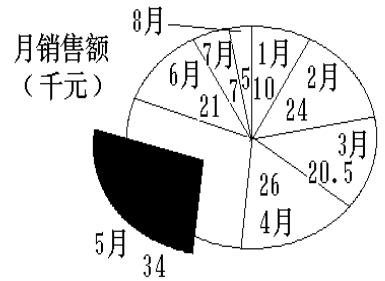
- 信息输出界面设计
— 信息的表示方法

- 文字：
 - 字体
 - 颜色背景
- 数值
 - 表格
 - 图形

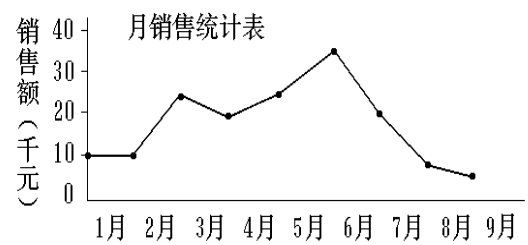
姓 名	数学成绩	英语成绩	物理成绩	化学成绩
卢晓波	91	83	88	94
浦柯玖	90	91	95	87
张超回	94	86	92	89



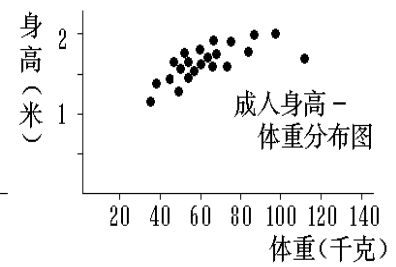
(a) 直方图



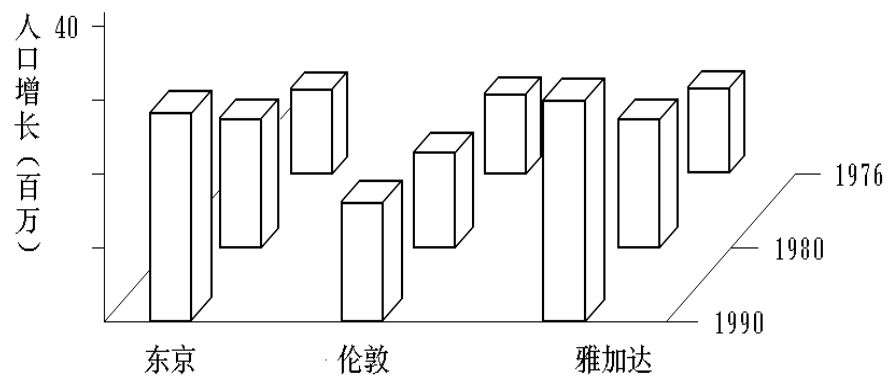
(b) 饼图



(c) 折线图

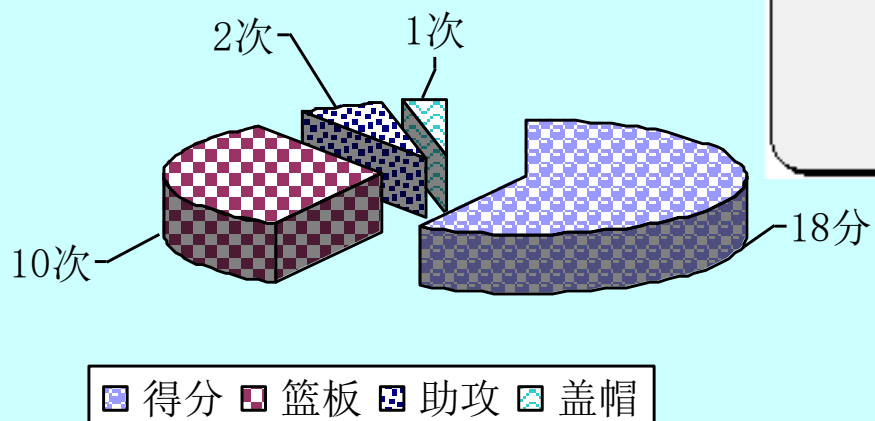


(d) 散布图

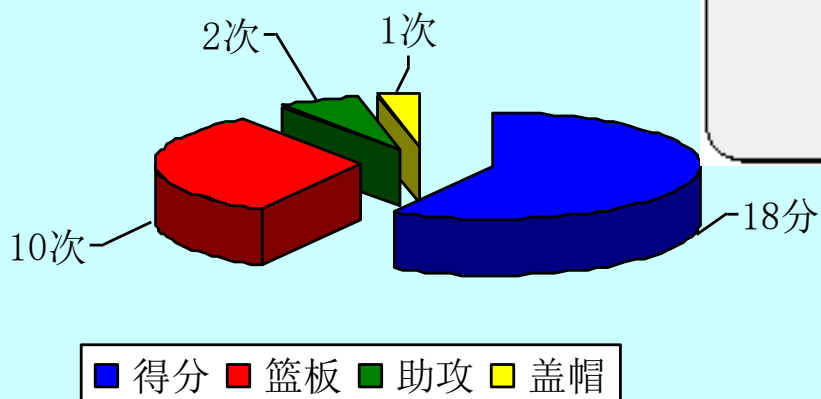




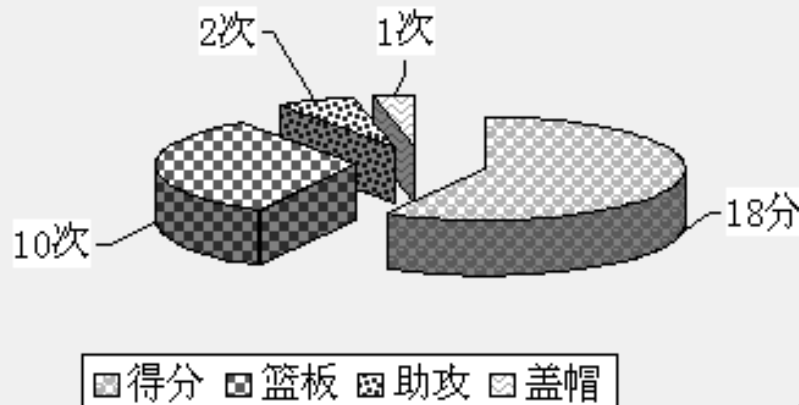
4月24日与洛杉矶湖人比赛技术统计



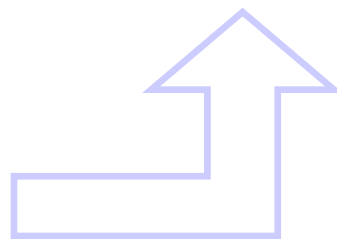
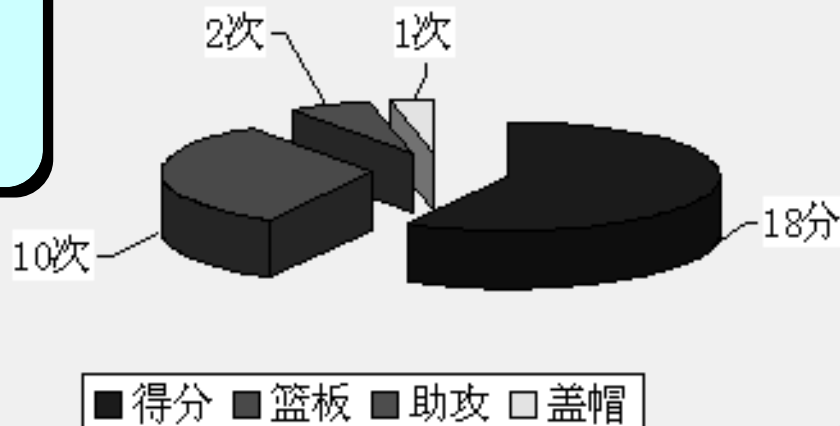
4月24日与洛杉矶湖人比赛技术统计



4月24日与洛杉矶湖人比赛技术统计



4月24日与洛杉矶湖人比赛技术统计



界面

姓名

工作单位

履历

2000-2001：在某某单位工作
2001-2002：在某某单位工作
2003-2004：在某某单位工作
2005-2006：在某某单位工作

确定

取消

是(Y)

否(N)

确认

?

是否真的要删除该人员的信息?

否(N)

是(Y)

界面

人员信息：

	Name	Age	City
▶	Wang	50	BeiJing
	Zhao	23	
	Ma	33	ShenYang
	Xu	23	ShangHai
	Liu		NingBo

确定

取消

界面

人员信息：

	Name	Age	City
▶	Wang	50	BeiJing
	Zhao	23	
	Ma	33	ShenYang
	Xu	23	ShangHai
	Liu		NingBo

确定

取消

界面

人员信息：

	Name	Age	City
▶	Wang	50	BeiJing
	Zhao	23	
	Ma	33	ShenYang
	Xu	23	ShangHai
	Liu		NingBo

确定

取消

Probl



6.8 网站设计指导原则

- 将机构名称和标志放在所有网页上，并建立标志与主页的链接。
- 网页超过100时，最好提供搜索功能。
- 书写简介的标题行和页面标题，有助于理解页面功能
- 构造页面的原则是便于使读者浏览。比如分组方式...
- 不要将某一主题或某一产品的所有内容拥挤地塞满单一页面，应使用超链接显示一个特定主题
- 利用链接标题为用户提供链接内容预览信息。
- 可以使用产品照片，但避免堆积过多照片，以免加载过慢
- 考虑不同用户，语种，残疾...
- 工作方式与主流保持一致。许多大型网站类似



6.9 相关文档规范

- 相关设计文档
 - 1) 软件工程实践者的研究方法
 - 2) RUP: 软件架构文档
 - 3) 国家标准: 软件设计书的编写内容
 - 概要设计说明书 (GB8567—88)
 - 详细设计说明书 (GB8567—88)
 - 数据库设计说明书 (GB8567—88)
 - 4) 企业标准